

无线电干扰源的快速自动定位与清除

摘要

针对复杂环境下无线电干扰源人工排查效率低、作业风险高的问题,本文研究搭载测向机、光学探测仪和激光枪的机器狗对干扰源进行自动搜索、定位与清除的方法。综合考虑示向度的有界误差、未知信号接收半径、定向覆盖盲区以及移动、检测和清除等时间成本,建立由**集合定位、鲁棒选点、滚动调度和概率补盲**构成的一体化模型。

针对问题一,将示向度误差转化为以检测点为顶点的**方向扇形约束**,并与目标区域及最大接收圆盘求交,得到干扰源的**紧凸可行域**。通过枚举直线与圆弧边界交点构造区域边界,利用支持函数计算定位区域直径,并采用**最小外接圆**刻画区域的实际覆盖能力。由此得到判定结论:以定位区域直径为直径的圆能够覆盖整个定位区域,当且仅当该区域的最小外接圆半径不超过其直径的一半;同时以最小外接圆圆心作为后续光学定位的目标点。

针对问题二,在第一次测向形成的不确定区域上建立局部坐标系,将第二检测点表示为轴向距离与横向距离的组合。综合**最小接收半径和最坏交会角**,推导第二检测点的**鲁棒候选区域**,并得到关于首次示向中心线对称的解析候选点。进一步以真实位置和测向误差均取最不利情形时的定位区域直径为目标函数,建立**极小化最坏定位直径的鲁棒优化模型**;结果表明,解析候选点与最坏定位直径的低值区域基本一致,且最优区域附近目标函数变化平缓,说明该选点策略能够有效提高交会定位精度并具有良好的鲁棒性。

针对问题三,按照最小有效接收半径设计由中心点和半径为 1150 m 的正六边形外围点构成的**七点搜索骨架**,实现对目标圆域的最坏情形覆盖。在此基础上,设置搜索中、定位中、可清除和已清除等频道状态,利用有信号、无信号、近距离及清除反馈动态更新可行域,并通过**未来基准点复用、主动检测点选择和最小增量插入**,将基础搜索、主动定位与清除任务统一纳入滚动路线规划。三次正式测试分别清除 12、10 和 12 个干扰源,平均定位清除时间分别为 345.60 s、437.30 s 和 328.44 s,程序运行时间均不超过 18.00 s,表明算法具有较好的执行效率和计算稳定性。

针对问题四,考虑定向干扰源可能因方向遮蔽产生无信号观测,构建包含干扰源位置、接收半径、类型和发射方向的**粒子后验模型**,并利用贝叶斯更新融合正、负观测信息。进一步引入**频道存在概率、累计方向覆盖率和源数量修正**,建立兼顾补盲收益、方位公平性与路线增量的**多频道联合价值函数**,并据此在七点骨架搜索基础上动态触发**主动补盲与外环搜索**,实现搜索范围与定位需求的自适应调整。三次正式测试分别清除 10、14 和 10 个干扰源,平均定位清除时间分别为 589.43 s、265.81 s 和 362.81 s,验证了所提模型对全向-定向混合干扰源场景的适应能力。

综上,本文模型实现了从**单源几何定位、鲁棒检测点选择到多源在线调度和定向盲区补偿**的逐层拓展,在有界测向误差和信息不完全条件下具有较强的可解释性、鲁棒性与在线可实施性,可为无人化无线电干扰排查系统的设计提供参考。

关键词: 无线电测向 集合定位 鲁棒优化 滚动路径规划 粒子后验

1 问题重述

1.1 问题背景

无线电干扰源的快速定位与清除是无线电频谱管理中的重要任务。传统定位通常需要技术人员携带测向设备进入目标区域，根据干扰信号的场强变化和方向特征逐步确定干扰源位置。在危险或复杂环境中，人工排查存在作业风险较高、定位效率较低等问题。

为提高定位与清除效率，某公司拟研发一套搭载于机器狗的自动作业系统。机器狗通过无线电测向机获取干扰源的示向度，在接近干扰源后利用光学探测仪完成精确定位，并通过激光枪将其清除。

目标区域为半径 1800 m 的圆形区域，区域内干扰源的数量、位置及部分信号特征未知。由于测向结果存在误差，且机器狗的移动、频道切换、信号检测和清除操作均会产生时间成本，因此需要建立相应的定位模型与搜索策略，使机器狗在确保清除全部干扰源的前提下尽可能缩短任务完成时间。

1.2 问题提出

根据题目给出的干扰源特征、测向机工作原理及机器狗运行条件，需要建立数学模型解决以下四个问题。

1.2.1 问题一：多检测点交会定位

已知若干检测点的坐标以及同一干扰源在各检测点处的示向度，考虑示向度存在不超过 1° 的测量误差，利用各检测点形成的方向约束确定干扰源的多边形定位区域。建立该定位区域的几何模型，设计计算区域内任意两点最大距离，即定位区域直径的算法，并判断以该直径为直径构造的圆能否覆盖整个定位区域。

1.2.2 问题二：第二检测点的选择

已知某全向干扰源在第一个检测点处的示向度，研究第二检测点的位置对交会定位结果的影响。建立衡量定位效果的评价准则，在满足目标区域范围和信号有效接收条件的基础上，给出第二检测点的选择策略，并确定其候选区域。

1.2.3 问题三：全向干扰源的自动定位与清除

已知目标区域内分布有 10 ~ 16 个全向干扰源，但干扰源的具体数量、频道、位置及有效接收半径均未知。机器狗从坐标原点出发，测向机初始频道为 1。综合考虑移动、频道切换、信号检测、光学精确定位及激光清除等过程的时间成本，建立机器狗的自主搜索、定位与清除模型，设计能够确保全部干扰源被清除且任务完成时间尽可能短的搜索策略与实现算法。

利用模拟器进行演练测试，计算被清除干扰源个数、被清除干扰源个数比例和平均定位清除时间等指标，以检验所设计算法的有效性。完成充分演练后进行三次正式测试，记录各次测试的案例编码、清除干扰源个数、平均定位清除时间和程序运行时间，并导出相应的测试日志。

1.2.4 问题四：混合干扰源的自动定位与清除

在问题三的基础上，进一步考虑目标区域内同时存在全向干扰源和定向干扰源的情形。干扰源总数仍为 10 ~ 16 个，但两类干扰源的具体数量、位置及定向干扰源的发射方向均未知。针对定向干扰源存在有限信号覆盖范围和检测盲区的特点，改进问题三中的搜索、检测、定位与清除模型，设计适用于混合干扰源环境的自动化策略与算法，并通过演练测试和三次正式测试评价算法的有效性。

2 问题分析

四个问题按照定位区域刻画、检测位置优化、多源任务调度和定向盲区补偿逐层推进。问题一将测向误差转化为可计算的定位区域，并建立后续精确探测的判据；问题二在保证信号接收的同时改善观测几何，降低定位不确定性；问题三面向数量、位置和频道未知的多源场景，在搜索、追加测向和清除之间进行动态调度；问题四考虑定向发射造成的无信号歧义，通过保留位置与方向不确定性并增加多方位观测来降低漏检风险。四问由此形成由单源定位到多源调度、由确定性搜索到不确定性补偿的递进关系。

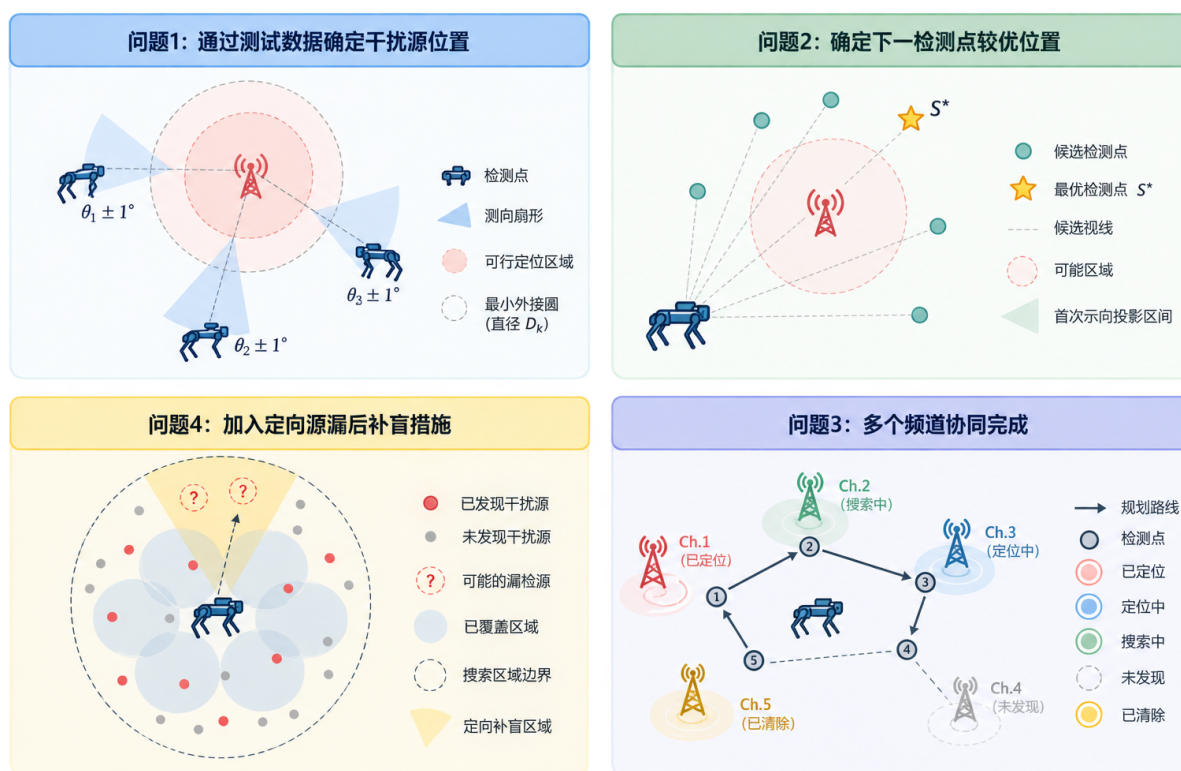


图 1 四个问题的递进关系与关键决策

3 模型假设

1. 忽略高度差异，将目标区域视为半径 1800 m 的二维圆域；机器狗沿直线以 5 m/s 匀速行进，移动时不进行检测。

2. 每个干扰源占用唯一且固定的频道，位置和有效接收半径在一次测试中保持不变，且 $1000\text{ m} \leq R_G \leq 1500\text{ m}$ ；全向源覆盖 360° ，定向源覆盖定向方向两侧各 90° 。
3. 示向度误差为未知但有界量，满足 $[-1^\circ, 1^\circ]$ ；同一地点重复检测采用相同误差。全向源定位区域由方向扇形、接收范围和目标圆域求交得到，定向源无信号时采用概率更新。
4. 当检测点距干扰源不超过 5 m 且位于有效覆盖角内时，视为获得 **near** 状态；清除点距干扰源不超过 20 m 时，光学定位和激光清除成功。
5. 频道切换、检测、光学定位和清除时间分别按 1 s 、 5 s 、 3 s 和 2 s 计，并忽略其他附加时间；候选检测点采用离散搜索，定向源后验采用粒子集合近似。

4 符号说明

表 1 统一符号说明

符号	含义
O	目标区域中心，即坐标原点
$\mathcal{D}$	半径为 1800 m 的目标区域
S_i	机器狗第 i 个检测点
G	干扰源真实位置
α_i	第 i 次测得的示向度
δ	测向误差上界， $\delta = 1^\circ$
R_G	干扰源有效接收半径
$\mathcal{W}_i$	第 i 次测向对应的方向约束区域
Ω	干扰源位置可行域
$D(\Omega)$	定位区域直径
C_Ω	定位区域最小外接圆圆心
r_Ω	定位区域最小外接圆半径
Z	干扰源类型，0 表示全向，1 表示定向
ϕ	定向干扰源发射方向
v	机器狗移动速度， $v = 5\text{ m/s}$
T_{total}	完成任务所需的总时间

5 模型准备

5.1 坐标系与目标区域

根据题意，文本的所有求解都是建立在平面直角坐标系的基础上进行，其中以目标区域中心为坐标原点 O ，正东方向为 x 轴正方向，正北方向为 y 轴正方向。目标区域为半径 $R_T = 1800\text{ m}$ 的圆形区域，记为 $\mathcal{D} = \{X = (x, y)^\top \in \mathbb{R}^2 : \|X\|_2 \leq R_T\}$ 。

5.2 方向角与测向误差模型

一次测向只能确定干扰源所在的大致方向，而无法直接确定其具体位置。考虑测向设备存在最大为 $\pm 1^\circ$ 的测向误差，每次观测均可表示为一个以检测点 S_i 为顶点、以测得方向 α_i 为中心方向、张角为 2δ 的方向约束扇形。干扰源真实位置必须位于该扇形区域 $\mathcal{W}_i$ 内，如图 2 所示。随着来自不同检测点的方向约束不断叠加，定位区域逐步收缩，为后续精确定位提供几何基础。

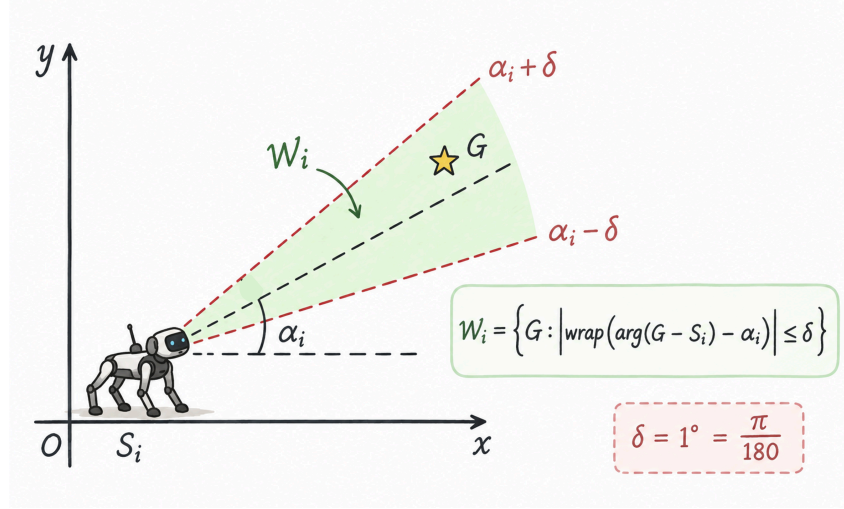


图 2 单次测向形成的方向约束区域示意图

5.3 干扰源接收范围模型

设干扰源的有效接收半径为 R_G ，满足 $1000 \text{ m} \leq R_G \leq 1500 \text{ m}$ 。由于真实接收半径未知，为保证定位区域能够覆盖真实干扰源，本文对所有成功接收到信号的检测点均采用最大接收半径约束，即认为干扰源位于检测点半径 1500 m 的圆域内。

对于全向干扰源，只要机器狗位于其有效接收范围内即可接收到信号，因此第 i 次测向对应的可行区域可表示为

$$\Omega_i = \mathcal{D} \cap \mathcal{W}_i \cap \{G : \|G - S_i\|_2 \leq 1500\}, \quad (1)$$

多次测向后，最终定位区域为各次可行区域的交集

$$\Omega^{(k)} = \bigcap_{i=1}^k \Omega_i. \quad (2)$$

对于定向干扰源，除满足上述距离约束外，还要求机器狗位于干扰源的发射方向覆盖范围内。设发射方向单位向量为 $u(\phi)$ ，则接收到信号还需满足

$$(S_i - G)^T u(\phi) \geq 0. \quad (3)$$

因此，问题一至问题三采用全向干扰源模型，而问题四在此基础上进一步加入定向覆盖约束，以描述定向干扰源的方向性发射特征。

5.4 无信号观测模型

无信号观测同样能够提供定位信息。对于全向干扰源，当机器狗在某检测点未接收到指定频道信号时，可排除以该检测点为圆心、最小接收半径 1000 m 内的区域，从而进一步缩小干扰源可能存在的范围。

对于定向干扰源，无信号既可能来源于目标超出接收距离，也可能由于机器狗位于发射方向覆盖范围之外。因此，问题四中不再直接利用无信号结果排除整个区域，而是结合干扰源类型及历史观测信息进行综合判断，以避免误删真实可行区域。

5.5 定位与清除判据

随着检测点不断增加，定位区域逐渐收缩。当定位区域足够小时，即认为干扰源位置已经能够满足光学定位要求。本文采用定位区域最小外接圆作为定位结果，并以其圆心作为机器人执行光学定位和激光清除的目标位置。

设最小外接圆半径为 r_Ω ，当满足

$$r_\Omega \leq 20 \text{ m}, \quad (4)$$

则认为该干扰源达到清除条件，并进入清除任务。

5.6 任务状态与时间模型

为了统一描述各频道的处理过程，本文将每个频道划分为四种状态：搜索（SEARCHING）、定位（LOCALIZING）、可清除（CLEARABLE）和已清除（CLEARED）。其中，搜索表示尚未发现该频道；定位表示已接收到信号但位置仍存在不确定性；可清除表示定位精度已满足要求；已清除表示对应干扰源已经完成清除。各频道均按照上述状态依次演化，并由机器人根据当前状态动态安排检测、定位和清除任务。

机器狗以恒定速度 5 m/s 运动，同时需要完成频道切换、信号检测、光学定位及激光清除等操作。因此，总任务时间由移动时间以及各类操作时间共同组成，可表示为

$$T_{\text{total}} = \frac{L}{5} + n_s + 5n_d + 3n_o + 2n_c, \quad (5)$$

其中，各项分别表示移动、频道切换、信号检测、光学定位和激光清除所消耗的时间。

6 问题一模型的建立与求解

6.1 问题一模型建立

根据公共模型，当机器狗在检测点成功接收到某频道信号时，干扰源位置同时受到测向角误差约束和最大接收距离约束。因此，单次测向可将干扰源限制在一个截断扇形区域内，其可行域表示为

$$W_i = \left\{ X : \begin{array}{l} \text{cross}(u(\alpha_i - \delta), X - S_i) \geq 0, \\ \text{cross}(u(\alpha_i + \delta), X - S_i) \leq 0, \\ |X - S_i| \leq 1500. \end{array} \right\}. \quad (6)$$

从几何上看，该区域由两条测向边界和最大接收距离共同围成，能够充分利用一次测向获得的信息，对干扰源位置形成初步约束。

随着检测点不断增加，干扰源位置必须同时满足所有测向约束，因此最终定位区域可表示为各次可行域的交集：

$$P = \Omega \cap \bigcap_{i=1}^m W_i. \quad (7)$$

由于目标区域、测向约束和接收距离约束均为凸集，因此定位区域 P 仍保持凸性，为后续几何计算提供了良好的数学性质。

为了评价定位精度，本文采用定位区域的最大直径作为误差指标，即

$$D(P) = \max_{X, Y \in P} |X - Y|. \quad (8)$$

定位区域直径越小，说明干扰源位置的不确定性越低。当需要判断题目所要求的“直径为 $D(P)$ 的圆是否能够覆盖整个定位区域”时，只需计算定位区域的最小外接圆，并检验其半径是否满足

$$R_{\text{MEC}}(P) \leq \frac{D(P)}{2}. \quad (9)$$

若上述条件成立，则说明以定位区域直径为直径构造的圆能够覆盖整个定位区域；否则，需要采用更大的圆才能实现完全覆盖。最小外接圆同时也是后续判断是否满足光学定位与清除条件的重要几何依据。

根据检测点坐标及测得的示向度，可将干扰源位置约束为以检测点为顶点、以测向误差为角宽的方向扇形。多个检测点对应的方向扇形相交后，得到干扰源的可行定位区域。

6.2 问题一求解算法

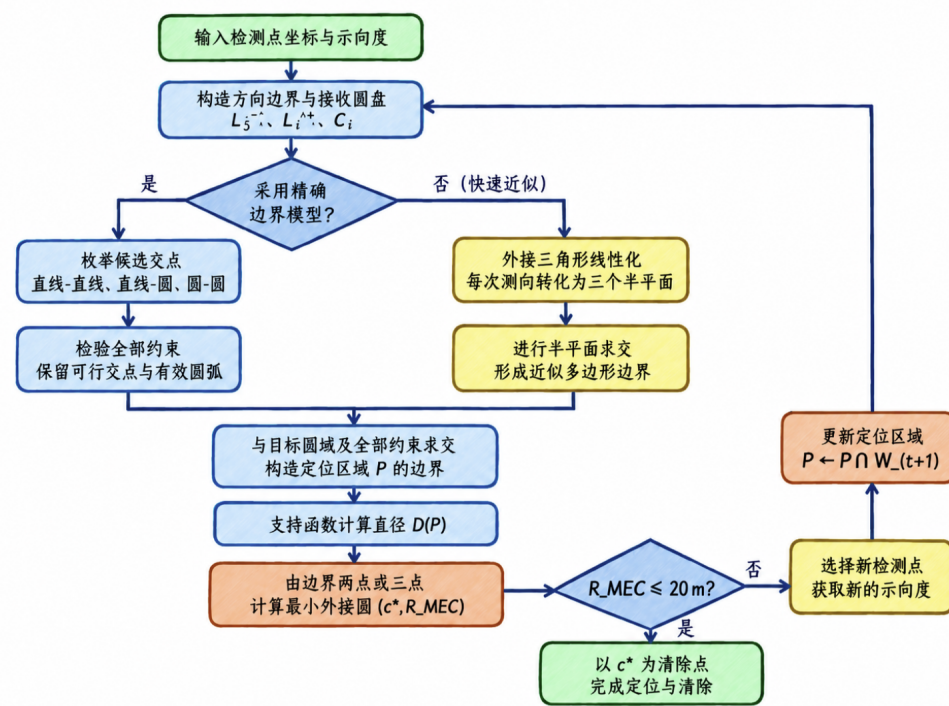


图 3 求解流程图

根据测向误差模型，单次测量结果对应一个具有角度不确定性的方向区域。结合接收半径限制和目标区域边界，可逐步构造干扰源可能存在的定位区域，具体求解流程如下。

Step 1: 构造定位区域边界

对于每个检测点，根据测向结果建立方向约束，并结合有效接收范围构造约束区域。将所有检测点对应的方向约束、接收圆盘约束以及目标区域约束进行求交，得到干扰源定位区域 P 。

为兼顾计算精度与效率，本文设置精确模型和快速近似模型两种求解方式。在精确模型下，枚举方向边界、接收圆以及目标边界之间的交点，并通过约束检验筛选有效边界点，同时保留满足条件的圆弧部分，最终由线段和圆弧共同构成定位区域边界。

在快速近似模型下，将每次测向约束转化为三个半平面，通过半平面求交获得定位区域的近似多边形边界。

Step 2: 计算定位精度

在获得定位区域 P 后，利用支持函数方法计算区域最大直径，并根据区域边界点计算最小外接圆半径 $R_{MEC}(P)$ ，用于衡量当前定位精度。

Step 3: 判断定位结果并迭代更新

若满足

$$R_{MEC}(P) \leq 20 \text{ m},$$

则认为定位区域满足清除要求，并将最小外接圆圆心作为干扰源清除位置。

否则，选择新的检测点获取额外测向信息，并利用新增约束更新定位区域：

$$P^{(t+1)} = P^{(t)} \cap W_{t+1}.$$

重复上述过程，直到定位区域满足清除精度要求，最终形成“测向约束构造—区域收缩—精确定位—清除”的闭环求解流程。

6.3 结果分析

为了直观展示定位区域的形成过程，图 4 给出了两个检测点交会定位的几何示意图。

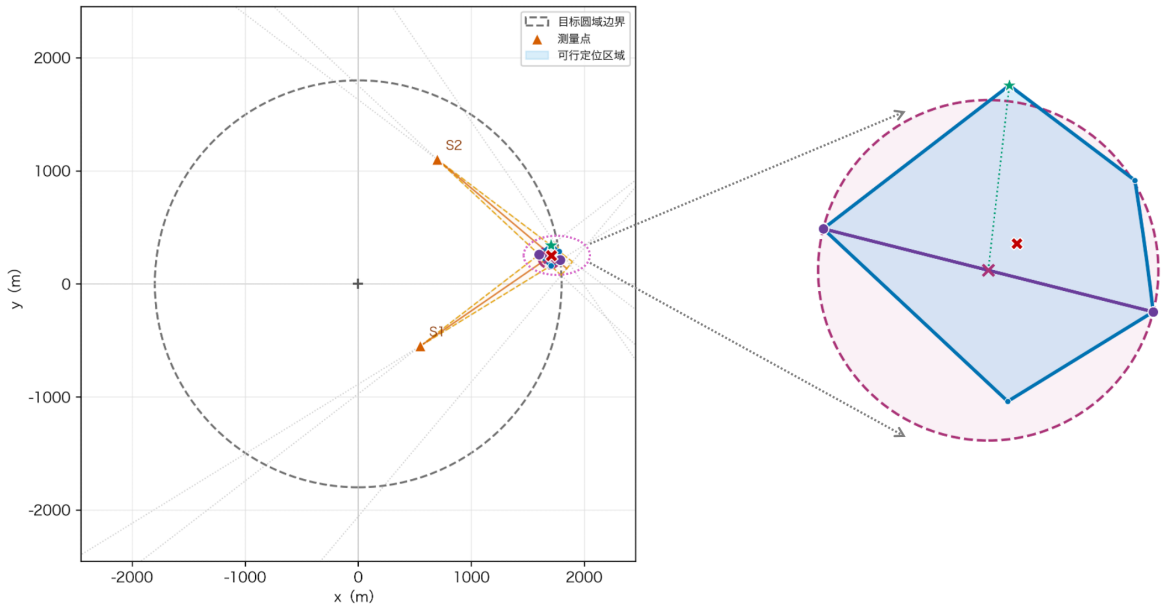


图 4 两检测点交会定位及可行区域示意图

由图可见，单个检测点仅能将干扰源约束在一个具有测向误差角宽的截断扇形内，此时可行区域范围较大，定位结果存在较高的不确定性。当引入第二个检测点后，两次测向形成的扇形区域发生交会，仅保留同时满足两次观测约束的公共区域，因此定位区域显著收缩，干扰源的位置得到进一步确定。

进一步观察局部放大图可以发现，可行区域由测向边界和接收距离边界共同围成，并始终保持凸集特性。这一性质保证了定位区域直径、最小外接圆等几何量均能够通过成熟的凸几何算法稳定计算，为后续定位精度评价提供了可靠基础。

7 问题二模型的建立与求解

7.1 问题二模型建立

7.1.1 第一次测向后的不确定区域

问题一已经建立了单次测向后的几何定位模型。当机器狗在检测点 S_1 首次接收到目标信号后，根据测向角误差和最大接收距离约束，可得到干扰源的初始可行区域

$$\Omega_1 = \mathcal{D} \cap \mathcal{B}(S_1, 1500) \cap \mathcal{W}(S_1, \theta_1), \quad (10)$$

其中 $\mathcal{D}$ 为目标区域， $\mathcal{B}$ 为接收距离约束， $\mathcal{W}$ 为测向形成的扇形约束。由于后续所有目标位置均位于 Ω_1 内，因此第二检测点的设计应充分考虑该区域内所有可能目标的位置，以提高定位精度。

7.1.2 第二检测点候选区域

建立以第一次示向方向为基准的局部坐标系，将第二检测点表示为

$$S_2 = S_1 + a\mathbf{u} + b\mathbf{v}, \quad (11)$$

其中 $\mathbf{u}$ 为第一次测向方向单位向量， $\mathbf{v}$ 为其法向单位向量。设第一次测向形成的不确定区间中心为 m ，半宽为 h 。

为了保证第二检测点能够对所有可能目标完成再次测向，需要同时满足鲁棒接收约束和最小交会角约束，二者共同确定第二检测点的可行区域：

$$\mathcal{C}_{\text{cand}} = \left\{ S_2 : \begin{array}{l} (h + |a - m|)^2 + b^2 \leq 1000^2, \\ |b| \geq (h + |a - m|) \tan \varphi_0 \end{array} \right\}. \quad (12)$$

图 5 给出了鲁棒接收约束与交会角约束共同形成的第二检测点候选区域。可以看出，可行区域关于第一次示向中心线对称，并呈现明显的“双翼”结构。其中，外边界由鲁棒接收约束决定，保证所有可能目标均能够再次被检测；内边界由最小交会角约束决定，用于避免两次测向几乎平行而导致定位误差放大。

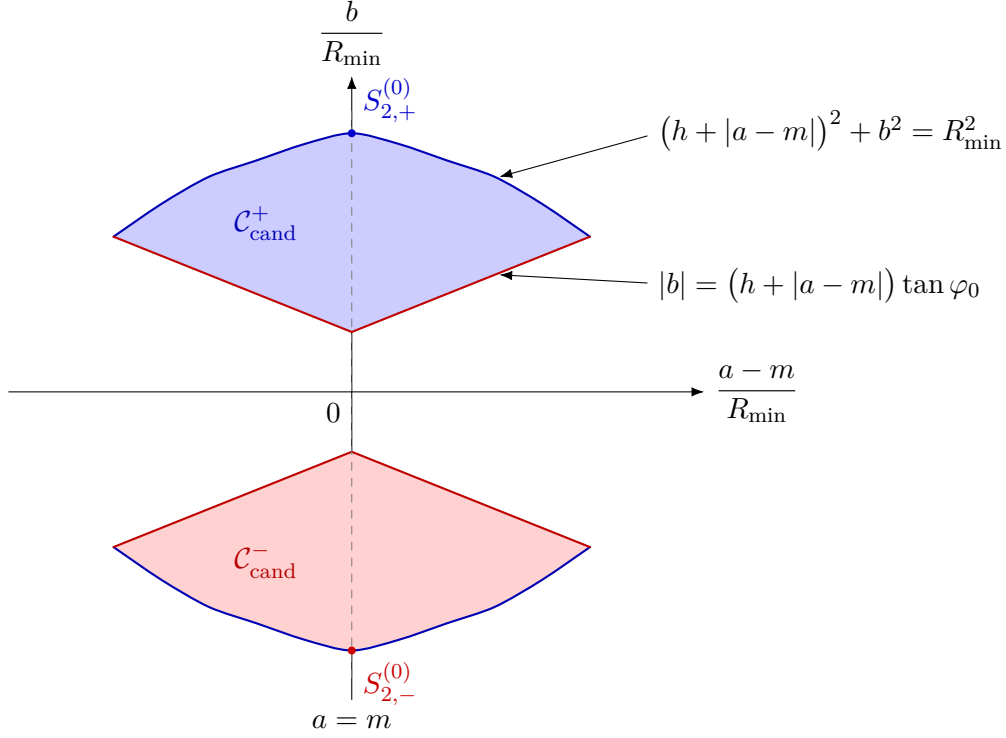


图 5 鲁棒接收约束与最小交会角约束共同确定的第二检测点候选区域

7.1.3 第二检测点优化模型

解析初值能够保证鲁棒接收并获得较大的交会角，但实际定位效果还受到目标真实位置、测向误差以及区域边界等因素的共同影响。因此，本文以问题一中定义的定位区域直径作为评价指标，采用最坏情况准则构造目标函数

$$J(S_2) = \max_{G \in \Omega_1} D(\Omega_{12}(S_2, G)), \quad (13)$$

其中， Ω_{12} 表示两次测向后的定位区域， $D(\cdot)$ 为问题一一定义的定位区域直径。

最终，第二检测点的求解可表示为

$$S_2^* = \arg \min_{S_2 \in \mathcal{C}_{\text{cand}}} J(S_2). \quad (14)$$

该模型以最终定位精度为优化目标，在满足再次接收和有效交会的基础上，选择能够使定位区域最小的第二检测点，为后续在线搜索和多次测向提供最优初始布局。

7.2 问题二求解算法

7.2.1 候选区域搜索

根据上一节建立的解析模型，第二检测点必须同时满足鲁棒接收约束和最小交会角约束，因此其搜索范围被限制在理论候选区域 $\mathcal{C}_{\text{cand}}$ 内。与直接在整个目标区域进行搜索相比，解析候选区域能够提前剔除大量不满足约束的检测点，从而显著缩小搜索空间，提高后续数值优化效率。

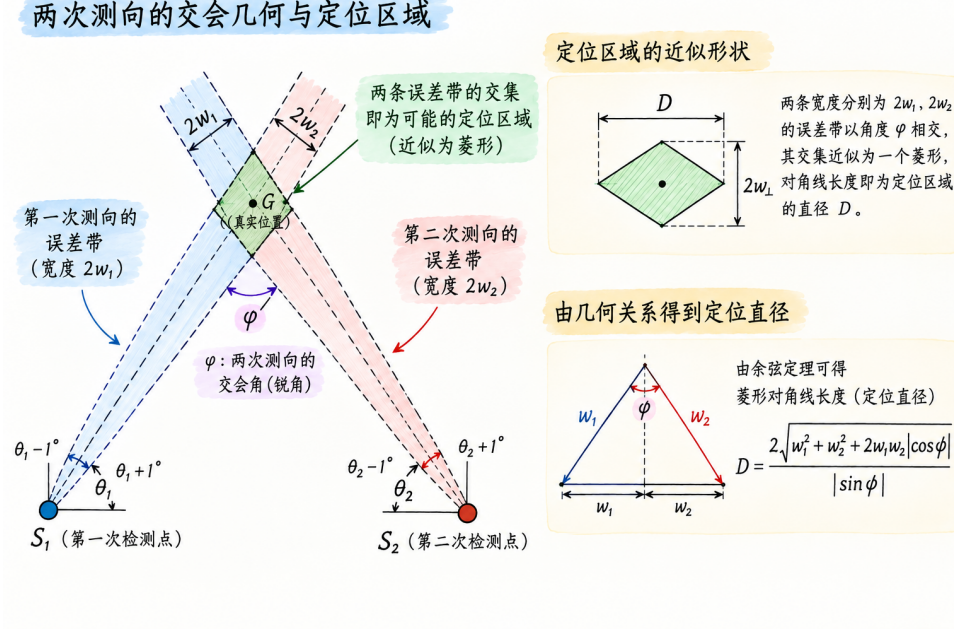


图 6 线性近似模型的几何关系

在候选区域内, 首先采用均匀网格进行离散采样, 生成第二检测点候选集。由于候选点数量较多, 若直接调用问题一建立的精确定位算法, 需要对每个候选点重复计算定位区域及其直径, 计算代价较高。因此, 本文首先建立局部线性评价模型, 对候选点进行快速预筛选, 仅保留少量具有较高潜力的候选点进入精确计算阶段。

对于每一个候选检测点, 在第一次测向形成的不确定区域 Ω_1 内均匀选取若干代表性目标点 G_k 。这些采样点覆盖整个可能区域, 用于近似描述不同真实目标位置对定位效果的影响。对于每个采样点, 根据两次测向之间的交会几何关系 (图 6), 利用局部误差带模型估计定位区域直径:

$$D_{\text{lin}} = \frac{2}{|\sin\varphi|} \sqrt{w_1^2 + w_2^2 + 2w_1w_2|\cos\varphi|}, \quad (15)$$

其中, w_1 和 w_2 分别表示两次测向形成的横向误差带宽, φ 为两次测向的交会角。

为了保证不同真实目标位置下均具有较好的定位性能, 采用最坏情况准则构造候选点评价函数:

$$J_{\text{lin}}(S_2) = \max_k D_{\text{lin}}(S_2, G_k), \quad (16)$$

该评价价值越小, 说明第二检测点在整个可能目标区域内均能够获得较小的定位误差。随后按照 J_{lin} 从小到大对所有候选点进行排序, 仅保留排名靠前的候选点组成集合 C_{top} , 进入下一阶段精确评价。

7.2.2 最优检测点求解

经过预筛选后, 候选点数量已大幅减少。对于保留的候选点, 进一步调用问题一建立的精确交会定位模型, 在不同目标位置及测向误差条件下计算定位区域直径, 并采用最坏情况作为评价指标:

$$J_{\text{exact}}(S_2) = \max_{G \in \Omega_1} D(\Omega_{12}(S_2, G)), \quad (17)$$

其中， Ω_{12} 表示两次测向后的定位区域， $D(\cdot)$ 为问题一中定义的定位区域直径。最终，第二检测点通过求解如下优化问题得到：

$$S_2^* = \arg \min_{S_2 \in \mathcal{C}_{\text{top}}} J_{\text{exact}}(S_2), \quad (18)$$

其中， $\mathcal{C}_{\text{top}}$ 为经过线性预筛选后保留的候选点集合。

与线性评价模型相比，精确评价进一步考虑了测向误差、目标区域边界以及两次测向几何关系等因素，因此能够更加真实地反映第二检测点对最终定位结果的影响。为进一步提高计算效率，本文采用“粗网格搜索—局部加密搜索”的两阶段优化策略：首先利用粗网格快速确定最优区域，再在局部区域内减小网格步长进行精细搜索，在保证定位精度的同时有效降低整体计算量。

7.3 结果分析

图 7 给出了最坏定位直径随第二检测点位置变化的关系。

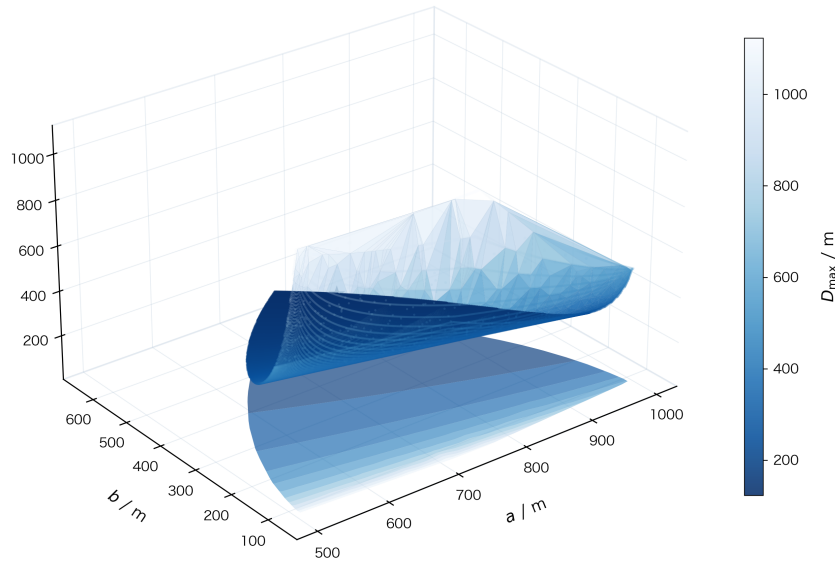


图 7 最坏定位直径随第二检测点位置变化关系

由图可见，目标函数整体变化较为平滑，在候选区域中心附近形成明显的低值区域，而远离该区域后定位直径迅速增大。这说明第二检测点的位置对最终定位精度具有显著影响，合理的检测点布局能够有效减小定位区域，提高交会定位精度。

进一步分析发现，目标函数最低点与上一节解析模型得到的初始候选位置基本一致，说明鲁棒接收约束与最小交会角约束能够较准确地刻画第二检测点的可行范围。解析模型不仅有效缩小了搜索空间，而且能够为数值优化提供高质量的初始解，使后续搜

索更加稳定。

此外，目标函数在最优区域附近变化相对平缓，而远离最优区域后迅速增大，表明该优化模型具有较好的鲁棒性。即使第二检测点存在一定的位置偏差，定位性能仍能够保持稳定，不会因微小扰动而明显恶化，因此所得检测点具有较好的工程应用价值。

8 问题三模型的建立与求解

8.1 问题三模型建立

8.1.1 完备搜索骨架的建立

由于干扰源的数量和位置均未知，机器狗首先必须保证“找得到”所有干扰源。若只根据当前已经发现的信号进行移动，可能始终在局部区域内活动，从而漏掉远处的干扰源。

因此，本文先建立一组具有覆盖保证的固定基准点，将其作为整个搜索任务的骨架。由于干扰源的有效接收半径至少为

$$R_{\min} = 1000 \text{ m}, \quad (19)$$

基准点设计按照最不利接收半径进行，而不使用较为乐观的 1500 m。

在目标区域中心设置一个基准点：

$$B_0 = (0, 0). \quad (20)$$

在半径 r_b 的圆周上均匀设置六个外围基准点（设置六个外围基准点必要性的证明见支撑材料/Q3_prove.pdf）：

$$B_j = \left(r_b \cos \frac{2\pi(j-1)}{6}, r_b \sin \frac{2\pi(j-1)}{6} \right), \quad j = 1, 2, \dots, 6. \quad (21)$$

由于干扰源的有效接收半径为 1000 ~ 1500 m，因此希望整个半径为 $R = 1800 \text{ m}$ 的目标区域尽可能处于外围基准点的有效覆盖范围内。

设外围六个基准点构成正六边形，则目标区域边界上距离相邻两个基准点最远的位置为两点连线的中点，其到最近基准点的距离为

$$d_{\max} = \sqrt{\left(R - \frac{r_b}{2}\right)^2 + \left(\frac{\sqrt{3}}{2}r_b\right)^2}. \quad (22)$$

令该最坏距离接近接收半径下限 1000 m，可得

$$d_{\max} \approx 1000 \text{ m} \implies r_b \approx 1140 \text{ m}. \quad (23)$$

综合考虑路径长度、区域覆盖裕量及后续联合定位效率，本文取

$$r_b = 1150 \text{ m}. \quad (24)$$

于是，机器狗的初始搜索骨架由中心点和六个外围点组成：

$$\mathcal{B} = \{B_0, B_1, B_2, \dots, B_6\}. \quad (25)$$

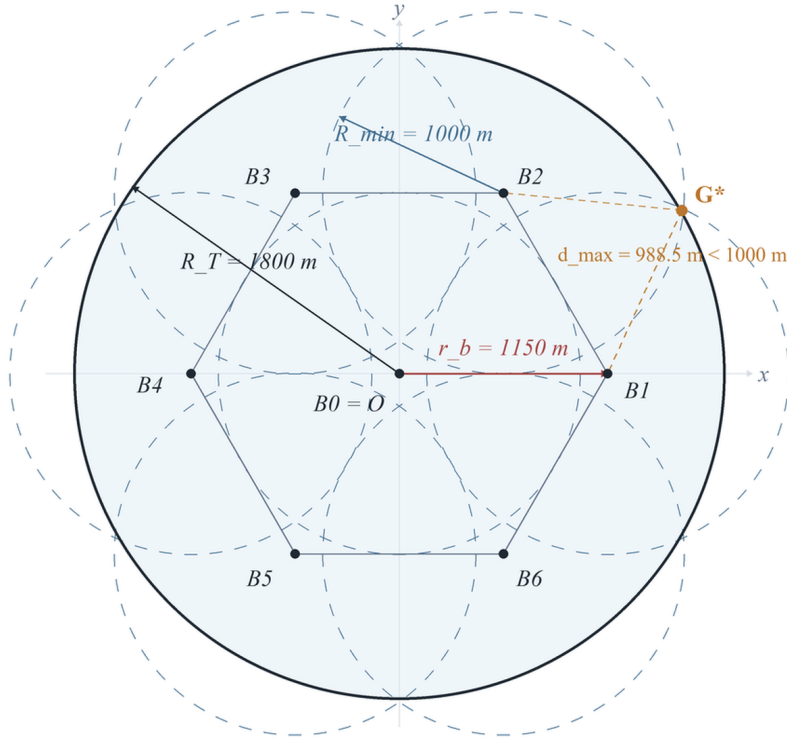


图 8 中心点与正六边形外围点构成的完备搜索骨架

8.1.2 检测结果驱动的候选区域更新

假设机器狗在位置 S 处检测频道 k ，获得示向度 $\hat{\theta}$ 。由于测向误差不超过 1° ，干扰源真实位置应位于以 S 为顶点的角域内，同时距离 S 不超过最大有效接收半径。

因此，频道 k 的候选区域更新为

$$\Omega_k^{\text{new}} = \Omega_k^{\text{old}} \cap \mathcal{B}(S, 1500) \cap \mathcal{W}(S, \hat{\theta}, 1^\circ). \quad (26)$$

如果更新后的候选区域非空，则说明该频道中存在干扰源，状态转移为

$$\text{SEARCHING} \longrightarrow \text{LOCALIZING}. \quad (27)$$

从这一刻开始，机器狗不再把该频道当作普通待搜索频道，而是为其单独维护定位任务。

若机器狗在位置 S 处没有检测到频道 k 的信号，由于问题三中的干扰源为全向干扰源，因此可以排除以 S 为圆心、半径为 1000 m 的区域：

$$\Omega_k^{\text{new}} = \Omega_k^{\text{old}} \setminus \mathcal{B}(S, 1000). \quad (28)$$

在实际程序中，为避免多边形离散误差和浮点误差导致真实位置被误删，采用略小于 1000 m 的半径进行保守排除。

由此可见，机器狗没有检测到信号并不意味着这次检测没有价值。相反，该结果能够排除一大片不可能存在干扰源的区域。

当经过多次检测后有

$$\Omega_k = \emptyset, \quad (29)$$

即可判定频道 k 中不存在干扰源，并令

$$s_k = \text{ABSENT}. \quad (30)$$

当接口返回 **near** 时，说明机器狗已经接近干扰源。此时继续进行普通测向不仅没有必要，还可能增加额外时间。

因此，机器狗直接在当前位置调用光学精确定位和激光清除操作。若清除成功，则有

$$s_k = \text{CLEARED}. \quad (31)$$

8.1.3 基于候选区域的清除判定

对于频道 k 的候选区域 Ω_k ，定义其直径为

$$D_k = \max_{P, Q \in \Omega_k} \|P - Q\|. \quad (32)$$

候选区域直径越小，表示机器狗对于干扰源位置的判断越准确。

本文利用第一问中的区域直径算法计算 D_k ，并将其作为衡量当前定位精度的重要指标。

仅利用候选区域直径不能直接确定清除位置。因此，本文进一步求解候选区域的最小包围圆：

$$\text{MEC}(\Omega_k) = (c_k, r_k), \quad (33)$$

其中， c_k 为最小包围圆圆心， r_k 为最小包围半径。
若满足

$$r_k \leq 20 \text{ m}, \quad (34)$$

则以 c_k 为清除点时，候选区域内任意可能的干扰源位置都与清除点的距离不超过 20 m，从而可以保证光学精确定位和清除操作成功。

因此，满足式 (34) 后，将频道状态更新为

$$\text{LOCALIZING} \longrightarrow \text{CLEARABLE}, \quad (35)$$

并将 c_k 加入清除任务集合。

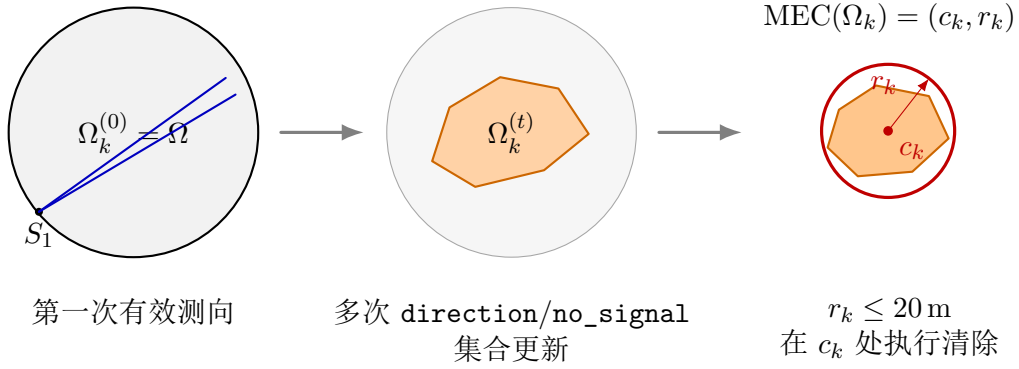


图 9 候选区域在线收缩与最小包围圆清除判定

8.2 问题三模型求解

8.2.1 基准点覆盖性验证

对于目标区域外环中的任意位置，设其到目标区域中心的距离为 ρ ，则

$$1000 \leq \rho \leq 1800.$$

六个外围基准点均匀分布，因此目标点与最近外围基准点之间的最大圆心角不超过

$$\alpha = \frac{\pi}{6}. \quad (36)$$

设外围基准点到原点的距离为 r_b ，则目标点与最近外围基准点之间的距离满足余弦定理：

$$d^2(\rho) = \rho^2 + r_b^2 - 2\rho r_b \cos \alpha. \quad (37)$$

由于 $d^2(\rho)$ 关于 ρ 为开口向上的二次函数，所以在 $\rho \in [1000, 1800]$ 上的最大值出现在区间端点：

$$d_{\max} = \max \{d(1000), d(1800)\}. \quad (38)$$

只要满足

$$d_{\max} \leq 1000, \quad (39)$$

就能保证目标区域内任意干扰源至少有一个基准点可以收到其信号。

因此，本文在程序开始运行前对式 (39) 进行检验。只有当搜索骨架满足最坏情形覆盖条件时，机器狗才开始正式执行搜索任务。

8.2.2 基准点复用策略

对于已经发现但尚未定位清楚的干扰源，机器狗可以直接利用第二问模型选择主动检测点。但是，第二问只关心测量后的定位效果，而第三问还需要考虑机器狗的总移动时间。

另一方面，机器狗后续本来就要经过若干固定基准点。如果某个未来基准点能够顺便改善某个频道的定位结果，就没有必要额外增加主动检测点。

因此，本文采用“先判断基准点能否复用，必要时再主动插点”的策略。

设尚未访问的基准点集合为

$$\mathcal{B}_{\text{rem}} = \{B_{\pi_1}, B_{\pi_2}, \dots, B_{\pi_m}\}. \quad (40)$$

以机器狗当前位置 X_{cur} 为起点，规划一条最短开放路线：

$$L_{\text{base}} = \|X_{\text{cur}} - B_{\pi_1}\| + \sum_{i=1}^{m-1} \|B_{\pi_i} - B_{\pi_{i+1}}\|. \quad (41)$$

由于基准点数量较少，可以枚举剩余基准点的访问顺序，选取使式 (41) 最小的路线。

设频道 k 当前候选区域为 Ω_k ，将未来基准点 B_i 作为下一次检测点，利用第二问模型计算检测后的最坏定位区域直径：

$$J_k(B_i) = \max_{\substack{G \in \Omega_k \\ \varepsilon \in [-1^\circ, 1^\circ]}} D(\Omega_k^{\text{after}}(B_i; G, \varepsilon)). \quad (42)$$

设问题二主动检测模型得到的最优评价值为

$$J_{k,\text{active}}^* = \min_S J_k(S). \quad (43)$$

当未来基准点满足

$$J_k(B_i) \leq \lambda J_{k,\text{active}}^*, \quad (44)$$

则说明该基准点虽然不一定是理论最优检测点，但其定位效果已经足够接近主动最优优点，可以直接复用。

此外，如果该基准点能够使当前候选区域明显缩小，即满足

$$J_k(B_i) \leq (1 - \eta) D_k, \quad (45)$$

则认为该基准点具有较高的信息增益，也可以安排该频道在此处进行检测。

8.2.3 主动检测点的选择

当所有未来基准点都无法满足复用条件时，机器狗才为频道 k 增加主动检测任务。对于主动检测候选点 S ，定义其最坏定位评价函数：

$$J_k(S) = \max_{\substack{G \in \Omega_k \\ \varepsilon \in [-1^\circ, 1^\circ]}} \text{diam}(\Omega_k^{\text{after}}(S; G, \varepsilon)). \quad (46)$$

理想主动检测点为

$$S_k^* = \arg \min_S J_k(S). \quad (47)$$

为了提高在线计算效率，采用两阶段搜索：

1. 在当前候选区域附近生成有限个主动检测候选点；
2. 排除无法保证接收信号的候选点；
3. 使用交会角和局部误差模型进行快速预筛选；
4. 保留评价较好的前 K 个候选点；
5. 调用问题一的精确定位区域算法计算其最坏定位直径；
6. 选择精确评价值最小的候选点。

8.2.4 定位质量与移动代价的分层优化

如果完全按照式 (47) 选择主动检测点，机器狗可能需要大幅偏离当前搜索路线。因此，本文采用“定位质量优先、移动代价次优”的分层优化方法。首先构造定位性能近优集合：

$$\mathcal{S}_k^{\text{near}} = \{S : J_k(S) \leq (1 + \delta)J_k^*\}. \quad (48)$$

在该集合中，所有候选点的定位效果均与理论最优点接近。然后再从中选择路线插入代价最小的点：

$$S_k^{\text{Q3}} = \arg \min_{S \in \mathcal{S}_k^{\text{near}}} \Delta L(S). \quad (49)$$

这样既避免主动检测点的定位质量明显下降，又能够减少机器狗的额外移动。

8.2.5 搜索、定位和清除任务的统一调度

将机器狗当前可能执行的任务统一表示为三类节点：

表 2 第三问任务节点

节点类型	任务含义	产生条件
BASE	前往固定基准点搜索频道	基准点尚未访问
ACTIVE	前往主动检测点进行定位	基准点复用效果不足
CLEAR	前往清除点执行清除	最小包围圆满足清除条件

机器狗首先以 BASE 节点构造搜索主路线，再将 ACTIVE 和 CLEAR 节点插入其中。
设当前路线为

$$R = (P_0, P_1, \dots, P_m), \quad (50)$$

其中 P_0 为机器狗当前位置。

若将任务点 Q 插入 P_i 与 P_{i+1} 之间，则路线长度增加量为

$$\Delta L_i(Q) = \|P_i - Q\| + \|Q - P_{i+1}\| - \|P_i - P_{i+1}\|. \quad (51)$$

任务点 Q 的最小插入代价为

$$\Delta L(Q) = \min_i \Delta L_i(Q). \quad (52)$$

对于所有待安排的主动检测任务和清除任务，每次选择增量最小的任务插入当前路线：

$$Q^* = \arg \min_Q \Delta L(Q). \quad (53)$$

通过这种方式，机器狗可以在原有搜索路线的基础上顺路完成定位和清除，减少大范围折返。

8.2.6 在线任务调度算法

Algorithm 1 第三问在线搜索、定位与清除算法

Require: 目标区域 Ω ; 频道集合 $\mathcal{K}$; 基准点集合 $\mathcal{B}$

Ensure: 已清除频道集合 $\mathcal{C}_{\text{clear}}$; 机器狗轨迹 $\mathcal{R}$

```

1: 初始化各频道候选区域  $\Omega_k \leftarrow \Omega$ , 状态  $q_k \leftarrow \text{SEARCHING}$ 
2: while 任务尚未完成 do
3:   规划经过所有未访问基准点的最短路线
4:   for all  $k : q_k = \text{LOCALIZING}$  do
5:     计算  $\text{MEC}(\Omega_k) = (c_k, r_k)$ 
6:     if  $r_k \leq 20 \text{ m}$  then
7:       将频道  $k$  标记为 CLEARABLE
8:     else
9:       若未来基准点具有足够定位效果, 则优先复用; 否则利用问题二模型选择
       主动检测点
10:    end if
11:  end for
12:  按最小路线增量联合调度 BASE、ACTIVE 和 CLEAR 任务
13:  执行当前首个任务并获取反馈
14:  根据示向、无信号、近距离及清除结果更新  $\Omega_k$  与  $q_k$ 
15: end while
16: return  $\mathcal{C}_{\text{clear}}, \mathcal{R}$ 

```

8.3 结果分析

为检验所建立搜索、定位与清除策略的实际效果, 本文首先利用模拟器进行多次演练, 在确认程序运行稳定后进行三次正式测试。图 10 给出了其中一次测试过程中机器狗的搜索、定位与清除路径。

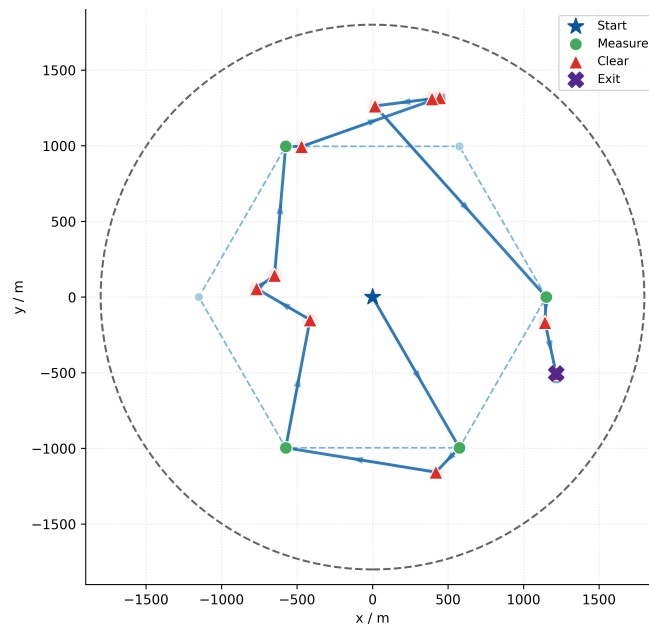


图 10 问题三正式测试中的机器狗搜索、定位与清除路径

由上图可以看出，机器狗从原点出发，按照预设搜索路线依次访问各检测点。在检测到干扰信号后，机器狗根据测向结果前往相应位置进行进一步定位，并在满足清除条件后执行激光清除。测试过程中，搜索路线和定位清除任务能够保持较好的衔接，说明所建立的在线调度方法具有一定的可执行性。

三次正式测试的具体结果如表 3 所示。

表 3 问题三正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间/s	程序运行时间/s
<i>TWHW-CAN9-HJ5E-7VD5</i>	12	345.60	17.98
<i>Z334-EYC3-NHJZ-32P8</i>	10	437.30	16.52
<i>K4CD-R5H8-YVRB-H2P8</i>	12	328.44	18.00

由表 3 可知，三次测试中机器狗分别清除了 12、10 和 12 个干扰源，平均每次清除干扰源约 11.33 个。三次测试的平均定位清除时间分别为 345.60 s、437.30 s 和 328.44 s。其中，测试二的平均定位清除时间较长，说明该测试中干扰源的位置分布或搜索路线对定位清除过程提出了更高要求；测试三的平均定位清除时间最短，表明在该测试场景下，机器狗的搜索路线与定位策略具有较好的匹配效果。

三次测试的程序运行时间分别为 17.98 s、16.52 s 和 18.00 s，最大差值为

$$18.00 - 16.52 = 1.48 \text{ s}. \quad (54)$$

这表明程序运行时间波动较小，算法能够在较稳定的计算时间内完成任务规划，满足在线决策的基本要求。

9 问题四模型的建立与求解

9.1 问题四模型建立

9.1.1 混合干扰源状态变量

对于频道 i ，定义干扰源的隐状态为

$$X_i = (Z_i, G_i, R_i, \Phi_i), \quad (55)$$

其中， Z_i 表示干扰源类型， $Z_i = 0$ 表示全向干扰源， $Z_i = 1$ 表示定向干扰源； $G_i = (x_i, y_i)$ 表示干扰源位置； R_i 表示有效接收半径； Φ_i 表示定向干扰源的发射方向。

对于每个频道，维护其存在概率

$$\pi_i = P(Z_i^{\text{exist}} = 1 \mid \mathcal{H}_i), \quad (56)$$

其中， $\mathcal{H}_i$ 表示频道 i 的历史观测信息。

采用粒子集合近似干扰源的联合后验状态：

$$\mathcal{P}_i = \{(x_{im}, w_{im})\}_{m=1}^M, \quad (57)$$

其中, x_{im} 为第 m 个粒子的状态, w_{im} 为对应权重, 并满足

$$\sum_{m=1}^M w_{im} = 1. \quad (58)$$

9.1.2 信号可见性模型

设检测点为 s , 第 m 个粒子对应的干扰源位置、接收半径和发射方向分别为 g_{im} 、 R_{im} 和 ϕ_{im} 。检测点到干扰源的距离为

$$d_{im}(s) = \|s - g_{im}\|. \quad (59)$$

对于全向干扰源, 只需满足

$$d_{im}(s) \leq R_{im}. \quad (60)$$

对于定向干扰源, 还需满足

$$(s - g_{im})^\top u(\phi_{im}) \geq 0. \quad (61)$$

因此, 第 m 个粒子的信号可见性指标定义为

$$I_{im}(s) = \mathbf{1}\{d_{im}(s) \leq R_{im}\} \mathbf{1}\{Z_{im} = 0 \text{ 或 } (s - g_{im})^\top u(\phi_{im}) \geq 0\}. \quad (62)$$

由粒子加权平均可得频道 i 在检测点 s 处接收到信号的概率:

$$P_i^{\text{sig}}(s) = \sum_{m=1}^M w_{im} I_{im}(s). \quad (63)$$

为刻画定向干扰源的方向覆盖效果, 定义条件定向检出概率:

$$P_i^{\text{dir}}(s) = \frac{\sum_{m=1}^M w_{im} \mathbf{1}\{Z_{im} = 1\} I_{im}(s)}{\sum_{m=1}^M w_{im} \mathbf{1}\{Z_{im} = 1\}}. \quad (64)$$

9.1.3 观测后验更新模型

当频道 i 在检测点 s 处检测到信号时, 可以确定该频道存在干扰源, 因此令 $\pi_i \leftarrow 1$. 若测向机获得示向度 θ_i , 则利用问题一中的测向误差约束构造方向扇形区域:

$$\Theta_i(s) = \{g : |\arg(g - s) - \theta_i| \leq 1^\circ\}. \quad (65)$$

结合最大接收半径约束，位置可行域更新为

$$\Omega_i^+ = \Omega_i \cap B(s, 1500) \cap \Theta_i(s). \quad (66)$$

当检测结果为无信号时，设

$$q_i(s) = P(\text{no_signal} \mid Z_i^{\text{exist}} = 1, s), \quad (67)$$

则根据贝叶斯公式，频道存在概率更新为

$$\pi_i^+ = \frac{\pi_i q_i(s)}{(1 - \pi_i) + \pi_i q_i(s)}. \quad (68)$$

粒子权重按照下式更新：

$$w_{im}^+ \propto w_{im} P(y_i \mid x_{im}, s), \quad (69)$$

归一化后得到

$$w_{im}^+ = \frac{w_{im} P(y_i \mid x_{im}, s)}{\sum_{l=1}^M w_{il} P(y_i \mid x_{il}, s)}. \quad (70)$$

9.1.4 累计方向覆盖模型

设频道 i 当前尚未被发现，定义其未覆盖概率为 S_i ，初始时令 $S_i = 1$ ，在检测点 s 处进行检测后，其边际新增方向覆盖量定义为

$$\Delta C_i(s) = S_i P_i^{\text{dir}}(s). \quad (71)$$

若本次检测仍未发现信号，则更新为

$$S_i^+ = S_i (1 - P_i^{\text{dir}}(s)). \quad (72)$$

于是，频道 i 的累计方向覆盖率为

$$C_i = 1 - S_i. \quad (73)$$

该指标可以避免重复选择覆盖方向相同的检测点，使后续搜索优先覆盖尚未观测的方向区域。

9.1.5 源数量约束模型

设当前已发现的干扰源数量为 D ，未发现频道集合为 $\mathcal{U}$ 。由于题目给出干扰源总数满足 $10 \leq N \leq 16$ ，因此本文采用了修正方法：

$$\tilde{\pi}_i = \max(\pi_i, \tau), \quad (74)$$

其中, τ 通过二分法确定, 使得

$$\sum_{i \in \mathcal{U}} \tilde{\pi}_i = \max(0, 10 - D). \quad (75)$$

进一步定义期望未发现干扰源数量:

$$E[N_{\text{unseen}}] = \sum_{i \in \mathcal{U}} \tilde{\pi}_i. \quad (76)$$

9.1.6 多频道联合补盲优化模型

设候选补盲点为 s , 在该点检测的频道集合为 $\mathcal{B}(s)$ 。考虑频道存在概率、方向覆盖率和方位公平性, 定义频道 i 在点 s 处的收益为

$$G_i(s) = \tilde{\pi}_i F_i \Delta C_i(s), \quad (77)$$

其中, F_i 为方位公平性系数:

$$F_i = 1 + \eta \max(0, N_{\min} - N_i^{\text{sector}}). \quad (78)$$

候选点总收益为

$$G(s) = \sum_{i \in \mathcal{B}(s)} G_i(s). \quad (79)$$

设候选点插入当前路线产生的最小额外距离为

$$\Delta L(s) = \min_j [\|P_j - s\| + \|s - P_{j+1}\| - \|P_j - P_{j+1}\|]. \quad (80)$$

若机器狗速度为 v , 候选点服务时间为 $T_{\text{ser}}(s)$, 则总耗时为

$$T(s) = \frac{\Delta L(s)}{v} + T_{\text{ser}}(s). \quad (81)$$

定义候选点综合价值函数:

$$V(s) = \frac{G(s) + \lambda \sum_{i \in \mathcal{B}(s)} \Delta C_i(s)}{\sqrt{T(s)}}. \quad (82)$$

最终补盲点选择模型为

$$s^* = \arg \max_{s \in \mathcal{C}} V(s). \quad (83)$$

9.2 问题四求解算法

9.2.1 总体求解思路

本文采用滚动时域在线决策方法求解问题四。算法以问题三的七点搜索骨架为基础，在每次检测完成后根据最新观测结果重新更新频道状态，并重新规划下一步动作。算法始终按照以下优先级进行决策：

$$\text{安全清除} \succ \text{主动定位} \succ \text{定向补盲} \succ \text{基础搜索} \quad (84)$$

其中，安全清除优先保证已经定位的干扰源及时清除；主动定位用于缩小已发现干扰源的位置不确定性；定向补盲用于降低未发现频道的漏检风险；基础搜索则负责完成初始区域覆盖。

9.2.2 三级搜索策略

首先按照问题三设计的中心点和外围六点搜索骨架进行检测。在执行主路线过程中，若某一侧向候选点能够带来较大的新增方向覆盖，且插入路线所需的额外距离较小，则将其作为顺路补盲点加入当前路线。

当七点搜索骨架完成后，在内环、中环和外环上生成候选点，同时加入针对高风险频道构造的对置检测点。每轮计算所有候选点的综合价值 $V(s)$ ，选择价值最大的检测点，并在同一位置联合检测多个未发现频道。

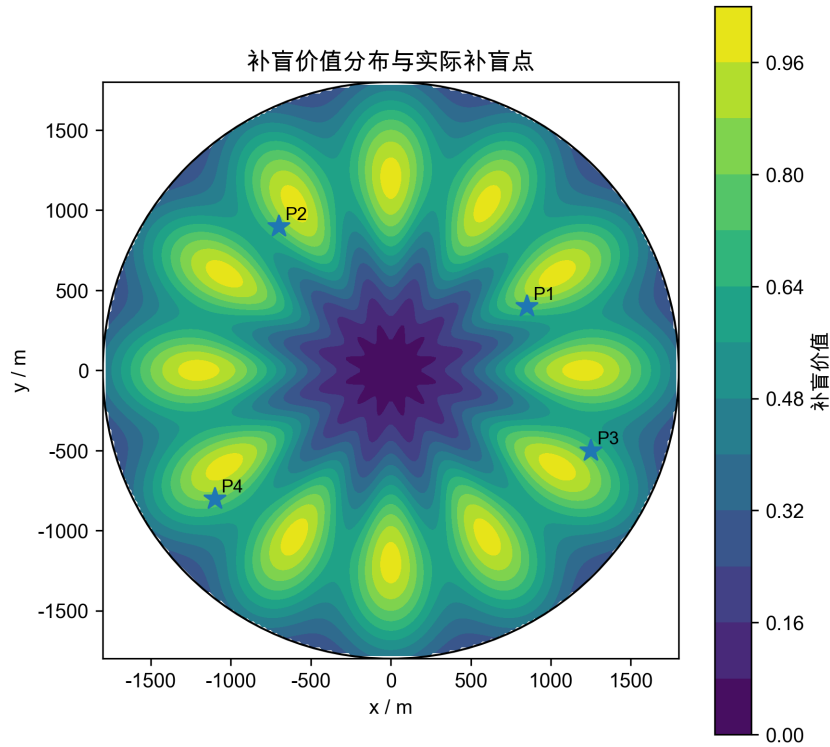


图 11 补盲价值分布与实际补盲点

若普通补盲结束后仍存在较大的未发现源风险，或已发现干扰源数量不足 10 个，则启动外环兜底搜索。该阶段优先选择能够覆盖新方向、且对多个高风险频道具有较大联合收益的候选点。

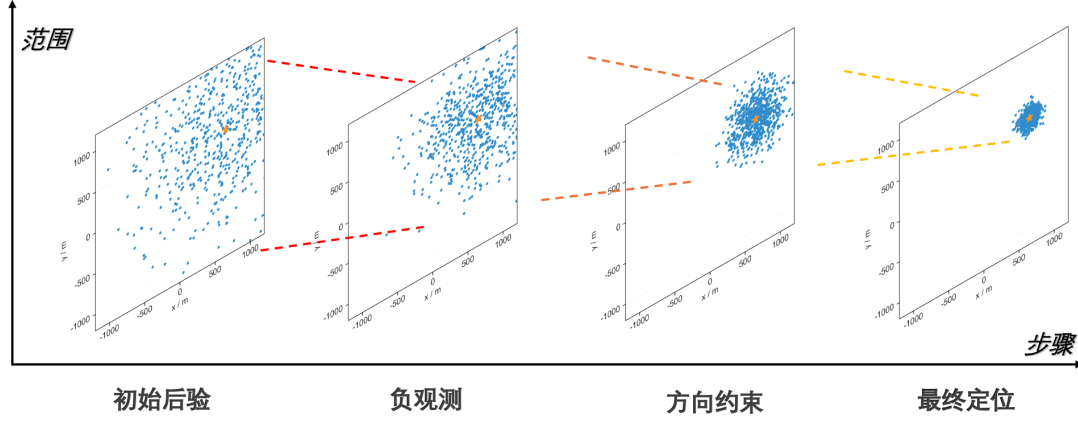


图 12 定向干扰源后验概率的逐步收缩过程

由图 12 可以看出，初始后验分布覆盖范围较大，目标位置存在较强不确定性。加入负观测信息后，部分不满足信号覆盖条件的粒子被剔除，候选区域明显缩小；进一步引入测向结果后，粒子分布受到方向约束，逐渐向干扰源真实位置附近集中。经过多轮更新，最终定位区域显著收缩，说明所建立的后验更新方法能够有效利用负观测与方向观测信息，提高定向干扰源的定位精度。

9.2.3 已发现干扰源的主动定位

对于已经获得示向度的频道，继续使用问题一中的几何可行域更新方法。当可行域较大时，利用问题二建立的主动检测点优化模型选择下一检测位置。

设当前频道的位置可行域为 Ω_i ，其最小包围圆圆心和半径分别为 c_i 和 r_i 。当满足

$$r_i \leq 20 \text{ m}, \quad (85)$$

则认为该干扰源已经满足光学定位条件，机器狗移动至 c_i 附近执行光学定位和激光清除。

若清除失败，则保留当前频道的定位状态，重新选择主动检测点进行补充定位。

9.2.4 停止判据

当同时满足以下条件时，停止主动搜索：

$$D \geq 10, \quad (86)$$

$$E[N_{\text{unseen}}] < 0.30, \quad (87)$$

$$C_i \geq C_{\min}, \quad (88)$$

并且高风险频道满足

$$N_i^{\text{sector}} \geq 4. \quad (89)$$

其中, D 表示已发现干扰源数量, C_i 表示频道 i 的累计方向覆盖率, N_i^{sector} 表示已覆盖的方向扇区数量。

9.3 结果分析

为检验混合干扰源环境下所建立搜索、定位、补盲与清除策略的实际效果, 本文首先利用模拟器进行多次演练测试, 在确认程序运行稳定后进行三次正式测试。与问题三相比, 问题四中进一步考虑了定向干扰源的方向覆盖盲区, 因此算法在完成基础搜索和主动定位的同时, 还需要执行针对未发现频道的定向补盲。

三次正式测试结果如表 4 所示。

表 4 问题三正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间/s	程序运行时间/s
CV8P-U7VD-6GZC-K3CH	10	589.43	99.90
YUSF-E99S-5SHB-QA6R	14	265.81	57.91
RBKH-9TQ8-C575-Q2HU	10	362.81	81.56

由表 4 可知, 三次正式测试分别清除了 10、14 和 10 个干扰源, 平均清除干扰源数量为

$$\frac{10 + 14 + 10}{3} = 11.33. \quad (90)$$

三次测试的平均定位清除时间分别为 589.43 s、265.81 s 和 362.81 s。其中第二次测试的平均定位清除时间最低, 说明在该场景下, 搜索骨架、主动定位与定向补盲任务之间具有较好的协同性; 第一次测试耗时相对较长, 表明该场景可能存在较多处于定向覆盖盲区的干扰源, 需要执行更多补盲检测和额外移动。

三次测试的程序运行时间分别为 99.90 s、57.91 s 和 81.56 s。相比问题三, 问题四需要额外维护频道存在概率、方向覆盖状态及补盲候选点, 因此在线决策计算量有所增加, 但程序运行时间仍显著低于模拟任务本身的虚拟运行时间, 能够满足在线滚动决策的需要。

10 模型评价与推广

10.1 模型优点

1. **模型结构统一。**将测向误差、信号接收范围、干扰源可见性、机器狗移动和操作时间纳入同一框架，较完整地描述搜索、定位与清除过程。
2. **具有一定的鲁棒性。**对全向干扰源采用有界误差和最坏情况约束，对定向干扰源引入信号可见性和后验更新，能够降低测量误差及方向盲区的影响。
3. **精度与效率兼顾，便于实现。**通过几何可行域、交会角、路线插入代价和任务时间共同评价检测点，并采用“固定搜索骨架 + 主动检测 + 滚动重规划”策略，具有较好的可解释性和可执行性。

10.2 模型缺点

1. 部分阈值、权重和候选点网格需要结合设备性能与测试数据标定，并在计算精度和实时性之间作适当权衡。
2. 当前模型主要针对平面、静态干扰源场景；面对复杂障碍物或动态干扰源时，需要相应调整路径和状态更新模型。

10.3 模型推广

1. **多机器狗协同搜索。**将区域、频道或定位任务进行分配，并共享候选区域和后验信息，可缩短整体任务时间。
2. **动态干扰源定位。**若干扰源位置或发射方向随时间变化，可在粒子后验模型中加入运动状态和状态转移方程，形成动态跟踪模型。
3. **复杂环境与多目标决策。**对含障碍物或不同地形的区域，可将欧氏距离替换为地图上的可行路径距离，并进一步加入能耗、通信稳定性和作业风险等指标，形成多目标优化模型。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于程序代码的调试与检查、图表及 L^AT_EX 排版辅助、论文语言表达润色等工作。AI 工具仅作为辅助工具使用，本文的核心建模方案、数学模型与公式推导、算法设计与参数选择、程序实现与模拟器测试、实验数据处理、结果分析及最终结论均由参赛队成员独立完成，并对 AI 辅助产生的相关内容进行了人工审查、修改与核实。详细使用情况见支撑材料《AI 工具使用详情.pdf》。

参考文献

- [1] OPENAI. ChatGPT (GPT-5) [CP/OL]. 开发与提供机构: OpenAI; 使用日期: 2026-09-10-2026-09-13. <https://chatgpt.com/>.

- [2] DOĞANÇAY K. Bearings-only target localization using total least squares[J]. *Signal Processing*, 2005, 85(9): 1695–1710. DOI: <https://doi.org/10.1016/j.sigpro.2005.03.007>.
- [3] GARULLI A, VICINO A. Set membership localization of mobile robots via angle measurements[J]. *IEEE Transactions on Robotics and Automation*, 2001, 17(4): 450–463. DOI: <https://doi.org/10.1109/70.954757>.
- [4] DOĞANÇAY K, HMAM H. Optimal angular sensor separation for AOA localization[J]. *Signal Processing*, 2008, 88(5): 1248–1260. DOI: <https://doi.org/10.1016/j.sigpro.2007.11.013>.
- [5] TOKEKAR P, ISLER V. Sensor placement and selection for bearing sensors with bounded uncertainty[C]// 2013 IEEE International Conference on Robotics and Automation. Piscataway: IEEE, 2013: 2515–2520. DOI: <https://doi.org/10.1109/ICRA.2013.6630920>.
- [6] SCHNEIDER R. *Convex Bodies: The Brunn–Minkowski Theory*[M]. 2nd expanded ed. Cambridge: Cambridge University Press, 2014. DOI: <https://doi.org/10.1017/CB09781139003858>.
- [7] JUNG H W E. Ueber die kleinste Kugel, die eine räumliche Figur einschliesst[J]. *Journal für die reine und angewandte Mathematik*, 1901, 123: 241–257. DOI: <https://doi.org/10.1515/crll.1901.123.241>.
- [8] WELZL E. Smallest enclosing disks (balls and ellipsoids)[C]// MAURER H, ed. *New Results and New Trends in Computer Science. Lecture Notes in Computer Science*, vol. 555. Berlin: Springer, 1991: 359–370. DOI: <https://doi.org/10.1007/BFb0038202>.
- [9] ARULAMPALAM M S, MASKELL S, GORDON N, et al. A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking[J]. *IEEE Transactions on Signal Processing*, 2002, 50(2): 174–188. DOI: <https://doi.org/10.1109/78.978374>.
- [10] ROSENKRANTZ D J, STEARNS R E, LEWIS P M II. An analysis of several heuristics for the traveling salesman problem[J]. *SIAM Journal on Computing*, 1977, 6(3): 563–581. DOI: <https://doi.org/10.1137/0206041>.
- [11] PILLAC V, GENDREAU M, GUÉRET C, et al. A review of dynamic vehicle routing problems[J]. *European Journal of Operational Research*, 2013, 225(1): 1–11. DOI: <https://doi.org/10.1016/j.ejor.2012.08.015>.

附录 A 支撑材料文件列表

支撑材料文件及其用途如表 5 所示。

表 5 支撑材料文件列表

文件名	用途
problem1.py	问题一几何建模与定位区域计算。
problem1_visualize.py	问题一测试案例及可视化。
problem2.py	问题二鲁棒检测点选择与定位精度评价。
problem3.py	问题三多频道搜索、定位、路径规划与清除。
problem4.py	问题四混合全向/定向干扰源在线控制。
problem4_visualize.py	问题四粒子后验与补盲价值可视化。
trajectory.py	搜索、测量、清除及退出轨迹绘制。
flow_figure.png	四个问题整体求解框架流程图。
image.png	定位区域逐步收缩过程示意图。
image1.png	四个问题之间的整体技术路线图。
image2.png	问题三搜索骨架及覆盖范围示意图。
image3.png	单次测向形成的扇形约束区域几何示意图。
image4.png	问题一定位区域计算与清除判据流程图。
image5.png	两次测向交会定位的局部线性误差模型及定位区域直径近似示意图。
problem1_global_view.png	问题一全局测向约束与可行定位区域图。
problem1_local_view.png	问题一局部定位区域及边界示意图。
problem2.png	问题二候选检测点定位误差评价图。
problem2_1.png	问题二候选检测区域与约束边界图。
problem3_trajectory.png	问题三机器人实际运行轨迹图。
problem5_value.png	问题四补盲价值分布及实际补盲点示意图。
formal-p3-1-*.jlog	问题三第 1 次正式运行日志。
formal-p3-2-*.jlog	问题三第 2 次正式运行日志。
formal-p3-3-*.jlog	问题三第 3 次正式运行日志。
formal-p4-1-*.jlog	问题四第 1 次正式运行日志。
formal-p4-2-*.jlog	问题四第 2 次正式运行日志。
formal-p4-3-*.jlog	问题四第 3 次正式运行日志。
Q3_prove.pdf	设置六个外围基准点最优证明。
AI 工具使用详情.pdf	人工智能工具使用情况说明。

附录 B 程序代码

本附录仅保留建模与求解所需的核心算法。命令行参数解析、HTTP 重试、日志记录、自测、运行状态打印及可视化等工程性代码均已省略。公共实现首次出现后，后续问题直接复用，不再重复列出。

附录 B.1 公共几何与问题一：可行域及直径计算

```
1 # 公共二维点、距离、半平面裁剪和圆域边界实现；后续问题直接复用。
2 from __future__ import annotations
3 from dataclasses import dataclass
4 from math import atan2, cos, hypot, pi, radians, sin, sqrt
5 from typing import Iterable, Literal, Optional
6 TAU = 2.0 * pi
7 DEFAULT_MAX_RECEIVE_RANGE = 1500.0
8 TriangleMode = Literal['inscribed', 'conservative']
9
10 @dataclass(frozen=True)
11 class Point:
12     x: float
13     y: float
14
15     def __add__(self, other: 'Point') -> 'Point':
16         return Point(self.x + other.x, self.y + other.y)
17
18     def __sub__(self, other: 'Point') -> 'Point':
19         return Point(self.x - other.x, self.y - other.y)
20
21     def __mul__(self, k: float) -> 'Point':
22         return Point(self.x * k, self.y * k)
23     __rmul__ = __mul__
24
25 def dot(p: Point, q: Point) -> float:
26     return p.x * q.x + p.y * q.y
27
28 def norm(p: Point) -> float:
29     return hypot(p.x, p.y)
30
31 def distance(p: Point, q: Point) -> float:
32     return norm(p - q)
33
34 def circle_point(radius: float, theta: float) -> Point:
35     return Point(radius * cos(theta), radius * sin(theta))
36
37 def normalize_angle(theta: float) -> float:
38     return theta % TAU
39
40 @dataclass(frozen=True)
41 class HalfPlane:
42     a: float
43     b: float
44     c: float
45
46     @staticmethod
47     def make(a: float, b: float, c: float) -> 'HalfPlane':
48         length = hypot(a, b)
49         if length == 0.0:
50             raise ValueError('半平面法向量不能为零')
```

```

51         return HalfPlane(a / length, b / length, c / length)
52
53     def contains(self, p: Point, eps_len: float) -> bool:
54         return self.a * p.x + self.b * p.y <= self.c + eps_len
55
56 @dataclass(frozen=True)
57 class Segment:
58     p: Point
59     q: Point
60
61 @dataclass(frozen=True)
62 class Arc:
63     start: float
64     end: float
65
66     def contains_angle(self, theta: float, eps_ang: float) -> bool:
67         t = normalize_angle(theta)
68         if t < self.start - eps_ang:
69             t += TAU
70         return t <= self.end + eps_ang
71
72 @dataclass
73 class Boundary:
74     segments: list[Segment]
75     arcs: list[Arc]
76     points: list[Point]
77
78 @dataclass(frozen=True)
79 class DiameterResult:
80     a: Point
81     b: Point
82     distance: float
83
84 @dataclass(frozen=True)
85 class CoverResult:
86     covered: bool
87     center: Point
88     radius: float
89     max_distance: float
90     worst_point: Point
91     margin: float
92
93 class Region:
94
95     def __init__(self, radius: float=1800.0):
96         if radius <= 0.0:
97             raise ValueError('目标圆半径必须为正')
98         self.radius = radius
99         self.halfplanes: list[HalfPlane] = []
100         self.eps_len = 1e-09 * max(1.0, radius)
101         self.eps_ang = 1e-12
102
103     def add_halfplane(self, a: float, b: float, c: float) -> None:
104         self.halfplanes.append(HalfPlane.make(a, b, c))
105
106     def add_direction_wedge(self, station: Point, alpha_rad: float, error_rad: float=pi / 180.0) -> None:
107         beta_low = alpha_rad - error_rad
108         beta_high = alpha_rad + error_rad
109         a1 = sin(beta_low)
110         b1 = -cos(beta_low)
111         c1 = a1 * station.x + b1 * station.y

```

```

112     a2 = -sin(beta_high)
113     b2 = cos(beta_high)
114     c2 = a2 * station.x + b2 * station.y
115     self.add_halfplane(a1, b1, c1)
116     self.add_halfplane(a2, b2, c2)
117
118     def add_direction_wedge_deg(self, station: Point, alpha_deg: float, error_deg: float=1.0) -> None:
119         self.add_direction_wedge(station, radians(alpha_deg), radians(error_deg))
120
121     def add_bearing(self, station: Point, alpha_rad: float, error_rad: float=pi / 180.0, max_range: float=
122         DEFAULT_MAX_RECEIVE_RANGE, triangle_mode: TriangleMode='inscribed') -> None:
123         if max_range <= 0.0:
124             raise ValueError('有效接收距离上界必须为正')
125         if not 0.0 <= error_rad < pi / 2.0:
126             raise ValueError('测向误差角必须位于 [0, pi/2) 内')
127         if triangle_mode not in ('inscribed', 'conservative'):
128             raise ValueError("triangle_mode 只能是 'inscribed' 或 'conservative'")
129         self.add_direction_wedge(station, alpha_rad, error_rad)
130         direction_x = cos(alpha_rad)
131         direction_y = sin(alpha_rad)
132         if triangle_mode == 'inscribed':
133             cutoff = max_range * cos(error_rad)
134         else:
135             cutoff = max_range
136         c = direction_x * station.x + direction_y * station.y + cutoff
137         self.add_halfplane(direction_x, direction_y, c)
138
139     def add_bearing_deg(self, station: Point, alpha_deg: float, error_deg: float=1.0, max_range: float=
140         DEFAULT_MAX_RECEIVE_RANGE, triangle_mode: TriangleMode='inscribed') -> None:
141         self.add_bearing(station, radians(alpha_deg), radians(error_deg), max_range, triangle_mode)
142
143     def contains(self, p: Point) -> bool:
144         if dot(p, p) > self.radius * self.radius + 2.0 * self.radius * self.eps_len:
145             return False
146         return all((h.contains(p, self.eps_len) for h in self.halfplanes))
147
148     def _line_circle_points(self, h: HalfPlane) -> list[Point]:
149         r = self.radius
150         if abs(h.c) > r + self.eps_len:
151             return []
152         foot = Point(h.a * h.c, h.b * h.c)
153         tangent_sq = max(0.0, r * r - h.c * h.c)
154         offset = sqrt(tangent_sq)
155         direction = Point(-h.b, h.a)
156         p = foot + direction * offset
157         if offset <= self.eps_len:
158             return [p]
159         q = foot - direction * offset
160         return [p, q]
161
162     def _extract_segments(self) -> tuple[list[Segment], list[Point]]:
163         r = self.radius
164         segments: list[Segment] = []
165         degenerate_points: list[Point] = []
166         for h in self.halfplanes:
167             if abs(h.c) > r + self.eps_len:
168                 continue
169             p0 = Point(h.a * h.c, h.b * h.c)
170             direction = Point(-h.b, h.a)
171             extent = sqrt(max(0.0, r * r - h.c * h.c))
172             low = -extent

```

```

171         high = extent
172         feasible = True
173         for g in self.halfplanes:
174             q = g.a * direction.x + g.b * direction.y
175             rhs = g.c - g.a * p0.x - g.b * p0.y
176             if abs(q) <= self.eps_ang:
177                 if rhs < -self.eps_len:
178                     feasible = False
179                     break
180                 continue
181             bound = rhs / q
182             if q > 0.0:
183                 high = min(high, bound)
184             else:
185                 low = max(low, bound)
186             if low > high + self.eps_len:
187                 feasible = False
188                 break
189         if not feasible:
190             continue
191         if high - low <= self.eps_len:
192             p = p0 + direction * ((low + high) / 2.0)
193             if self.contains(p):
194                 degenerate_points.append(p)
195         else:
196             p = p0 + direction * low
197             q = p0 + direction * high
198             if self.contains(p) and self.contains(q):
199                 segments.append(Segment(p, q))
200         return (self._deduplicate_segments(segments), degenerate_points)
201
202     def _extract_arcs(self) -> list[Arc]:
203         break_angles: list[float] = []
204         for h in self.halfplanes:
205             for p in self._line_circle_points(h):
206                 break_angles.append(normalize_angle(atan2(p.y, p.x)))
207         break_angles = self._deduplicate_angles(break_angles)
208         if not break_angles:
209             if self.contains(Point(self.radius, 0.0)):
210                 return [Arc(0.0, TAU)]
211             return []
212         arcs: list[Arc] = []
213         count = len(break_angles)
214         for (i, start) in enumerate(break_angles):
215             end = break_angles[(i + 1) % count]
216             if i == count - 1:
217                 end += TAU
218             elif end <= start:
219                 end += TAU
220             middle = (start + end) / 2.0
221             if self.contains(circle_point(self.radius, middle)):
222                 arcs.append(Arc(start, end))
223         return arcs
224
225     def boundary(self) -> Boundary:
226         (segments, degenerate) = self._extract_segments()
227         arcs = self._extract_arcs()
228         points = list(degenerate)
229         for s in segments:
230             points.extend((s.p, s.q))
231         for arc in arcs:

```

```

232         points.append(circle_point(self.radius, arc.start))
233         points.append(circle_point(self.radius, arc.end))
234     points = self._deduplicate_points(points)
235     if not segments and (not arcs) and (not points):
236         raise ValueError('定位区域为空: 请检测测向数据、角度约定和误差模型')
237     return Boundary(segments=segments, arcs=arcs, points=points)
238
239 def _find_antipodal_arc_pair(self, arcs: Iterable[Arc]) -> Optional[tuple[Point, Point]]:
240     arc_list = list(arcs)
241     for first in arc_list:
242         for second in arc_list:
243             for k in range(-2, 3):
244                 shifted_low = second.start - pi + k * TAU
245                 shifted_high = second.end - pi + k * TAU
246                 low = max(first.start, shifted_low)
247                 high = min(first.end, shifted_high)
248                 if low <= high + self.eps_ang:
249                     theta = (low + high) / 2.0
250                     p = circle_point(self.radius, theta)
251                     q = circle_point(self.radius, theta + pi)
252                     if self.contains(p) and self.contains(q):
253                         return (p, q)
254     return None
255
256 def diameter(self, boundary: Optional[Boundary]=None) -> DiameterResult:
257     if boundary is None:
258         boundary = self.boundary()
259     antipodal = self._find_antipodal_arc_pair(boundary.arcs)
260     if antipodal is not None:
261         return DiameterResult(antipodal[0], antipodal[1], 2.0 * self.radius)
262     candidates = list(boundary.points)
263     if not candidates:
264         raise ValueError('非空区域没有可用边界候选点')
265     best_a = candidates[0]
266     best_b = candidates[0]
267     best_sq = 0.0
268
269     def consider(p: Point, q: Point) -> None:
270         nonlocal best_a, best_b, best_sq
271         d = p - q
272         value = dot(d, d)
273         if value > best_sq:
274             best_sq = value
275             (best_a, best_b) = (p, q)
276     for (i, p) in enumerate(candidates):
277         for q in candidates[i + 1:]:
278             consider(p, q)
279     for p in candidates:
280         length = norm(p)
281         if length <= self.eps_len:
282             continue
283         q = Point(-self.radius * p.x / length, -self.radius * p.y / length)
284         if self.contains(q):
285             consider(p, q)
286     return DiameterResult(best_a, best_b, sqrt(best_sq))
287
288 def check_diameter_circle(self, diameter: Optional[DiameterResult]=None, boundary: Optional[Boundary]=
None) -> CoverResult:
289     if boundary is None:
290         boundary = self.boundary()
291     if diameter is None:

```

```

292         diameter = self.diameter(boundary)
293         center = (diameter.a + diameter.b) * 0.5
294         radius = diameter.distance * 0.5
295         candidates = list(boundary.points)
296         if not candidates and boundary.arcs:
297             candidates.append(circle_point(self.radius, boundary.arcs[0].start))
298         if not candidates:
299             raise ValueError('非空区域没有可用覆盖候选点')
300         center_norm = norm(center)
301         if center_norm > self.eps_len:
302             opposite = Point(-self.radius * center.x / center_norm, -self.radius * center.y / center_norm)
303             if self.contains(opposite):
304                 candidates.append(opposite)
305         worst_point = candidates[0]
306         max_distance = distance(center, worst_point)
307         for p in candidates[1:]:
308             current = distance(center, p)
309             if current > max_distance:
310                 max_distance = current
311                 worst_point = p
312         cover_eps = 10.0 * self.eps_len
313         margin = radius - max_distance
314         return CoverResult(covered=max_distance <= radius + cover_eps, center=center, radius=radius,
max_distance=max_distance, worst_point=worst_point, margin=margin)
315
316     def _deduplicate_points(self, points: Iterable[Point]) -> list[Point]:
317         result: list[Point] = []
318         eps = 10.0 * self.eps_len
319         for p in points:
320             if not any((distance(p, q) <= eps for q in result)):
321                 result.append(p)
322         return result
323
324     def _deduplicate_angles(self, angles: Iterable[float]) -> list[float]:
325         values = sorted((normalize_angle(a) for a in angles))
326         result: list[float] = []
327         for value in values:
328             if not result or abs(value - result[-1]) > self.eps_ang:
329                 result.append(value)
330         if len(result) >= 2 and TAU - result[-1] + result[0] <= self.eps_ang:
331             result[0] = 0.5 * (result[0] + result[-1] - TAU) % TAU
332             result.pop()
333             result.sort()
334         return result
335
336     def _deduplicate_segments(self, segments: Iterable[Segment]) -> list[Segment]:
337         result: list[Segment] = []
338         eps = 10.0 * self.eps_len
339         for segment in segments:
340             duplicate = False
341             for old in result:
342                 same_order = distance(segment.p, old.p) <= eps and distance(segment.q, old.q) <= eps
343                 reverse_order = distance(segment.p, old.q) <= eps and distance(segment.q, old.p) <= eps
344                 if same_order or reverse_order:
345                     duplicate = True
346                     break
347             if not duplicate:
348                 result.append(segment)
349         return result

```

附录 B.2 问题二：第二测站的鲁棒选址

```
1 # Point、Region、distance 等公共几何实现已在前一小节定义。
2 from __future__ import annotations
3 from dataclasses import dataclass
4 from math import atan2, cos, degrees, hypot, isfinite, pi, radians, sin, sqrt, tan
5 from typing import Sequence
6 from problem1 import Point, Region, TriangleMode, distance
7
8 @dataclass(frozen=True)
9 class Problem2Config:
10     first_station: Point
11     first_bearing_deg: float
12     rho_minus: float
13     rho_plus: float
14     phi_0: float
15     target_radius: float = 1800.0
16     receive_radius_min: float = 1000.0
17     receive_radius_max: float = 1500.0
18     bearing_error_rad: float = radians(1.0)
19     optical_radius: float = 20.0
20     use_2d_receive_filter: bool = True
21     limit_station_to_target_disk: bool = True
22     use_optical_shortcut: bool = True
23     triangle_mode: TriangleMode = 'inscribed'
24     n_a: int = 81
25     n_b_per_side: int = 41
26     n_rho: int = 31
27     n_first_angle: int = 5
28     n_second_error: int = 5
29     top_k_for_exact: int = 40
30
31 @dataclass(frozen=True)
32 class Candidate:
33     a: float
34     b: float
35     xy: Point
36
37 @dataclass(frozen=True)
38 class SelectionResult:
39     best: Candidate
40     score: float
41     score_name: str
42     candidates: tuple[Candidate, ...]
43     proxy_scores: tuple[float, ...]
44     exact_scores: tuple[float | None, ...]
45     targets: tuple[Point, ...]
46
47 def linspace(start: float, stop: float, count: int) -> list[float]:
48     if count <= 0:
49         raise ValueError('count 必须为正整数')
50     if count == 1:
51         return [(start + stop) / 2.0]
52     step = (stop - start) / (count - 1)
53     return [start + index * step for index in range(count)]
54
55 def unit_basis(theta_deg: float) -> tuple[Point, Point]:
56     theta = radians(theta_deg)
57     return (Point(cos(theta), sin(theta)), Point(-sin(theta), cos(theta)))
58
59 def local_to_global(config: Problem2Config, a: float, b: float) -> Point:
```

```

60     (u, v) = unit_basis(config.first_bearing_deg)
61     return config.first_station + a * u + b * v
62
63 def global_to_local(config: Problem2Config, point: Point) -> tuple[float, float]:
64     (u, v) = unit_basis(config.first_bearing_deg)
65     relative = point - config.first_station
66     return (relative.x * u.x + relative.y * u.y, relative.x * v.x + relative.y * v.y)
67
68 def wrap_deg(angle_deg: float) -> float:
69     return angle_deg % 360.0
70
71 def validate_config(config: Problem2Config) -> tuple[float, float]:
72     if not 0.0 <= config.rho_minus < config.rho_plus:
73         raise ValueError('必须满足 0 <= rho_minus < rho_plus')
74     if config.rho_plus > config.receive_radius_max + 1e-12:
75         raise ValueError('rho_plus 不能超过第一次有效接收距离上界')
76     if not 0.0 < config.phi_0 < pi / 2.0:
77         raise ValueError('phi_0 必须位于 (0, pi/2) 内, 单位为弧度')
78     if config.target_radius <= 0.0:
79         raise ValueError('target_radius 必须为正')
80     if config.receive_radius_min <= 0.0:
81         raise ValueError('receive_radius_min 必须为正')
82     if config.receive_radius_max < config.receive_radius_min:
83         raise ValueError('receive_radius_max 不能小于 receive_radius_min')
84     if not 0.0 <= config.bearing_error_rad < pi / 2.0:
85         raise ValueError('bearing_error_rad 必须位于 [0, pi/2) 内')
86     if config.optical_radius < 0.0:
87         raise ValueError('optical_radius 不能为负')
88     if config.triangle_mode not in ('inscribed', 'conservative'):
89         raise ValueError("triangle_mode 只能是 'inscribed' 或 'conservative'")
90     integer_parameters = {'n_a': config.n_a, 'n_b_per_side': config.n_b_per_side, 'n_rho': config.n_rho, 'n_first_angle': config.n_first_angle, 'n_second_error': config.n_second_error, 'top_k_for_exact': config.top_k_for_exact}
91     for (name, value) in integer_parameters.items():
92         if value <= 0:
93             raise ValueError(f'{name} 必须为正整数')
94     m = 0.5 * (config.rho_minus + config.rho_plus)
95     h = 0.5 * (config.rho_plus - config.rho_minus)
96     if h > config.receive_radius_min + 1e-12:
97         raise ValueError('h > receive_radius_min, 纯接收鲁棒区域为空')
98     if h > config.receive_radius_min * cos(config.phi_0) + 1e-12:
99         raise ValueError('候选区域为空: h > receive_radius_min*cos(phi_0)。请降低 phi_0 或缩小 [rho_minus, rho_plus]。')
100     return (m, h)
101
102 def candidate_a_interval(config: Problem2Config) -> tuple[float, float]:
103     (m, h) = validate_config(config)
104     half_width = config.receive_radius_min * cos(config.phi_0) - h
105     return (m - half_width, m + half_width)
106
107 def candidate_b_bounds(config: Problem2Config, a: float) -> tuple[float, float]:
108     (m, h) = validate_config(config)
109     q = h + abs(a - m)
110     if q > config.receive_radius_min:
111         raise ValueError('该 a 不满足鲁棒接收约束')
112     b_min = q * tan(config.phi_0)
113     b_max = sqrt(max(0.0, config.receive_radius_min ** 2 - q ** 2))
114     if b_min > b_max + 1e-10:
115         raise ValueError('该 a 下交会角约束与鲁棒接收约束不相容')
116     return (b_min, b_max)
117

```

```

118 def analytic_centers(config: Problem2Config) -> list[Candidate]:
119     (m, h) = validate_config(config)
120     b_star = sqrt(max(0.0, config.receive_radius_min ** 2 - h ** 2))
121     return [Candidate(m, +b_star, local_to_global(config, m, +b_star)), Candidate(m, -b_star,
        local_to_global(config, m, -b_star))]
122
123 def maximum_guaranteed_angle(config: Problem2Config) -> float:
124     (_, h) = validate_config(config)
125     ratio = min(1.0, max(0.0, h / config.receive_radius_min))
126     return atan2(sqrt(max(0.0, 1.0 - ratio ** 2)), ratio)
127
128 def candidate_grid(config: Problem2Config) -> list[Candidate]:
129     (m, h) = validate_config(config)
130     (a_low, a_high) = candidate_a_interval(config)
131     candidates: list[Candidate] = []
132     for a in linspace(a_low, a_high, config.n_a):
133         (b_min, b_max) = candidate_b_bounds(config, a)
134         if abs(b_max - b_min) <= 1e-12:
135             absolute_b_values = [0.5 * (b_min + b_max)]
136         else:
137             absolute_b_values = linspace(b_min, b_max, config.n_b_per_side)
138         for absolute_b in absolute_b_values:
139             if absolute_b <= 1e-10:
140                 continue
141             for sign in (-1.0, +1.0):
142                 b = sign * absolute_b
143                 q = h + abs(a - m)
144                 weak_physical_ok = abs(b) <= config.receive_radius_min + 1e-09
145                 robust_axis_ok = q ** 2 + b ** 2 <= config.receive_radius_min ** 2 + 1e-07
146                 angle_ok = abs(b) + 1e-09 >= q * tan(config.phi_0)
147                 xy = local_to_global(config, a, b)
148                 disk_ok = not config.limit_station_to_target_disk or hypot(xy.x, xy.y) <= config.
        target_radius + 1e-09
149                 if weak_physical_ok and robust_axis_ok and angle_ok and disk_ok:
150                     candidates.append(Candidate(a, b, xy))
151     if not candidates:
152         raise RuntimeError('离散候选点为空, 请检查参数或提高网格分辨率')
153     return candidates
154
155 def possible_target_scenarios(config: Problem2Config, two_dimensional: bool | None=None) -> list[Point]:
156     validate_config(config)
157     if two_dimensional is None:
158         two_dimensional = config.use_2d_receive_filter
159     rhos = linspace(config.rho_minus, config.rho_plus, config.n_rho)
160     offsets = linspace(-config.bearing_error_rad, config.bearing_error_rad, config.n_first_angle) if
        two_dimensional else [0.0]
161     theta_center = radians(config.first_bearing_deg)
162     targets: list[Point] = []
163     for rho in rhos:
164         for offset in offsets:
165             radial_distance = rho / cos(offset)
166             theta = theta_center + offset
167             target = config.first_station + Point(radial_distance * cos(theta), radial_distance * sin(theta)
        ))
168             if radial_distance <= config.receive_radius_max + 1e-09 and hypot(target.x, target.y) <= config
        .target_radius + 1e-09:
169                 targets.append(target)
170     if not targets:
171         raise RuntimeError('可能目标场景为空, 请检查首次检测点、示向度和 rho 区间')
172     return targets
173

```

```

174 def passes_sampled_receive_check(config: Problem2Config, candidate: Candidate, targets: Sequence[Point]) ->
    bool:
175     return max((distance(candidate.xy, target) for target in targets)) <= config.receive_radius_min + 1e-09
176
177 def acute_intersection_angle(config: Problem2Config, second_station: Point, target: Point) -> float:
178     first_vector = target - config.first_station
179     second_vector = target - second_station
180     first_norm = hypot(first_vector.x, first_vector.y)
181     second_norm = hypot(second_vector.x, second_vector.y)
182     if first_norm <= 1e-12 or second_norm <= 1e-12:
183         return pi / 2.0
184     cosine = abs((first_vector.x * second_vector.x + first_vector.y * second_vector.y) / (first_norm *
        second_norm))
185     cosine = min(1.0, max(0.0, cosine))
186     return atan2(sqrt(max(0.0, 1.0 - cosine ** 2)), cosine)
187
188 def linearized_diameter(config: Problem2Config, second_station: Point, target: Point) -> float:
189     r1 = distance(target, config.first_station)
190     r2 = distance(target, second_station)
191     if config.use_optical_shortcut and r2 <= config.optical_radius:
192         return 0.0
193     phi = acute_intersection_angle(config, second_station, target)
194     sine = abs(sin(phi))
195     if sine <= 1e-10:
196         return float('inf')
197     w1 = r1 * tan(config.bearing_error_rad)
198     w2 = r2 * tan(config.bearing_error_rad)
199     return 2.0 / sine * sqrt(w1 ** 2 + w2 ** 2 + 2.0 * abs(cos(phi)) * w1 * w2)
200
201 def proxy_robust_score(config: Problem2Config, candidate: Candidate, targets: Sequence[Point]) -> float:
202     return max((linearized_diameter(config, candidate.xy, target) for target in targets))
203
204 class Problem1Adapter:
205
206     def __init__(self, config: Problem2Config):
207         self.config = config
208
209     def diameter_from_measurements(self, measurements: Sequence[tuple[Point, float]]) -> float:
210         region = Region(radius=self.config.target_radius)
211         for (station, measured_deg) in measurements:
212             region.add_bearing_deg(station, measured_deg, error_deg=degrees(self.config.bearing_error_rad),
                max_range=self.config.receive_radius_max, triangle_mode=self.config.triangle_mode)
213         boundary = region.boundary()
214         return region.diameter(boundary).distance
215
216 def exact_robust_score(config: Problem2Config, adapter: Problem1Adapter, candidate: Candidate, targets:
    Sequence[Point]) -> float:
217     second_errors = linspace(-config.bearing_error_rad, config.bearing_error_rad, config.n_second_error)
218     worst = 0.0
219     for target in targets:
220         r2 = distance(target, candidate.xy)
221         if config.use_optical_shortcut and r2 <= config.optical_radius:
222             scenario_value = 0.0
223         else:
224             true_second_deg = degrees(atan2(target.y - candidate.xy.y, target.x - candidate.xy.x))
225             scenario_value = 0.0
226             for error_rad in second_errors:
227                 measured_second_deg = wrap_deg(true_second_deg + degrees(error_rad))
228                 measurements = [(config.first_station, wrap_deg(config.first_bearing_deg)), (candidate.xy,
                measured_second_deg)]
229                 try:

```

```

230         diameter = adapter.diameter_from_measurements(measurements)
231     except ValueError:
232         diameter = float('inf')
233     scenario_value = max(scenario_value, diameter)
234     worst = max(worst, scenario_value)
235     return worst
236
237 def choose_second_station(config: Problem2Config, use_problem1: bool=False, adapter: Problem1Adapter | None
238     =None) -> SelectionResult:
239     targets = possible_target_scenarios(config)
240     candidates = candidate_grid(config)
241     if config.use_2d_receive_filter:
242         candidates = [candidate for candidate in candidates if passes_sampled_receive_check(config,
243             candidate, targets)]
244         if not candidates:
245             raise RuntimeError('加入二维角域宽度后候选点为空。可降低 phi_0、缩小 rho 区间，或提高候选网格分辨率。')
246     proxy_scores = [proxy_robust_score(config, candidate, targets) for candidate in candidates]
247     exact_scores: list[float | None] = [None] * len(candidates)
248     if not use_problem1:
249         best_index = min(range(len(candidates)), key=proxy_scores.__getitem__)
250         return SelectionResult(best=candidates[best_index], score=proxy_scores[best_index], score_name='线性化最坏直径', candidates=tuple(candidates), proxy_scores=tuple(proxy_scores), exact_scores=tuple(
251             exact_scores), targets=tuple(targets))
252     if adapter is None:
253         adapter = Problem1Adapter(config)
254     ranked_indices = sorted(range(len(candidates)), key=proxy_scores.__getitem__)
255     selected_indices = ranked_indices[:min(config.top_k_for_exact, len(candidates))]
256     for index in selected_indices:
257         exact_scores[index] = exact_robust_score(config, adapter, candidates[index], targets)
258     finite_indices = [index for index in selected_indices if exact_scores[index] is not None and isfinite(
259         exact_scores[index])]
260     if not finite_indices:
261         raise RuntimeError('问题一算法未得到有限直径，请检查角度约定和模型参数')
262     best_index = min(finite_indices, key=lambda index: exact_scores[index] if exact_scores[index] is not
263         None else float('inf'))
264     return SelectionResult(best=candidates[best_index], score=float(exact_scores[best_index]), score_name='问题一离散鲁棒直径', candidates=tuple(candidates), proxy_scores=tuple(proxy_scores), exact_scores=tuple(
265         exact_scores), targets=tuple(targets))

```

附录 B.3 问题三：区域更新、主动测量与路径控制

```

1 from __future__ import annotations
2
3 import itertools
4 import math
5 import random
6 from dataclasses import dataclass, field
7 from typing import Any, Iterable, Optional
8
9 import numpy as np
10 from shapely.geometry import Point, Polygon
11 from shapely.geometry.base import BaseGeometry
12
13 from problem2 import (
14     Candidate,
15     Problem2Config,
16     candidate_grid,
17     global_to_local,
18     passes_sampled_receive_check,

```

```

19     possible_target_scenarios,
20     proxy_robust_score,
21     wrap_deg,
22 )
23
24 # 第二问的配置、坐标变换、候选网格、接收约束和鲁棒评分均直接复用;
25 # P2Point 仅是在线控制器所需的 NumPy 转换适配层。
26
27 TARGET_RADIUS = 1800.0
28 SIGNAL_RADIUS_MIN = 1000.0
29 SIGNAL_RADIUS_MAX = 1500.0
30 NEAR_RADIUS = 5.0
31 CLEAR_RADIUS = 20.0
32 BEARING_ERROR_DEG = 1.0
33 DOG_SPEED = 5.0
34 OUTER_COUNT = 6
35 OUTER_RADIUS = 1150.0
36 OUTER_ROTATION_DEG = 0.0
37 ARENA_MODEL_MARGIN = 0.8
38 SIGNAL_MAX_MODEL_MARGIN = 0.8
39 ANGLE_MODEL_MARGIN_DEG = 0.03
40 NO_SIGNAL_REMOVE_RADIUS = 999.0
41 CLEAR_FAILURE_REMOVE_RADIUS = 19.0
42 CLEAR_MEC_TRIGGER = 19.8
43 CIRCLE_QUAD_SEGS = 96
44 PHIO_DEG = 35.0
45 ROBUST_RX_LIMIT = 999.0
46 MIN_NEW_STATION_SEPARATION = 25.0
47 Q2_N_A = 25
48 Q2_N_B_PER_SIDE = 9
49 Q2_GENERIC_ANGLES = 24
50 Q2_GENERIC_RADII = (250.0, 450.0, 650.0, 850.0)
51 Q2_ROUGH_TARGETS = 8
52 Q2_EXACT_TARGETS = 22
53 Q2_EXACT_TOP_K = 14
54 Q2_ERROR_SCENARIOS_DEG = (-1.0, 0.0, 1.0)
55 ACTIVE_NEAR_OPT_REL = 0.12
56 BASE_REUSE_FACTOR = 1.35
57 BASE_REUSE_MIN_REDUCTION = 0.2
58 BASE_OPPORTUNISTIC_MIN_OVERLAP = 0.2
59 BASE_OPPORTUNISTIC_MAX_CHANNELS = 6
60 CLEAR_ROUTE_INSERT_MAX_M = 300.0
61 CLEAR_ESCAPE_CURRENT_MAX_M = 750.0
62 CLEAR_ESCAPE_FUTURE_MIN_M = 650.0
63 MAX_DEFERRED_CLEAR = 2
64 ACTIVE_CLEAR_BIAS = 0.15
65 REAL_TIME_SAFETY_MARGIN_S = 12.0
66
67 def dist(a: np.ndarray | tuple[float, float], b: np.ndarray | tuple[float, float]) -> float:
68     aa = np.asarray(a, dtype=float)
69     bb = np.asarray(b, dtype=float)
70     return float(np.linalg.norm(aa - bb))
71
72 def angle_deg_from_to(a: np.ndarray, b: np.ndarray) -> float:
73     d = b - a
74     return float(math.degrees(math.atan2(d[1], d[0])) % 360.0)
75
76 def regular_base_points(outer_count: int=OUTER_COUNT, outer_radius: float=OUTER_RADIUS, rotation_deg: float
    =OUTER_ROTATION_DEG) -> list[np.ndarray]:
77     pts = [np.array([0.0, 0.0], dtype=float)]
78     phi0 = math.radians(rotation_deg)

```

```

79     for j in range(outer_count):
80         phi = phi0 + 2.0 * math.pi * j / outer_count
81         pts.append(np.array([outer_radius * math.cos(phi), outer_radius * math.sin(phi)], dtype=float))
82     return pts
83
84 def regular_backbone_worst_distance(outer_count: int, outer_radius: float) -> float:
85     if outer_count < 1:
86         return float('inf')
87     alpha = math.pi / outer_count
88
89     def d_at(rho: float) -> float:
90         value = rho * rho + outer_radius * outer_radius - 2.0 * rho * outer_radius * math.cos(alpha)
91         return math.sqrt(max(0.0, value))
92     return max(d_at(1000.0), d_at(TARGET_RADIUS))
93
94 def validate_backbone_or_raise(outer_count: int, outer_radius: float) -> None:
95     worst = regular_backbone_worst_distance(outer_count, outer_radius)
96     print(f'[geometry] outer_count={outer_count}, outer_radius={outer_radius:.3f} m, worst guaranteed
97           source-to-outer-station distance={worst:.3f} m')
98     if worst > SIGNAL_RADIUS_MIN + 1e-09:
99         raise ValueError(f'当前基准点参数不能保证半径 1800 m 圆域的完备 1000 m 覆盖: 最坏距离 {worst:.3f} > {
100             SIGNAL_RADIUS_MIN:.3f} m')
101
102 def disk(center: np.ndarray, radius: float) -> BaseGeometry:
103     return Point(float(center[0]), float(center[1])).buffer(float(radius), quad_segs=CIRCLE_QUAD_SEGS)
104
105 def initial_arena_region() -> BaseGeometry:
106     return disk(np.array([0.0, 0.0]), TARGET_RADIUS + ARENA_MODEL_MARGIN)
107
108 def make_wedge(station: np.ndarray, measured_deg: float, half_width_deg: float) -> Polygon:
109     theta1 = math.radians(measured_deg - half_width_deg)
110     theta2 = math.radians(measured_deg + half_width_deg)
111     far = max(10000.0, float(np.linalg.norm(station)) + TARGET_RADIUS + 5000.0)
112     p1 = station + far * np.array([math.cos(theta1), math.sin(theta1)])
113     p2 = station + far * np.array([math.cos(theta2), math.sin(theta2)])
114     return Polygon([(float(station[0]), float(station[1])), (float(p1[0]), float(p1[1])), (float(p2[0]),
115         float(p2[1]))])
116
117 def clean_geometry(g: BaseGeometry) -> BaseGeometry:
118     if g.is_empty:
119         return g
120     try:
121         if not g.is_valid:
122             g = g.buffer(0)
123     except Exception:
124         pass
125     return g
126
127 def region_hull_points(region: BaseGeometry) -> np.ndarray:
128     if region.is_empty:
129         return np.empty((0, 2), dtype=float)
130     hull = region.convex_hull
131     if hull.geom_type == 'Point':
132         return np.array([[hull.x, hull.y]], dtype=float)
133     if hull.geom_type == 'LineString':
134         return np.asarray(hull.coords, dtype=float)
135     return np.asarray(hull.exterior.coords[:-1], dtype=float)
136
137 def region_diameter(region: BaseGeometry) -> float:
138     pts = region_hull_points(region)
139     n = len(pts)

```

```

137     if n <= 1:
138         return 0.0
139     diff = pts[:, None, :] - pts[None, :, :]
140     d2 = np.sum(diff * diff, axis=2)
141     return float(np.sqrt(np.max(d2)))
142
143 def max_distance_to_region(point: np.ndarray, region: BaseGeometry) -> float:
144     pts = region_hull_points(region)
145     if len(pts) == 0:
146         return 0.0
147     return float(np.max(np.linalg.norm(pts - point[None, :], axis=1)))
148
149 @dataclass(frozen=True)
150 class Circle:
151     x: float
152     y: float
153     r: float
154
155     @property
156     def center(self) -> np.ndarray:
157         return np.array([self.x, self.y], dtype=float)
158
159 def _is_in_circle(c: Circle, p: tuple[float, float]) -> bool:
160     return math.hypot(p[0] - c.x, p[1] - c.y) <= c.r + 1e-07
161
162 def _diameter_circle(a: tuple[float, float], b: tuple[float, float]) -> Circle:
163     cx = (a[0] + b[0]) / 2.0
164     cy = (a[1] + b[1]) / 2.0
165     return Circle(cx, cy, math.hypot(a[0] - b[0], a[1] - b[1]) / 2.0)
166
167 def _circumcircle(a: tuple[float, float], b: tuple[float, float], c: tuple[float, float]) -> Optional[
168     Circle]:
169     (ax, ay) = a
170     (bx, by) = b
171     (cx, cy) = c
172     d = 2.0 * (ax * (by - cy) + bx * (cy - ay) + cx * (ay - by))
173     if abs(d) < 1e-12:
174         return None
175     aa = ax * ax + ay * ay
176     bb = bx * bx + by * by
177     cc = cx * cx + cy * cy
178     ux = (aa * (by - cy) + bb * (cy - ay) + cc * (ay - by)) / d
179     uy = (aa * (cx - bx) + bb * (ax - cx) + cc * (bx - ax)) / d
180     return Circle(ux, uy, math.hypot(ux - ax, uy - ay))
181
182 def minimum_enclosing_circle(region: BaseGeometry) -> Circle:
183     pts_np = region_hull_points(region)
184     if len(pts_np) == 0:
185         return Circle(0.0, 0.0, float('inf'))
186     pts = [(float(x), float(y)) for (x, y) in pts_np]
187     rng = random.Random(20260911)
188     rng.shuffle(pts)
189     c: Optional[Circle] = None
190     for (i, p) in enumerate(pts):
191         if c is not None and _is_in_circle(c, p):
192             continue
193         c = Circle(p[0], p[1], 0.0)
194         for j in range(i):
195             q = pts[j]
196             if _is_in_circle(c, q):
197                 continue

```

```

197         c = _diameter_circle(p, q)
198         for k in range(j):
199             r = pts[k]
200             if _is_in_circle(c, r):
201                 continue
202             cc = _circumcircle(p, q, r)
203             if cc is not None:
204                 c = cc
205             else:
206                 candidates = [_diameter_circle(p, q), _diameter_circle(p, r), _diameter_circle(q, r)]
207                 valid = [x for x in candidates if _is_in_circle(x, p) and _is_in_circle(x, q) and
208                     _is_in_circle(x, r)]
209                 c = min(valid, key=lambda x: x.r)
210         assert c is not None
211         return c
212
213 def update_region_direction(region: BaseGeometry, station: np.ndarray, svd_deg: float) -> BaseGeometry:
214     upper_disk = disk(station, SIGNAL_RADIUS_MAX + SIGNAL_MAX_MODEL_MARGIN)
215     wedge = make_wedge(station, svd_deg, BEARING_ERROR_DEG + ANGLE_MODEL_MARGIN_DEG)
216     out = region.intersection(upper_disk).intersection(wedge)
217     return clean_geometry(out)
218
219 def update_region_no_signal(region: BaseGeometry, station: np.ndarray) -> BaseGeometry:
220     out = region.difference(disk(station, NO_SIGNAL_REMOVE_RADIUS))
221     return clean_geometry(out)
222
223 def update_region_clear_failure(region: BaseGeometry, station: np.ndarray) -> BaseGeometry:
224     out = region.difference(disk(station, CLEAR_FAILURE_REMOVE_RADIUS))
225     return clean_geometry(out)
226
227 def sample_region_points(region: BaseGeometry, max_points: int) -> np.ndarray:
228     if region.is_empty:
229         return np.empty((0, 2), dtype=float)
230     hull_pts = region_hull_points(region)
231     if len(hull_pts) > max_points:
232         idx = np.linspace(0, len(hull_pts) - 1, max_points, dtype=int)
233         hull_pts = hull_pts[idx]
234     extras: list[tuple[float, float]] = []
235     if region.geom_type == 'MultiPolygon':
236         for geom in region.geoms:
237             p = geom.representative_point()
238             extras.append((p.x, p.y))
239     else:
240         p = region.representative_point()
241         extras.append((p.x, p.y))
242     if extras:
243         pts = np.vstack([hull_pts, np.asarray(extras, dtype=float)])
244     else:
245         pts = hull_pts
246     if len(pts) == 0:
247         return pts
248     rounded = np.round(pts, decimals=6)
249     (_, unique_idx) = np.unique(rounded, axis=0, return_index=True)
250     return pts[np.sort(unique_idx)]
251
252 @dataclass(frozen=True)
253 class P2Point:
254     x: float
255     y: float
256
257     def __add__(self, other: 'P2Point') -> 'P2Point':

```

```

257         return P2Point(self.x + other.x, self.y + other.y)
258
259     def __sub__(self, other: 'P2Point') -> 'P2Point':
260         return P2Point(self.x - other.x, self.y - other.y)
261
262     def __mul__(self, value: float) -> 'P2Point':
263         return P2Point(self.x * value, self.y * value)
264     __rmul__ = __mul__
265
266     def as_np(self) -> np.ndarray:
267         return np.array([self.x, self.y], dtype=float)
268
269 def p2_distance(a: P2Point, b: P2Point) -> float:
270     return math.hypot(a.x - b.x, a.y - b.y)
271
272 def first_direction_observation(observations: list[dict[str, Any]]) -> Optional[dict[str, Any]]:
273     for obs in observations:
274         if obs.get('result') == 'direction':
275             return obs
276     return None
277
278 def build_q2_config_from_current_region(region: BaseGeometry, observations: list[dict[str, Any]], phi0_deg:
279     float=PHI0_DEG) -> Problem2Config:
280     first = first_direction_observation(observations)
281     if first is None:
282         raise ValueError('没有 direction 观测, 不能构造问题二配置')
283     s_np = np.asarray(first['position'], dtype=float)
284     theta_deg = float(first['angle_deg'])
285     theta_rad = math.radians(theta_deg)
286     u_np = np.array([math.cos(theta_rad), math.sin(theta_rad)])
287     pts = region_hull_points(region)
288     if len(pts) == 0:
289         raise ValueError('当前候选区域为空')
290     proj = (pts - s_np[None, :]) @ u_np
291     rho_minus = max(0.0, float(np.min(proj)))
292     rho_plus = min(SIGNAL_RADIUS_MAX, float(np.max(proj)))
293     if rho_plus <= rho_minus + 1e-06:
294         rho_plus = min(SIGNAL_RADIUS_MAX, rho_minus + 1.0)
295     h = 0.5 * (rho_plus - rho_minus)
296     phi_candidates = [phi0_deg, 30.0, 25.0, 20.0, 15.0, 10.0, 5.0]
297     feasible_phi = None
298     for deg in phi_candidates:
299         if h <= SIGNAL_RADIUS_MIN * math.cos(math.radians(deg)) + 1e-09:
300             feasible_phi = deg
301             break
302     if feasible_phi is None:
303         raise ValueError('当前轴向区间过长, 问题二鲁棒候选域为空')
304     return Problem2Config(first_station=P2Point(float(s_np[0]), float(s_np[1])), first_bearing_deg=
305         theta_deg, rho_minus=rho_minus, rho_plus=rho_plus, phi_0=math.radians(feasible_phi), target_radius=
306         TARGET_RADIUS, receive_radius_min=SIGNAL_RADIUS_MIN, receive_radius_max=SIGNAL_RADIUS_MAX,
307         bearing_error_rad=math.radians(BEARING_ERROR_DEG), optical_radius=CLEAR_RADIUS, use_2d_receive_filter=
308         True, limit_station_to_target_disk=False, use_optical_shortcut=True, n_a=Q2_N_A, n_b_per_side=
309         Q2_N_B_PER_SIDE, n_rho=Q2_EXACT_TARGETS, n_first_angle=5, n_second_error=len(Q2_ERROR_SCENARIOS_DEG),
310         top_k_for_exact=Q2_EXACT_TOP_K)
311
312 def target_inside_current_region(target: P2Point, region: BaseGeometry) -> bool:
313     return region.buffer(1.0).covers(Point(target.x, target.y))
314
315 def _deduplicate_p2_targets(targets: Iterable[P2Point], decimals: int=4) -> list[P2Point]:
316     seen: set[tuple[float, float]] = set()
317     result: list[P2Point] = []

```

```

311     for t in targets:
312         key = (round(t.x, decimals), round(t.y, decimals))
313         if key in seen:
314             continue
315         seen.add(key)
316         result.append(t)
317     return result
318
319 def _downsample_targets(targets: list[P2Point], max_count: int) -> list[P2Point]:
320     if max_count <= 0:
321         return targets
322     if len(targets) <= max_count:
323         return targets
324     idx = np.linspace(0, len(targets) - 1, max_count, dtype=int)
325     return [targets[int(i)] for i in idx]
326
327 def current_q2_targets(config: Problem2Config, region: BaseGeometry) -> list[P2Point]:
328     scenario_targets = possible_target_scenarios(config)
329     filtered = [t for t in scenario_targets if target_inside_current_region(t, region)]
330     boundary_pts = sample_region_points(region, Q2_EXACT_TARGETS)
331     merged: list[P2Point] = list(filtered)
332     for (x, y) in boundary_pts:
333         merged.append(P2Point(float(x), float(y)))
334     merged = _deduplicate_p2_targets(merged)
335     if merged:
336         return merged
337     pts = sample_region_points(region, max(8, Q2_EXACT_TARGETS))
338     return [P2Point(float(x), float(y)) for (x, y) in pts]
339
340 def exact_robust_score_current_region(config: Problem2Config, candidate: Candidate, targets: list[P2Point],
341     current_region: BaseGeometry) -> float:
342     worst = 0.0
343     s_np = candidate.xy.as_np()
344     for target in targets:
345         r2 = p2_distance(target, candidate.xy)
346         if config.use_optical_shortcut and r2 <= config.optical_radius:
347             scenario_value = 0.0
348         else:
349             true_second_deg = math.degrees(math.atan2(target.y - candidate.xy.y, target.x - candidate.xy.x))
350             scenario_value = 0.0
351             for error_deg in Q2_ERROR_SCENARIOS_DEG:
352                 measured_second_deg = wrap_deg(true_second_deg + error_deg)
353                 new_region = update_region_direction(current_region, s_np, measured_second_deg)
354                 if new_region.is_empty:
355                     diameter = float('inf')
356                 else:
357                     diameter = region_diameter(new_region)
358                 scenario_value = max(scenario_value, diameter)
359     worst = max(worst, scenario_value)
360     return worst
361
362 @dataclass
363 class ActiveOption:
364     point: np.ndarray
365     score: float
366
367 def measured_before(measured_positions: list[np.ndarray], candidate: np.ndarray) -> bool:
368     return any((dist(candidate, p) < MIN_NEW_STATION_SEPARATION for p in measured_positions))
369
370 def problem2_score_existing_station(region: BaseGeometry, observations: list[dict[str, Any]],

```

```

370     measured_positions: list[np.ndarray], station: np.ndarray) -> float:
371     if measured_before(measured_positions, station):
372         return float('inf')
373     if max_distance_to_region(station, region) > ROBUST_RX_LIMIT:
374         return float('inf')
375     try:
376         config = build_q2_config_from_current_region(region, observations)
377         targets = current_q2_targets(config, region)
378     except Exception:
379         return float('inf')
380     (a, b) = global_to_local(config, P2Point(float(station[0]), float(station[1])))
381     c = Candidate(a, b, P2Point(float(station[0]), float(station[1])))
382     return exact_robust_score_current_region(config, c, targets, region)
383
384 def generic_active_candidates(region: BaseGeometry, measured_positions: list[np.ndarray]) -> list[np.
385     ndarray]:
386     mec = minimum_enclosing_circle(region)
387     center = mec.center
388     result: list[np.ndarray] = []
389     radii = (0.0, *Q2_GENERIC_RADII)
390     for rr in radii:
391         n_ang = 1 if rr == 0.0 else Q2_GENERIC_ANGLES
392         for j in range(n_ang):
393             ang = 0.0 if rr == 0.0 else 2.0 * math.pi * j / n_ang
394             p = center + rr * np.array([math.cos(ang), math.sin(ang)])
395             if measured_before(measured_positions, p):
396                 continue
397             if max_distance_to_region(p, region) <= ROBUST_RX_LIMIT:
398                 result.append(p)
399     return deduplicate_points(result)
400
401 def deduplicate_points(points: Iterable[np.ndarray], tol: float=0.0001) -> list[np.ndarray]:
402     seen: set[tuple[int, int]] = set()
403     out: list[np.ndarray] = []
404     scale = 1.0 / tol
405     for p in points:
406         p = np.asarray(p, dtype=float)
407         key = (int(round(float(p[0]) * scale)), int(round(float(p[1]) * scale)))
408         if key not in seen:
409             seen.add(key)
410             out.append(p)
411     return out
412
413 def compute_active_options(region: BaseGeometry, observations: list[dict[str, Any]], measured_positions:
414     list[np.ndarray]) -> list[ActiveOption]:
415     try:
416         config = build_q2_config_from_current_region(region, observations)
417         targets = current_q2_targets(config, region)
418         candidates = candidate_grid(config)
419     except Exception as exc:
420         print(f'[q2] analytic candidate construction fallback: {exc}')
421         candidates = []
422         targets = []
423         config = None
424     valid: list[Candidate] = []
425     if config is not None:
426         for c in candidates:
427             p = c.xy.as_np()
428             if measured_before(measured_positions, p):
429                 continue
430             if config.use_2d_receive_filter and (not passes_sampled_receive_check(config, c, targets)):

```

```

428         continue
429     if max_distance_to_region(p, region) > ROBUST_RX_LIMIT:
430         continue
431     valid.append(c)
432 if config is not None and valid:
433     rough_targets = _downsample_targets(targets, Q2_ROUGH_TARGETS)
434     proxy_scores = [proxy_robust_score(config, c, rough_targets) for c in valid]
435     ranked = sorted(range(len(valid)), key=proxy_scores.__getitem__)
436     keep = ranked[:min(config.top_k_for_exact, len(valid))]
437     exact: list[ActiveOption] = []
438     for idx in keep:
439         score = exact_robust_score_current_region(config, valid[idx], targets, region)
440         if math.isfinite(score):
441             exact.append(ActiveOption(valid[idx].xy.as_np(), float(score)))
442     exact.sort(key=lambda x: x.score)
443     if exact:
444         return exact
445 fallback_points = generic_active_candidates(region, measured_positions)
446 if not fallback_points:
447     return []
448 target_pts = sample_region_points(region, Q2_EXACT_TARGETS)
449 options: list[ActiveOption] = []
450 for p in fallback_points:
451     worst = 0.0
452     for g in target_pts:
453         if dist(p, g) <= CLEAR_RADIUS:
454             scenario = 0.0
455         else:
456             true_deg = angle_deg_from_to(p, g)
457             scenario = 0.0
458             for error_deg in Q2_ERROR_SCENARIOS_DEG:
459                 nr = update_region_direction(region, p, true_deg + error_deg)
460                 if nr.is_empty:
461                     scenario = float('inf')
462                     break
463             scenario = max(scenario, region_diameter(nr))
464     worst = max(worst, scenario)
465     if math.isfinite(worst):
466         options.append(ActiveOption(p, worst))
467 options.sort(key=lambda x: x.score)
468 return options
469 SEARCHING = 'SEARCHING'
470 LOCALIZING = 'LOCALIZING'
471 CLEARABLE = 'CLEARABLE'
472 CLEARED = 'CLEARED'
473 ABSENT = 'ABSENT'
474
475 @dataclass
476 class ChannelState:
477     channel: int
478     status: str = SEARCHING
479     region: BaseGeometry = field(default_factory=initial_arena_region)
480     observations: list[dict[str, Any]] = field(default_factory=list)
481     measured_positions: list[np.ndarray] = field(default_factory=list)
482     ever_detected: bool = False
483     region_version: int = 0
484     mec_center: Optional[np.ndarray] = None
485     mec_radius: float = float('inf')
486     diameter: float = float('inf')
487     active_cache_version: int = -1
488     active_cache: list[ActiveOption] = field(default_factory=list)

```

```

489
490 def invalidate_plan_cache(self) -> None:
491     self.active_cache_version = -1
492     self.active_cache = []
493
494 def recompute_geometry(self) -> None:
495     if self.region.is_empty:
496         self.mec_center = None
497         self.mec_radius = float('inf')
498         self.diameter = 0.0
499         return
500     self.diameter = region_diameter(self.region)
501     mec = minimum_enclosing_circle(self.region)
502     self.mec_center = mec.center
503     self.mec_radius = mec.r
504     if self.ever_detected and self.status not in (CLEARED, ABSENT) and (self.mec_radius <=
CLEAR_MEC_TRIGGER):
505         self.status = CLEARABLE
506     elif self.ever_detected and self.status == CLEARABLE and (self.mec_radius > CLEAR_MEC_TRIGGER):
507         self.status = LOCALIZING
508
509 @dataclass
510 # HTTP 通信、重试、日志、命令行入口与运行状态打印属于工程接口，附录中省略。
511
512 class RouteNode:
513     kind: str
514     point: np.ndarray
515     base_id: Optional[int] = None
516     channel: Optional[int] = None
517
518 def shortest_open_base_route(current: np.ndarray, remaining_base_ids: list[int], base_points: list[np.
ndarray]) -> list[int]:
519     if not remaining_base_ids:
520         return []
521     if len(remaining_base_ids) <= 8:
522         best_perm: Optional[tuple[int, ...]] = None
523         best_len = float('inf')
524         for perm in itertools.permutations(remaining_base_ids):
525             total = dist(current, base_points[perm[0]])
526             for (a, b) in zip(perm[:-1], perm[1:]):
527                 total += dist(base_points[a], base_points[b])
528             if total < best_len:
529                 best_len = total
530                 best_perm = perm
531         assert best_perm is not None
532         return list(best_perm)
533     left = set(remaining_base_ids)
534     route: list[int] = []
535     p = current.copy()
536     while left:
537         nxt = min(left, key=lambda i: dist(p, base_points[i]))
538         route.append(nxt)
539         p = base_points[nxt]
540         left.remove(nxt)
541     return route
542
543 def route_points(route: list[RouteNode]) -> list[np.ndarray]:
544     return [n.point for n in route]
545
546 def best_insertion(current: np.ndarray, route: list[RouteNode], point: np.ndarray) -> tuple[float, int]:
547     if not route:

```

```

548         return (dist(current, point), 0)
549     best_cost = float('inf')
550     best_pos = 0
551     cost = dist(current, point) + dist(point, route[0].point) - dist(current, route[0].point)
552     if cost < best_cost:
553         best_cost = cost
554         best_pos = 0
555     for i in range(len(route) - 1):
556         a = route[i].point
557         b = route[i + 1].point
558         cost = dist(a, point) + dist(point, b) - dist(a, b)
559         if cost < best_cost:
560             best_cost = cost
561             best_pos = i + 1
562     cost = dist(route[-1].point, point)
563     if cost < best_cost:
564         best_cost = cost
565         best_pos = len(route)
566     return (best_cost, best_pos)
567
568 def insert_tasks_greedily(current: np.ndarray, route: list[RouteNode], tasks: list[RouteNode]) -> list[
    RouteNode]:
569     route = list(route)
570     pending = list(tasks)
571     while pending:
572         best = None
573         for (ti, task) in enumerate(pending):
574             (cost, pos) = best_insertion(current, route, task.point)
575             key = (cost, 0 if task.kind == 'CLEAR' else 1)
576             if best is None or key < best[0]:
577                 best = (key, ti, pos)
578         assert best is not None
579         (_, ti, pos) = best
580         task = pending.pop(ti)
581         route.insert(pos, task)
582     return route
583
584 class Problem3Controller:
585
586     def __init__(self, client: RobotClient, outer_count: int, outer_radius: float, rotation_deg: float):
587         self.client = client
588         validate_backbone_or_raise(outer_count, outer_radius)
589         self.base_points = regular_base_points(outer_count=outer_count, outer_radius=outer_radius,
rotation_deg=rotation_deg)
590         self.remaining_base_ids: set[int] = set(range(len(self.base_points)))
591         self.visited_base_ids: set[int] = set()
592         self.channels = {k: ChannelState(channel=k) for k in range(1, 21)}
593         self.cleared_count = 0
594         self.discovered_channels: set[int] = set()
595         self.base_assignments: dict[int, list[int]] = {}
596
597     def searching_channels(self) -> list[int]:
598         return [k for (k, st) in self.channels.items() if st.status == SEARCHING]
599
600     def localizing_channels(self) -> list[int]:
601         return [k for (k, st) in self.channels.items() if st.status == LOCALIZING]
602
603     def clearable_channels(self) -> list[int]:
604         return [k for (k, st) in self.channels.items() if st.status == CLEARABLE]
605
606     def done(self) -> bool:

```

```

607         if self.cleared_count >= 16:
608             return True
609         if self.remaining_base_ids:
610             return False
611         for st in self.channels.values():
612             if st.status == SEARCHING:
613                 st.status = ABSENT
614         return all((st.status in (CLEARED, ABSENT) for st in self.channels.values()))
615
616     def discovered_count(self) -> int:
617         return len(self.discovered_channels)
618
619     def mark_search_complete_by_upper_bound(self) -> None:
620         if self.discovered_count() >= 16:
621             self.remaining_base_ids.clear()
622             for st in self.channels.values():
623                 if st.status == SEARCHING:
624                     st.status = ABSENT
625
626     def _record_measurement(self, st: ChannelState, position: np.ndarray, result: str, angle_deg: Optional[
float]=None) -> None:
627         obs = {'position': [float(position[0]), float(position[1])], 'result': result}
628         if angle_deg is not None:
629             obs['angle_deg'] = float(angle_deg)
630         st.observations.append(obs)
631         st.measured_positions.append(np.asarray(position, dtype=float).copy())
632
633     def _set_region(self, st: ChannelState, new_region: BaseGeometry, reason: str) -> None:
634         new_region = clean_geometry(new_region)
635         if new_region.is_empty and st.ever_detected:
636             print(f'[warning] ch={st.channel:02d} region became empty after {reason}; keep previous
conservative region.')
637             return
638         st.region = new_region
639         st.region_version += 1
640         st.invalidate_plan_cache()
641         if st.status == SEARCHING and st.region.is_empty and (not st.ever_detected):
642             st.status = ABSENT
643         st.recompute_geometry()
644
645     def _mark_detected(self, st: ChannelState) -> None:
646         if not st.ever_detected:
647             st.ever_detected = True
648             self.discovered_channels.add(st.channel)
649         if st.status not in (CLEARED, CLEARABLE):
650             st.status = LOCALIZING
651
652     def handle_measure_response(self, channel: int, position: np.ndarray, response: dict[str, Any]) -> None
:
653         st = self.channels[channel]
654         result = response['measure_result']
655         if result == 'no_signal':
656             self._record_measurement(st, position, 'no_signal')
657             new_region = update_region_no_signal(st.region, position)
658             self._set_region(st, new_region, 'no_signal')
659         elif result == 'direction':
660             angle = float(response['svd_deg'])
661             self._record_measurement(st, position, 'direction', angle)
662             self._mark_detected(st)
663             new_region = update_region_direction(st.region, position, angle)
664             self._set_region(st, new_region, 'direction')

```

```

665         elif result == 'near':
666             self._record_measurement(st, position, 'near')
667             self._mark_detected(st)
668             clear_response = self.client.clear(position, channel)
669             self.handle_clear_response(channel, position, clear_response)
670         else:
671             raise RuntimeError(f'未知 measure_result={result!r}')
672         self.mark_search_complete_by_upper_bound()
673
674     def handle_clear_response(self, channel: int, position: np.ndarray, response: dict[str, Any]) -> None:
675         st = self.channels[channel]
676         result = response['clear_result']
677         if result == 'success':
678             if st.status != CLEARED:
679                 self.cleared_count += 1
680                 st.status = CLEARED
681                 st.ever_detected = True
682                 self.discovered_channels.add(channel)
683                 print(f'[state] ch={channel:02d} CLEARED ({self.cleared_count} total)')
684             elif result == 'no_target_in_range':
685                 new_region = update_region_clear_failure(st.region, position)
686                 self._set_region(st, new_region, 'clear_failure')
687                 st.status = LOCALIZING
688                 print(f'[state] ch={channel:02d} clear failed -> LOCALIZING')
689             else:
690                 raise RuntimeError(f'未知 clear_result={result!r}')
691             self.mark_search_complete_by_upper_bound()
692
693     def _remaining_base_route(self) -> list[int]:
694         return shortest_open_base_route(current=self.client.position, remaining_base_ids=sorted(self.
695 remaining_base_ids), base_points=self.base_points)
696
697     def _base_score(self, st: ChannelState, base_id: int) -> float:
698         p = self.base_points[base_id]
699         if measured_before(st.measured_positions, p):
700             return float('inf')
701         if max_distance_to_region(p, st.region) > ROBUST_RX_LIMIT:
702             return float('inf')
703         return problem2_score_existing_station(region=st.region, observations=st.observations,
704 measured_positions=st.measured_positions, station=p)
705
706     def _active_options(self, st: ChannelState) -> list[ActiveOption]:
707         if st.active_cache_version == st.region_version and st.active_cache:
708             return st.active_cache
709         options = compute_active_options(region=st.region, observations=st.observations, measured_positions
710 =st.measured_positions)
711         st.active_cache_version = st.region_version
712         st.active_cache = options
713         return options
714
715     def _choose_active_point_for_route(self, st: ChannelState, base_route_nodes: list[RouteNode]) ->
716 Optional[ActiveOption]:
717         options = self._active_options(st)
718         if not options:
719             return None
720         best_score = options[0].score
721         threshold = best_score * (1.0 + ACTIVE_NEAR_OPT_REL) + 1e-09
722         near_opt = [o for o in options if o.score <= threshold]
723         clear_points: list[np.ndarray] = []
724         for clear_ch in self.clearable_channels():
725             clear_st = self.channels[clear_ch]

```

```

722         clear_st.recompute_geometry()
723         if clear_st.mec_center is not None and clear_st.mec_radius <= CLEAR_MEC_TRIGGER:
724             clear_points.append(clear_st.mec_center.copy())
725     best: Optional[tuple[float, float, ActiveOption]] = None
726     for opt in near_opt:
727         (insertion, _) = best_insertion(self.client.position, base_route_nodes, opt.point)
728         if clear_points:
729             nearest_clear = min((dist(opt.point, clear_point) for clear_point in clear_points))
730         else:
731             nearest_clear = 0.0
732         route_score = insertion + ACTIVE_CLEAR_BIAS * nearest_clear
733         key = (route_score, opt.score, opt)
734         if best is None or key[:2] < best[:2]:
735             best = key
736     assert best is not None
737     return best[2]
738
739 def _base_overlap_ratio(self, st: ChannelState, point: np.ndarray) -> float:
740     if st.region.is_empty():
741         return 0.0
742     area = float(st.region.area)
743     if area <= 1e-09:
744         return 1.0 if max_distance_to_region(point, st.region) <= ROBUST_RX_LIMIT else 0.0
745     inside = st.region.intersection(disk(point, NO_SIGNAL_REMOVE_RADIUS))
746     return float(inside.area / area)
747
748 def _opportunistic_base_channels(self, base_id: int, already_scheduled: Iterable[int]) -> list[int]:
749     p = self.base_points[base_id]
750     excluded = set((int(x) for x in already_scheduled))
751     scored: list[tuple[float, float, int]] = []
752     for ch in self.localizing_channels():
753         if ch in excluded:
754             continue
755         st = self.channels[ch]
756         if measured_before(st.measured_positions, p):
757             continue
758         ratio = self._base_overlap_ratio(st, p)
759         if ratio < BASE_OPPORTUNISTIC_MIN_OVERLAP:
760             continue
761         scored.append((-ratio, st.mec_radius, ch))
762     scored.sort()
763     selected = [ch for (_, _, ch) in scored[:BASE_OPPORTUNISTIC_MAX_CHANNELS]]
764     return selected
765
766 def plan(self) -> list[RouteNode]:
767     self.mark_search_complete_by_upper_bound()
768     base_ids = self._remaining_base_route()
769     base_route = [RouteNode(kind='BASE', point=self.base_points[i], base_id=i) for i in base_ids]
770     self.base_assignments = {i: [] for i in base_ids}
771     active_tasks: list[RouteNode] = []
772     for ch in self.localizing_channels():
773         st = self.channels[ch]
774         st.recompute_geometry()
775         if st.status == CLEARABLE:
776             continue
777     base_scores: list[tuple[float, int]] = []
778     for base_id in base_ids:
779         score = self._base_score(st, base_id)
780         if math.isfinite(score):
781             base_scores.append((score, base_id))
782     base_scores.sort(key=lambda x: x[0])

```

```

783     active_options = self._active_options(st)
784     active_best_score = active_options[0].score if active_options else float('inf')
785     use_base = False
786     selected_base: Optional[int] = None
787     if base_scores:
788         best_base_score = base_scores[0][0]
789         comparable_to_active = not math.isfinite(active_best_score) or best_base_score <=
BASE_REUSE_FACTOR * active_best_score
790         current_d = max(st.diameter, 1e-09)
791         meaningful_reduction = best_base_score <= (1.0 - BASE_REUSE_MIN_REDUCTION) * current_d
792         if comparable_to_active or meaningful_reduction:
793             best_value = best_base_score
794             candidate_ids = {bid for (score, bid) in base_scores if score <= 1.12 * best_value + 1e
-09}
795             for bid in base_ids:
796                 if bid in candidate_ids:
797                     selected_base = bid
798                     break
799             if selected_base is not None:
800                 use_base = True
801         if use_base and selected_base is not None:
802             self.base_assignments[selected_base].append(ch)
803             print(f'[plan] ch={ch:02d} reuse BASE {selected_base}')
804             continue
805         opt = self._choose_active_point_for_route(st, base_route)
806         if opt is not None:
807             active_tasks.append(RouteNode(kind='ACTIVE', point=opt.point, channel=ch))
808             print(f'[plan] ch={ch:02d} ACTIVE score={opt.score:.2f}')
809         else:
810             mec = minimum_enclosing_circle(st.region)
811             p = mec.center
812             if not measured_before(st.measured_positions, p):
813                 active_tasks.append(RouteNode(kind='ACTIVE', point=p, channel=ch))
814                 print(f'[plan] ch={ch:02d} fallback ACTIVE at MEC center')
815         clear_tasks: list[RouteNode] = []
816         for ch in self.clearable_channels():
817             st = self.channels[ch]
818             st.recompute_geometry()
819             if st.mec_center is not None and st.mec_radius <= CLEAR_MEC_TRIGGER:
820                 clear_tasks.append(RouteNode(kind='CLEAR', point=st.mec_center, channel=ch))
821         core_route = insert_tasks_greedily(current=self.client.position, route=base_route, tasks=
active_tasks)
822         if core_route:
823             insert_now: list[RouteNode] = []
824             deferred: list[tuple[float, float, float, RouteNode]] = []
825             for task in clear_tasks:
826                 (extra_dist, _) = best_insertion(self.client.position, core_route, task.point)
827                 current_dist = dist(self.client.position, task.point)
828                 future_nearest = min((dist(task.point, node.point) for node in core_route))
829                 on_route = extra_dist <= CLEAR_ROUTE_INSERT_MAX_M
830                 escaping_region = current_dist <= CLEAR_ESCAPE_CURRENT_MAX_M and future_nearest >=
CLEAR_ESCAPE_FUTURE_MIN_M
831                 if on_route:
832                     insert_now.append(task)
833                     print(f'[plan] ch={task.channel:02d} CLEAR on-route (extra={extra_dist:.1f}m, current={
current_dist:.1f}m, future_nearest={future_nearest:.1f}m)')
834                 elif escaping_region:
835                     insert_now.append(task)
836                     print(f'[plan] ch={task.channel:02d} CLEAR before-leaving-region (extra={extra_dist:.1f
}m, current={current_dist:.1f}m, future_nearest={future_nearest:.1f}m)')
837                 else:

```

```

838         deferred.append((extra_dist, current_dist, future_nearest, task))
839     if len(deferred) > MAX_DEFERRED_CLEAR:
840         deferred.sort(key=lambda x: (x[0], x[1]))
841         force_count = len(deferred) - MAX_DEFERRED_CLEAR
842         forced = deferred[:force_count]
843         deferred = deferred[force_count:]
844         for (extra_dist, current_dist, future_nearest, task) in forced:
845             insert_now.append(task)
846             print(f'[plan] ch={task.channel:02d} CLEAR forced-by-backlog (extra={extra_dist:.1f}m,
current={current_dist:.1f}m, future_nearest={future_nearest:.1f}m, max_deferred={MAX_DEFERRED_CLEAR})'
)
847         for (extra_dist, current_dist, future_nearest, task) in deferred:
848             print(f'[plan] ch={task.channel:02d} CLEAR deferred (extra={extra_dist:.1f}m, current={
current_dist:.1f}m, future_nearest={future_nearest:.1f}m)')
849             route = insert_tasks_greedily(current=self.client.position, route=core_route, tasks=insert_now)
850         else:
851             route = insert_tasks_greedily(current=self.client.position, route=[], tasks=clear_tasks)
852     return route
853
854 def _measurement_order(self, channels: Iterable[int]) -> list[int]:
855     unique = sorted(set((int(x) for x in channels)))
856     if self.client.current_channel in unique:
857         unique.remove(self.client.current_channel)
858     return [self.client.current_channel] + unique
859     return unique
860
861 def visit_base(self, base_id: int) -> None:
862     p = self.base_points[base_id]
863     search_tasks = self.searching_channels()
864     localize_tasks = [ch for ch in self.base_assignments.get(base_id, []) if self.channels[ch].status
== LOCALIZING]
865     opportunistic_tasks = self._opportunistic_base_channels(base_id, already_scheduled=search_tasks +
localize_tasks)
866     tasks = self._measurement_order(search_tasks + localize_tasks + opportunistic_tasks)
867     print(f'[base] visit B{base_id}, search={len(search_tasks)}, reuse_localize={localize_tasks},
opportunistic={opportunistic_tasks}')
868     for ch in tasks:
869         if self.client.real_time_remaining() <= REAL_TIME_SAFETY_MARGIN_S:
870             raise TimeoutError('现实剩余时间不足，停止新的检测动作。')
871         st = self.channels[ch]
872         if st.status in (CLEARED, ABSENT):
873             continue
874         if st.status == SEARCHING and self.discovered_count() >= 16:
875             st.status = ABSENT
876             continue
877         resp = self.client.measure(p, ch)
878         self.handle_measure_response(ch, p, resp)
879         self.remaining_base_ids.discard(base_id)
880         self.visited_base_ids.add(base_id)
881         if not self.remaining_base_ids:
882             for st in self.channels.values():
883                 if st.status == SEARCHING:
884                     st.status = ABSENT
885
886 def execute_active(self, node: RouteNode) -> None:
887     assert node.channel is not None
888     ch = node.channel
889     st = self.channels[ch]
890     if st.status != LOCALIZING:
891         return
892     resp = self.client.measure(node.point, ch)

```

```

893         self.handle_measure_response(ch, node.point, resp)
894
895     def execute_clear(self, node: RouteNode) -> None:
896         assert node.channel is not None
897         ch = node.channel
898         st = self.channels[ch]
899         if st.status != CLEARABLE:
900             return
901         resp = self.client.clear(node.point, ch)
902         self.handle_clear_response(ch, node.point, resp)
903
904     def execute_node(self, node: RouteNode) -> None:
905         if node.kind == 'BASE':
906             assert node.base_id is not None
907             self.visit_base(node.base_id)
908         elif node.kind == 'ACTIVE':
909             self.execute_active(node)
910         elif node.kind == 'CLEAR':
911             self.execute_clear(node)
912         else:
913             raise ValueError(f'unknown node kind={node.kind}')

```

附录 B.4 问题四：混合信源信念与价值驱动决策

```

1  from __future__ import annotations
2
3  import math
4  import time
5  from dataclasses import dataclass
6  from typing import Any, Optional
7
8  import numpy as np
9  import problem3 as q3
10
11  # 第三问的区域更新、基准搜索路径、插入式路径规划和通信接口均直接复用。
12
13  try:
14      import problem3 as q3
15  except ModuleNotFoundError as exc:
16      if exc.name != 'problem3_fused_final':
17          raise
18      raise SystemExit('缺少同目录 problem3_fused_final.py, 无法复用问题 3 的接口与几何实现。') from exc
19  DEFERRED = 'DEFERRED'
20  SEARCH_DEFERRED = 'SEARCH_DEFERRED'
21  PARTICLE_COUNT = 1500
22  PARTICLE_REBUILD_FACTOR = 5
23  PARTICLE_SOFT_LIKELIHOOD = 1e-06
24  PARTICLE_RESAMPLE_ESS_RATIO = 0.42
25  INITIAL_PRESENCE_PRIOR = 13.0 / 20.0
26  INITIAL_DIRECTIONAL_PRIOR = 0.5
27  RANDOM_SEED = 20260911
28  BEARING_BIN_DEG = 4.0
29  DIRECTION_UNCERTAINTY_SCALE_M = 260.0
30  TYPE_UNCERTAINTY_SCALE_M = 120.0
31  OBJECTIVE_ALPHA = 0.86
32  AVERAGE_TIME_REFERENCE_S = 600.0
33  INFORMATION_REWARD = 0.28
34  DISCOVERY_REWARD = 0.9
35  BASE_PROGRESS_BONUS = 0.2
36  CLEAR_ACTION_BONUS = 0.8

```

```

37 MAX_DIRECTIONAL_CANDIDATES = 20
38 MAX_ACTIVE_MEASUREMENTS = 6
39 MAX_ACTIVE_AFTER_DISCOVERY = 6
40 EXACT_CHANNELS_PER_ROUND = 5
41 PLAN_TIME_BUDGET_S = 15.0
42 FAST_FINISH_REMAINING_S = 60.0
43 FINAL_EXIT_REMAINING_S = 20.0
44 MIN_MEASUREMENTS_BEFORE_DEFER = 5
45 MIN_INFORMATION_GAIN = 0.015
46 MIN_ACTIVE_VALUE_RATE = 2.5e-05
47 REACTIVATE_DISTANCE_M = 420.0
48 FOLLOWUP_ACTION_TIME_S = 90.0
49 MAX_OPPORTUNISTIC_AT_BASE = 2
50 ACTIVE_Q2_NEAR_OPT_REL = 0.16
51 SOFT_INSERTION_LIMIT_M = 1000.0
52 HARD_INSERTION_LIMIT_M = 1800.0
53 MIN_MEC_REDUCTION_RATIO = 0.005
54 MAX_STAGNANT_ACTIVE_STEPS = 4
55 MIN_REQUIRED_DISCOVERIES = 10
56 MAX_POSSIBLE_SOURCES = 16
57 DISCOVERY_COVERAGE_TARGET = 0.975
58 HIGH_RISK_COVERAGE_TARGET = 0.985
59 HIGH_RISK_PRESENCE_THRESHOLD = 0.1
60 EXPECTED_UNSEEN_STOP_THRESHOLD = 0.3
61 MIN_DISTINCT_BEARING_SECTORS = 4
62 BEARING_SECTOR_COUNT = 8
63 MAX_BACKBONE_PROBE_ACTIONS = 3
64 MAX_POST_BACKBONE_PROBE_ACTIONS = 8
65 MAX_EMERGENCY_PROBE_ACTIONS = 6
66 MAX_SUPPLEMENT_ACTIONS = MAX_BACKBONE_PROBE_ACTIONS + MAX_POST_BACKBONE_PROBE_ACTIONS +
    MAX_EMERGENCY_PROBE_ACTIONS
67 BACKBONE_PROBE_INSERTION_LIMIT_M = 400.0
68 POST_PROBE_INSERTION_LIMIT_M = 2200.0
69 EMERGENCY_PROBE_INSERTION_LIMIT_M = 3600.0
70 MAX_CHANNELS_PER_SUPPLEMENT = 20
71 MIN_SUPPLEMENT_VALUE_RATE = 0.0
72 UNSEEN_PRESENCE_FLOOR = 0.06
73 SUPPLEMENT_SENSE_BUDGET_S = 125.0
74 SUPPLEMENT_TOTAL_BUDGET_S = 3600.0
75 SUPPLEMENT_DETOUR_BUDGET_M = 12000.0
76 SUPPLEMENT_CLEAR_PROBABILITY = 0.78
77 EMERGENCY_EXPECTED_UNSEEN_THRESHOLD = 0.45
78
79 def circular_difference_deg(a: np.ndarray, b: float) -> np.ndarray:
80     return (a - float(b) + 180.0) % 360.0 - 180.0
81
82 def binary_entropy(probability: float) -> float:
83     p = min(1.0, max(0.0, float(probability)))
84     if p <= 1e-12 or p >= 1.0 - 1e-12:
85         return 0.0
86     return -p * math.log(p) - (1.0 - p) * math.log(1.0 - p)
87
88 def systematic_resample(weights: np.ndarray, rng: np.random.Generator, output_count: Optional[int]=None) ->
    np.ndarray:
89     n = len(weights) if output_count is None else int(output_count)
90     positions = (rng.random() + np.arange(n, dtype=float)) / n
91     cumulative = np.cumsum(weights)
92     cumulative[-1] = 1.0
93     return np.searchsorted(cumulative, positions, side='left')
94
95 def weighted_mean(points: np.ndarray, weights: np.ndarray) -> np.ndarray:

```

```

96     return np.sum(points * weights[:, None], axis=0)
97
98 def weighted_rms_radius(points: np.ndarray, weights: np.ndarray) -> float:
99     if len(points) == 0:
100         return float('inf')
101     center = weighted_mean(points, weights)
102     squared = np.sum((points - center) ** 2, axis=1)
103     return float(math.sqrt(max(0.0, float(np.dot(weights, squared)))))
104
105 def mixed_position_region(observations: list[dict[str, Any]]) -> Any:
106     region = q3.initial_arena_region()
107     for obs in observations:
108         point = np.asarray(obs['position'], dtype=float)
109         result = obs['result']
110         if result == 'direction':
111             region = q3.update_region_direction(region, point, float(obs['angle_deg']))
112         elif result == 'near':
113             near_disk = q3.disk(point, q3.NEAR_RADIUS + q3.ARENA_MODEL_MARGIN)
114             region = q3.clean_geometry(region.intersection(near_disk))
115         elif result == 'clear_failure':
116             region = q3.update_region_clear_failure(region, point)
117         elif result == 'no_signal':
118             continue
119         else:
120             raise ValueError(f'未知历史观测类型: {result!r}')
121         if region.is_empty:
122             break
123     return q3.clean_geometry(region)
124
125 def guarantees_clearance(state: q3.ChannelState, point: np.ndarray) -> bool:
126     if state.region.is_empty:
127         return False
128     return q3.max_distance_to_region(np.asarray(point, dtype=float), state.region) <= q3.CLEAR_MEC_TRIGGER
129         + 1e-07
130
131 @dataclass(frozen=True)
132 class BeliefPrediction:
133     signal_probability: float
134     information_gain: float
135     spatial_gain: float
136     expected_spatial_radius: float
137     outcome_entropy: float
138
139 class MixedSourceBelief:
140
141     def __init__(self, channel: int, rng: np.random.Generator, count: int=PARTICLE_COUNT) -> None:
142         self.channel = int(channel)
143         self.rng = rng
144         self.count = int(count)
145         self.present_probability = INITIAL_PRESENCE_PRIOR
146         self.positions = np.empty((0, 2), dtype=float)
147         self.ranges = np.empty(0, dtype=float)
148         self.is_directional = np.empty(0, dtype=bool)
149         self.directions = np.empty(0, dtype=float)
150         self.weights = np.empty(0, dtype=float)
151         self._initialize_prior()
152
153     def _initialize_prior(self) -> None:
154         radius = q3.TARGET_RADIUS * np.sqrt(self.rng.random(self.count))
155         angle = self.rng.uniform(0.0, 2.0 * math.pi, self.count)
156         self.positions = np.column_stack((radius * np.cos(angle), radius * np.sin(angle)))

```

```

156         self._draw_latent(self.count)
157         self.weights = np.full(self.count, 1.0 / self.count, dtype=float)
158
159     def _draw_latent(self, count: int) -> None:
160         self.ranges = self.rng.uniform(q3.SIGNAL_RADIUS_MIN, q3.SIGNAL_RADIUS_MAX, count)
161         self.is_directional = self.rng.random(count) < INITIAL_DIRECTIONAL_PRIOR
162         self.directions = self.rng.uniform(0.0, 2.0 * math.pi, count)
163
164     def _signal_mask(self, station: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
165         station = np.asarray(station, dtype=float)
166         delta = station[None, :] - self.positions
167         distances = np.linalg.norm(delta, axis=1)
168         front_dot = delta[:, 0] * np.cos(self.directions) + delta[:, 1] * np.sin(self.directions)
169         angular_visible = ~self.is_directional | (front_dot >= 0.0)
170         return ((distances <= self.ranges) & angular_visible, distances)
171
172     def _likelihood(self, observation: dict[str, Any]) -> np.ndarray:
173         station = np.asarray(observation['position'], dtype=float)
174         (signal, distances) = self._signal_mask(station)
175         result = observation['result']
176         if result == 'no_signal':
177             valid = ~signal
178         elif result == 'near':
179             valid = signal & (distances <= q3.NEAR_RADIUS + q3.ARENA_MODEL_MARGIN)
180         elif result == 'clear_failure':
181             valid = distances > q3.CLEAR_FAILURE_REMOVE_RADIUS
182         elif result == 'direction':
183             predicted = np.degrees(np.arctan2(self.positions[:, 1] - station[1], self.positions[:, 0] -
184 station[0])) % 360.0
185             angular_ok = np.abs(circular_difference_deg(predicted, float(observation['angle_deg']))) <= q3.
186 BEARING_ERROR_DEG + q3.ANGLE_MODEL_MARGIN_DEG
187             valid = signal & (distances > q3.NEAR_RADIUS) & angular_ok
188         else:
189             raise ValueError(f'未知观测 {result!r}')
190         return np.where(valid, 1.0, PARTICLE_SOFT_LIKELIHOOD)
191
192     def _normalize_and_resample(self) -> None:
193         total = float(np.sum(self.weights))
194         if not math.isfinite(total) or total <= 0.0:
195             self.weights = np.full(self.count, 1.0 / self.count, dtype=float)
196             return
197         self.weights /= total
198         ess = 1.0 / max(float(np.dot(self.weights, self.weights)), 1e-15)
199         if ess < PARTICLE_RESAMPLE_ESS_RATIO * self.count:
200             indices = systematic_resample(self.weights, self.rng)
201             self.positions = self.positions[indices].copy()
202             self.ranges = self.ranges[indices].copy()
203             self.is_directional = self.is_directional[indices].copy()
204             self.directions = self.directions[indices].copy()
205             self.weights = np.full(self.count, 1.0 / self.count, dtype=float)
206
207     def _sample_positions_in_region(self, region: Any, wanted: int) -> np.ndarray:
208         if region.is_empty:
209             raise ValueError('不能从空位置区域重建粒子')
210         (min_x, min_y, max_x, max_y) = region.bounds
211         accepted: list[tuple[float, float]] = []
212         attempts = 0
213         maximum_attempts = max(50000, 220 * wanted)
214         batch = max(2000, 5 * wanted)
215         while len(accepted) < wanted and attempts < maximum_attempts:
216             xs = self.rng.uniform(min_x, max_x, batch)

```

```

215         ys = self.rng.uniform(min_y, max_y, batch)
216         attempts += batch
217         for (x, y) in zip(xs, ys):
218             if region.covers(q3.Point(float(x), float(y))):
219                 accepted.append((float(x), float(y)))
220                 if len(accepted) >= wanted:
221                     break
222         if not accepted:
223             representative = region.representative_point()
224             accepted = [(float(representative.x), float(representative.y))]
225         if len(accepted) < wanted:
226             boundary = q3.sample_region_points(region, min(wanted, 240))
227             if len(boundary):
228                 accepted.extend(((float(x), float(y)) for (x, y) in boundary))
229         values = np.asarray(accepted, dtype=float)
230         if len(values) < wanted:
231             indices = self.rng.integers(0, len(values), size=wanted - len(values))
232             values = np.vstack((values, values[indices]))
233         return values[:wanted]
234
235     def rebuild(self, region: Any, observations: list[dict[str, Any]]) -> None:
236         candidate_count = max(self.count, PARTICLE_REBUILD_FACTOR * self.count)
237         self.positions = self._sample_positions_in_region(region, candidate_count)
238         self._draw_latent(candidate_count)
239         self.weights = np.full(candidate_count, 1.0 / candidate_count, dtype=float)
240         for obs in observations:
241             self.weights *= self._likelihood(obs)
242         total = float(np.sum(self.weights))
243         if not math.isfinite(total) or total <= 0.0:
244             self.weights.fill(1.0 / candidate_count)
245         else:
246             self.weights /= total
247         indices = systematic_resample(self.weights, self.rng, self.count)
248         self.positions = self.positions[indices].copy()
249         self.ranges = self.ranges[indices].copy()
250         self.is_directional = self.is_directional[indices].copy()
251         self.directions = self.directions[indices].copy()
252         self.weights = np.full(self.count, 1.0 / self.count, dtype=float)
253
254     def observe(self, observation: dict[str, Any], known_present: bool) -> None:
255         likelihood = self._likelihood(observation)
256         if observation['result'] == 'no_signal' and (not known_present):
257             probability_no_signal_if_present = float(np.dot(self.weights, likelihood))
258             prior = self.present_probability
259             denominator = 1.0 - prior + prior * probability_no_signal_if_present
260             if denominator > 0.0:
261                 self.present_probability = prior * probability_no_signal_if_present / denominator
262         elif observation['result'] in ('direction', 'near'):
263             self.present_probability = 1.0
264         self.weights *= likelihood
265         self._normalize_and_resample()
266
267     @property
268     def directional_probability(self) -> float:
269         return float(np.dot(self.weights, self.is_directional.astype(float)))
270
271     @property
272     def effective_sample_size(self) -> float:
273         return 1.0 / max(float(np.dot(self.weights, self.weights)), 1e-15)
274
275     def direction_estimate_deg(self) -> Optional[float]:

```

```

276     mask = self.is_directional
277     directional_mass = float(np.sum(self.weights[mask]))
278     if directional_mass <= 1e-09:
279         return None
280     weights = self.weights[mask] / directional_mass
281     z = np.dot(weights, np.exp(1j * self.directions[mask]))
282     if abs(z) <= 1e-09:
283         return None
284     return float(math.degrees(math.atan2(z.imag, z.real)) % 360.0)
285
286 def signal_probability(self, station: np.ndarray) -> float:
287     (signal, _) = self._signal_mask(np.asarray(station, dtype=float))
288     return float(np.dot(self.weights, signal.astype(float)))
289
290 def directional_signal_probability(self, station: np.ndarray) -> float:
291     (signal, _) = self._signal_mask(np.asarray(station, dtype=float))
292     directional_mass = float(np.sum(self.weights[self.is_directional]))
293     if directional_mass <= 1e-12:
294         return 0.0
295     detected_mass = float(np.sum(self.weights[self.is_directional & signal]))
296     return min(1.0, max(0.0, detected_mass / directional_mass))
297
298 def position_center(self) -> np.ndarray:
299     return weighted_mean(self.positions, self.weights)
300
301 def spatial_radius(self) -> float:
302     return weighted_rms_radius(self.positions, self.weights)
303
304 def _joint_uncertainty(self, indices: np.ndarray, weights: np.ndarray) -> float:
305     conditional = weights / max(float(np.sum(weights)), 1e-15)
306     spatial = weighted_rms_radius(self.positions[indices], conditional)
307     is_dir = self.is_directional[indices]
308     q_dir = float(np.dot(conditional, is_dir.astype(float)))
309     type_term = TYPE_UNCERTAINTY_SCALE_M * binary_entropy(q_dir)
310     direction_term = 0.0
311     if q_dir > 1e-08 and np.any(is_dir):
312         dweights = conditional[is_dir]
313         dweights /= np.sum(dweights)
314         resultant = abs(np.dot(dweights, np.exp(1j * self.directions[indices][is_dir])))
315         circular_variance = max(0.0, 1.0 - float(resultant))
316         direction_term = DIRECTION_UNCERTAINTY_SCALE_M * q_dir * circular_variance
317     return spatial + type_term + direction_term
318
319 def predict(self, station: np.ndarray) -> BeliefPrediction:
320     station = np.asarray(station, dtype=float)
321     (signal, distances) = self._signal_mask(station)
322     bearings = np.degrees(np.arctan2(self.positions[:, 1] - station[1], self.positions[:, 0] - station
323 [0])) % 360.0
324     labels = np.full(self.count, -1, dtype=int)
325     labels[signal & (distances <= q3.NEAR_RADIUS)] = 1000
326     directional_signal = signal & (distances > q3.NEAR_RADIUS)
327     labels[directional_signal] = np.floor(bearings[directional_signal] / BEARING_BIN_DEG).astype(int)
328     all_indices = np.arange(self.count)
329     prior_joint = self._joint_uncertainty(all_indices, self.weights)
330     prior_spatial = self.spatial_radius()
331     expected_joint = 0.0
332     expected_spatial = 0.0
333     outcome_entropy = 0.0
334     for label in np.unique(labels):
335         indices = np.flatnonzero(labels == label)
336         mass = float(np.sum(self.weights[indices]))

```

```

336         if mass <= 0.0:
337             continue
338         outcome_entropy -= mass * math.log(mass)
339         expected_joint += mass * self._joint_uncertainty(indices, self.weights[indices])
340         conditional = self.weights[indices] / mass
341         expected_spatial += mass * weighted_rms_radius(self.positions[indices], conditional)
342         information = max(0.0, 1.0 - expected_joint / max(prior_joint, 1e-09))
343         spatial_gain = max(0.0, 1.0 - expected_spatial / max(prior_spatial, 1e-09))
344         return BeliefPrediction(signal_probability=float(np.dot(self.weights, signal.astype(float))),
            information_gain=min(1.0, information), spatial_gain=min(1.0, spatial_gain), expected_spatial_radius=
            expected_spatial, outcome_entropy=outcome_entropy)
345
346 @dataclass(frozen=True)
347 class CandidateEvaluation:
348     channel: int
349     point: np.ndarray
350     prediction: BeliefPrediction
351     action_time_s: float
352     expected_clear_probability: float
353     expected_remaining_cost_s: float
354     delta_utility: float
355     value_per_second: float
356     insertion_distance_m: float
357
358 @dataclass(frozen=True)
359 class P4Action:
360     kind: str
361     point: np.ndarray
362     score: float
363     channel: Optional[int] = None
364     channels: tuple[int, ...] = ()
365     base_id: Optional[int] = None
366     note: str = ''
367     insertion_distance_m: float = 0.0
368     estimated_time_s: float = 0.0
369     probe_phase: str = ''
370
371 def as_q3_route_nodes(actions: list[P4Action]) -> list[q3.RouteNode]:
372     return [q3.RouteNode(kind=action.kind, point=np.asarray(action.point, dtype=float), base_id=action.
        base_id, channel=action.channel) for action in actions]
373
374 def route_insertion(current: np.ndarray, route: list[P4Action], point: np.ndarray) -> tuple[float, int]:
375     return q3.best_insertion(np.asarray(current, dtype=float), as_q3_route_nodes(route), np.asarray(point,
        dtype=float))
376
377 def insert_actions_by_minimum_increment(current: np.ndarray, base_route: list[P4Action], tasks: list[
    P4Action]) -> list[P4Action]:
378     route = list(base_route)
379     pending = list(tasks)
380     priority = {'CLEAR': 0, 'ACTIVE': 1, 'PROBE': 2, 'BASE': 3}
381     while pending:
382         best: Optional[tuple[tuple[float, int, float], int, int]] = None
383         for (task_index, task) in enumerate(pending):
384             (extra, insert_at) = route_insertion(current, route, task.point)
385             key = (extra, priority.get(task.kind, 9), -task.score)
386             if best is None or key < best[0]:
387                 best = (key, task_index, insert_at)
388             assert best is not None
389             (_, task_index, insert_at) = best
390             route.insert(insert_at, pending.pop(task_index))
391     return route

```

```

392
393 def expected_delta_utility(cleared: int, elapsed: float, estimated_total: float, success_probability: float
    , added_cost: float) -> float:
394     alpha = OBJECTIVE_ALPHA
395     beta = 1.0 - alpha
396     n = int(cleared)
397     p = min(1.0, max(0.0, float(success_probability)))
398     cost = max(0.0, float(added_cost))
399     total = max(1.0, float(estimated_total))
400     if n <= 0:
401         return alpha * p / total - beta * cost / AVERAGE_TIME_REFERENCE_S
402     average = elapsed / n
403     average_term = p * (cost - average) / (n + 1.0) + (1.0 - p) * cost / n
404     return alpha * p / total - beta * average_term / AVERAGE_TIME_REFERENCE_S
405
406 def estimate_followup(state: q3.ChannelState, prediction: BeliefPrediction, first_action_time: float) ->
    tuple[float, float]:
407     current_radius = max(state.mec_radius, q3.CLEAR_RADIUS + 1e-06)
408     combined_gain = max(0.04, min(0.85, 0.62 * prediction.spatial_gain + 0.38 * prediction.information_gain
    ))
409     if current_radius <= q3.CLEAR_MEC_TRIGGER:
410         steps = 0
411     else:
412         ratio = q3.CLEAR_MEC_TRIGGER / current_radius
413         steps = int(math.ceil(math.log(max(ratio, 1e-06)) / math.log(1.0 - combined_gain)))
414         steps = min(MAX_ACTIVE_MEASUREMENTS, max(1, steps))
415     p_vis = prediction.signal_probability
416     eventual = 1.0 - (1.0 - p_vis) ** max(1, steps)
417     information_rescue = (1.0 - eventual) * prediction.information_gain
418     p_clear = min(0.995, max(0.02, eventual + 0.55 * information_rescue))
419     remaining_cost = first_action_time + max(0, steps - 1) * FOLLOWUP_ACTION_TIME_S + 5.0
420     return (p_clear, remaining_cost)
421
422 class Problem4Controller(q3.Problem3Controller):
423
424     def __init__(self, client: q3.RobotClient, outer_count: int, outer_radius: float, rotation_deg: float,
        seed: int=RANDOM_SEED) -> None:
425         super().__init__(client, outer_count, outer_radius, rotation_deg)
426         root_rng = np.random.default_rng(seed)
427         self.beliefs = {channel: MixedSourceBelief(channel, np.random.default_rng(int(root_rng.integers(0,
        2 ** 32 - 1)))) for channel in self.channels}
428         self.supplement_points = self._make_supplement_points(outer_radius)
429         self.emergency_points = self._make_emergency_points()
430         self.supplement_actions_done = 0
431         self.backbone_probe_actions_done = 0
432         self.post_backbone_probe_actions_done = 0
433         self.emergency_probe_actions_done = 0
434         self.supplement_virtual_spent_s = 0.0
435         self.supplement_detour_spent_m = 0.0
436         self.discovery_survival_probability = {channel: 1.0 for channel in self.channels}
437         self.exploration_exhausted = False
438         self.deferred_reasons: dict[int, str] = {}
439         self.active_attempts = {channel: 0 for channel in self.channels}
440         self.stagnant_active_steps = {channel: 0 for channel in self.channels}
441         self.last_active_mec_radius = {channel: float('inf') for channel in self.channels}
442         self.exact_age = {channel: 0 for channel in self.channels}
443         self.reactivation_count = {channel: 0 for channel in self.channels}
444         self.plan_durations_s: list[float] = []
445         self.fast_finish_mode = False
446
447     @staticmethod

```

```

448 def _make_supplement_points(outer_radius: float) -> list[np.ndarray]:
449     points: list[np.ndarray] = []
450     inner_radius = min(650.0, 0.58 * float(outer_radius))
451     middle_radius = float(outer_radius)
452     outer_ring_radius = q3.TARGET_RADIUS
453     for (radius, count, offset) in ((inner_radius, 6, 30.0), (middle_radius, 4, 45.0), (
454         outer_ring_radius, 6, 0.0)):
455         for index in range(count):
456             theta = math.radians(offset + 360.0 * index / count)
457             points.append(np.array([radius * math.cos(theta), radius * math.sin(theta)]))
458     return q3.deduplicate_points(points, tol=1e-05)
459
460 @staticmethod
461 def _make_emergency_points() -> list[np.ndarray]:
462     points: list[np.ndarray] = []
463     for index in range(12):
464         theta = math.radians(15.0 + 30.0 * index)
465         points.append(np.array([q3.TARGET_RADIUS * math.cos(theta), q3.TARGET_RADIUS * math.sin(theta)
466     ]))
467     return points
468
469 def _backbone_supplement_candidates(self, route: list[P4Action]) -> list[np.ndarray]:
470     base_points = [np.asarray(action.point, dtype=float) for action in route if action.kind == 'BASE']
471     if not base_points:
472         return []
473     chain = [np.asarray(self.client.position, dtype=float)] + base_points
474     candidates: list[np.ndarray] = []
475     lateral_offset = 220.0
476     for (start, end) in zip(chain[:-1], chain[1:]):
477         segment = end - start
478         length = float(np.linalg.norm(segment))
479         if length < 320.0:
480             continue
481         midpoint = 0.5 * (start + end)
482         normal = np.array([-segment[1], segment[0]], dtype=float) / length
483         for sign in (-1.0, 1.0):
484             point = midpoint + sign * lateral_offset * normal
485             norm = float(np.linalg.norm(point))
486             if norm > 2100.0:
487                 point = point * (2100.0 / norm)
488             candidates.append(point)
489     candidates.extend(self.supplement_points)
490     return q3.deduplicate_points(candidates, tol=1e-05)
491
492 def _opposite_probe_candidates(self, channels: list[int]) -> list[np.ndarray]:
493     points: list[np.ndarray] = []
494     for channel in channels:
495         state = self.channels[channel]
496         last_no_signal: Optional[np.ndarray] = None
497         for obs in reversed(state.observations):
498             if obs.get('result') == 'no_signal':
499                 last_no_signal = np.asarray(obs['position'], dtype=float)
500                 break
501         if last_no_signal is None:
502             continue
503         center = self.beliefs[channel].position_center()
504         point = 2.0 * center - last_no_signal
505         radius = float(np.linalg.norm(point))
506         if radius > 2100.0:
507             point = point * (2100.0 / radius)
508         points.append(point)

```

```

507         return q3.deduplicate_points(points, tol=1e-05)
508
509     def _post_supplement_candidates(self, pending: list[int]) -> list[np.ndarray]:
510         points = list(self.supplement_points)
511         points.extend(self._opposite_probe_candidates(pending))
512         return q3.deduplicate_points(points, tol=1e-05)
513
514     def _emergency_supplement_candidates(self, pending: list[int]) -> list[np.ndarray]:
515         points = self._make_emergency_points()
516         points.extend(self._opposite_probe_candidates(pending))
517         return q3.deduplicate_points(points, tol=1e-05)
518
519     def estimated_total(self) -> float:
520         discovered = float(self.discovered_count())
521         unseen = sum((self.beliefs[ch].present_probability for (ch, state) in self.channels.items() if
522 state.status in (q3.SEARCHING, SEARCH_DEFERRED)))
523         return min(16.0, max(10.0, discovered + unseen, float(self.cleared_count)))
524
525     def deferred_channels(self) -> list[int]:
526         return [ch for (ch, state) in self.channels.items() if state.status == DEFERRED]
527
528     def search_deferred_channels(self) -> list[int]:
529         return [ch for (ch, state) in self.channels.items() if state.status == SEARCH_DEFERRED]
530
531     def discovery_coverage(self, channel: int) -> float:
532         return 1.0 - self.discovery_survival_probability[channel]
533
534     def corrected_presence_probabilities(self) -> dict[int, float]:
535         unseen = self._unseen_channels()
536         corrected = {ch: float(self.beliefs[ch].present_probability) for ch in unseen}
537         required = max(0.0, float(MIN_REQUIRED_DISCOVERIES - self.discovered_count()))
538         current = sum(corrected.values())
539         if required <= current + 1e-12 or not unseen:
540             return corrected
541         (lo, hi) = (0.0, 1.0)
542         for _ in range(60):
543             tau = 0.5 * (lo + hi)
544             total = sum((max(corrected[ch], tau) for ch in unseen))
545             if total < required:
546                 lo = tau
547             else:
548                 hi = tau
549         tau = hi
550         return {ch: max(corrected[ch], tau) for ch in unseen}
551
552     def expected_unseen_sources(self) -> float:
553         return float(sum(self.corrected_presence_probabilities().values()))
554
555     def _bearing_sector_count(self, channel: int) -> int:
556         state = self.channels[channel]
557         if not state.measured_positions:
558             return 0
559         center = self.beliefs[channel].position_center()
560         occupied: set[int] = set()
561         for raw in state.measured_positions:
562             point = np.asarray(raw, dtype=float)
563             delta = point - center
564             if float(np.linalg.norm(delta)) < 1e-06:
565                 continue
566             angle = (math.degrees(math.atan2(delta[1], delta[0])) + 360.0) % 360.0
567             sector = int(math.floor(angle / (360.0 / BEARING_SECTOR_COUNT)))

```

```

567         occupied.add(min(BEARING_SECTOR_COUNT - 1, sector))
568     return len(occupied)
569
570 def _required_coverage(self, channel: int, corrected_presence: float) -> float:
571     if corrected_presence >= HIGH_RISK_PRESENCE_THRESHOLD:
572         return HIGH_RISK_COVERAGE_TARGET
573     return DISCOVERY_COVERAGE_TARGET
574
575 def _channel_search_satisfied(self, channel: int, corrected_presence: float) -> bool:
576     coverage_ok = self.discovery_coverage(channel) >= self._required_coverage(channel,
577     corrected_presence) - 1e-09
578     sectors_ok = corrected_presence < 0.04 or self._bearing_sector_count(channel) >=
579     MIN_DISTINCT_BEARING_SECTORS
580     return coverage_ok and sectors_ok
581
582 def _record_discovery_outcome(self, channel: int, position: np.ndarray, result: str) -> None:
583     probability = self.beliefs[channel].directional_signal_probability(position)
584     if result == 'no_signal':
585         survival = self.discovery_survival_probability[channel]
586         self.discovery_survival_probability[channel] = min(1.0, max(0.0, survival * (1.0 - probability)
587     ))
588     elif result in ('direction', 'near'):
589         self.discovery_survival_probability[channel] = 0.0
590     print(f'[coverage-dir] ch={channel:02d} conditional_p={probability:.3f} cumulative={self.
591     discovery_coverage(channel):.3f}')
592
593 def done(self) -> bool:
594     if self.cleared_count >= 16:
595         return True
596     if self.remaining_base_ids:
597         return False
598     if self.clearable_channels():
599         return False
600     if self.localizing_channels():
601         return False
602     if not self.exploration_exhausted and self._unseen_channels():
603         return False
604     return True
605
606 def mark_search_complete_by_upper_bound(self) -> None:
607     if self.discovered_count() >= 16:
608         self.remaining_base_ids.clear()
609         self.exploration_exhausted = True
610         for state in self.channels.values():
611             if state.status in (q3.SEARCHING, SEARCH_DEFERRED):
612                 state.status = q3.ABSENT
613
614 def _rebuild_region(self, state: q3.ChannelState, reason: str) -> None:
615     self._set_region(state, mixed_position_region(state.observations), reason)
616
617 def _latest_observation(self, state: q3.ChannelState) -> dict[str, Any]:
618     if not state.observations:
619         raise RuntimeError('缺少刚记录的观测')
620     return state.observations[-1]
621
622 def _update_belief_after_measurement(self, state: q3.ChannelState, positive: bool) -> None:
623     belief = self.beliefs[state.channel]
624     if positive:
625         belief.present_probability = 1.0
626         latest = self._latest_observation(state)
627         positive_count = sum((obs['result'] in ('direction', 'near') for obs in state.observations))

```

```

624         if latest['result'] == 'near':
625             pass
626         elif positive_count == 1:
627             belief.rebuild(state.region, state.observations)
628         else:
629             belief.observe(latest, known_present=True)
630     else:
631         belief.observe(self._latest_observation(state), known_present=state.ever_detected)
632     print(f'[belief] ch={state.channel:02d} present={belief.present_probability:.3f} directional={
belief.directional_probability:.3f} phi={belief.direction_estimate_deg()}')
633
634 def handle_measure_response(self, channel: int, position: np.ndarray, response: dict[str, Any]) -> None
:
635     state = self.channels[channel]
636     result = response['measure_result']
637     was_undiscovered = not state.ever_detected
638     if was_undiscovered and result in ('no_signal', 'direction', 'near'):
639         self._record_discovery_outcome(channel, position, result)
640     if result == 'no_signal':
641         self._record_measurement(state, position, 'no_signal')
642         state.region_version += 1
643         state.invalidate_plan_cache()
644         self._update_belief_after_measurement(state, positive=False)
645     elif result == 'direction':
646         angle = float(response['svd_deg'])
647         self._record_measurement(state, position, 'direction', angle)
648         self._mark_detected(state)
649         self._rebuild_region(state, 'direction')
650         self._update_belief_after_measurement(state, positive=True)
651     elif result == 'near':
652         self._record_measurement(state, position, 'near')
653         self._mark_detected(state)
654         self._rebuild_region(state, 'near')
655         self._update_belief_after_measurement(state, positive=True)
656         clear_response = self.client.clear(position, channel)
657         self.handle_clear_response(channel, position, clear_response)
658     else:
659         raise RuntimeError(f'未知 measure_result={result!r}')
660     self.mark_search_complete_by_upper_bound()
661
662 def handle_clear_response(self, channel: int, position: np.ndarray, response: dict[str, Any]) -> None:
663     state = self.channels[channel]
664     result = response['clear_result']
665     if result == 'success':
666         if state.status != q3.CLEARED:
667             self.cleared_count += 1
668             state.status = q3.CLEARED
669             state.ever_detected = True
670             self.discovered_channels.add(channel)
671             print(f'[state] ch={channel:02d} CLEARED ({self.cleared_count} total)')
672         elif result == 'no_target_in_range':
673             state.observations.append({'position': [float(position[0]), float(position[1])], 'result': '
clear_failure'})
674             self._rebuild_region(state, 'clear_failure')
675             state.status = q3.LOCALIZING
676             self.beliefs[channel].rebuild(state.region, state.observations)
677             print(f'[state] ch={channel:02d} clear failed -> LOCALIZING')
678         else:
679             raise RuntimeError(f'未知 clear_result={result!r}')
680     self.mark_search_complete_by_upper_bound()
681

```

```

682 def _cached_q2_options(self, state: q3.ChannelState) -> list[q3.ActiveOption]:
683     if state.active_cache_version == state.region_version:
684         return state.active_cache
685     try:
686         options = q3.compute_active_options(region=state.region, observations=state.observations,
        measured_positions=state.measured_positions)
687     except (ValueError, RuntimeError) as exc:
688         print(f'[q2-p4] ch={state.channel:02d} no option: {exc}')
689         options = []
690     state.active_cache_version = state.region_version
691     state.active_cache = options
692     return options
693
694 def _directional_candidates(self, channel: int) -> list[np.ndarray]:
695     state = self.channels[channel]
696     belief = self.beliefs[channel]
697     points: list[np.ndarray] = []
698     protected_q2: list[np.ndarray] = []
699     options = self._cached_q2_options(state)
700     if options:
701         threshold = options[0].score * (1.0 + ACTIVE_Q2_NEAR_OPT_REL) + 1e-09
702         near_optimal = [option for option in options if option.score <= threshold]
703         protected_q2 = [option.point for option in near_optimal[:8]]
704         points.extend(protected_q2)
705     center = state.mec_center.copy() if state.mec_center is not None else belief.position_center()
706     points.append(center)
707     phi = belief.direction_estimate_deg()
708     angles: list[float]
709     if phi is None or belief.directional_probability < 0.58:
710         angles = [float(v) for v in range(0, 360, 45)]
711     else:
712         angles = [phi + value for value in (0, -75, 75, 150, 180, 210)]
713     for radius in (320.0, 620.0, 900.0):
714         for angle in angles:
715             theta = math.radians(angle)
716             points.append(center + radius * np.array([math.cos(theta), math.sin(theta)]))
717     points.extend((self.base_points[index] for index in self.remaining_base_ids))
718     points.append(self.client.position.copy())
719     unique = q3.deduplicate_points(points, tol=0.0001)
720     unique = [point for point in unique if not q3.measured_before(state.measured_positions, point) and
    np.all(np.abs(point) <= 2000000.0)]
721     protected_q2 = [point for point in q3.deduplicate_points(protected_q2, tol=0.0001) if not q3.
    measured_before(state.measured_positions, point)]
722     other = [point for point in unique if not any((q3.dist(point, q2_point) < 0.0001 for q2_point in
    protected_q2))]
723     other.sort(key=lambda point: q3.dist(self.client.position, point))
724     return (protected_q2 + other)[:MAX_DIRECTIONAL_CANDIDATES]
725
726 def _evaluate_candidate(self, channel: int, point: np.ndarray, route: list[P4Action]) ->
    CandidateEvaluation:
727     state = self.channels[channel]
728     belief = self.beliefs[channel]
729     prediction = belief.predict(point)
730     switch = 0.0 if self.client.current_channel == channel else 1.0
731     (insertion_distance, _) = route_insertion(self.client.position, route, point)
732     action_time = insertion_distance / q3.DOG_SPEED + 5.0 + switch
733     (p_clear, remaining) = estimate_followup(state, prediction, action_time)
734     delta = expected_delta_utility(self.cleared_count, self.client.virtual_time, self.estimated_total()
    , p_clear, remaining)
735     information_value = INFORMATION_REWARD * (0.55 * prediction.information_gain + 0.45 * prediction.
    spatial_gain)

```

```

736         value = max(0.0, delta) + information_value
737         return CandidateEvaluation(channel=channel, point=np.asarray(point, dtype=float).copy(), prediction
=prediction, action_time_s=action_time, expected_clear_probability=p_clear, expected_remaining_cost_s=
remaining, delta_utility=delta, value_per_second=value / max(action_time, 1e-09), insertion_distance_m
=insertion_distance)

738
739     def _best_candidate(self, channel: int, route: list[P4Action]) -> Optional[CandidateEvaluation]:
740         candidates = self._directional_candidates(channel)
741         if not candidates:
742             return None
743         evaluations = [self._evaluate_candidate(channel, point, route) for point in candidates]
744         return max(evaluations, key=lambda item: (item.value_per_second, item.prediction.information_gain,
item.prediction.signal_probability, -item.action_time_s))

745
746     def _measurement_count(self, channel: int) -> int:
747         return sum((obs['result'] in ('direction', 'no_signal', 'near') for obs in self.channels[channel].
observations))

748
749     def _channel_center(self, channel: int) -> np.ndarray:
750         state = self.channels[channel]
751         if state.mec_center is not None:
752             return state.mec_center.copy()
753         return self.beliefs[channel].position_center()

754
755     def _cheap_channel_priority(self, channel: int, route: list[P4Action]) -> float:
756         state = self.channels[channel]
757         center = self._channel_center(channel)
758         (detour, _) = route_insertion(self.client.position, route, center)
759         radius_term = min(1.0, max(0.0, state.mec_radius) / q3.TARGET_RADIUS)
760         uncertainty = self.beliefs[channel].directional_probability + radius_term
761         age_bonus = 0.1 * min(self.exact_age[channel], 6)
762         return (uncertainty + age_bonus) / (1.0 + detour / 1000.0)

763
764     def _set_deferred(self, channel: int, reason: str) -> None:
765         self.channels[channel].status = DEFERRED
766         self.deferred_reasons[channel] = reason
767         print(f'[defer] ch={channel:02d} {reason}')
768
769     def _defer_or_reactivate(self, route: list[P4Action], planning_deadline: float) -> dict[int,
CandidateEvaluation]:
770         best: dict[int, CandidateEvaluation] = {}
771         for channel in self.deferred_channels():
772             if time.monotonic() >= planning_deadline:
773                 break
774             state = self.channels[channel]
775             if not state.ever_detected or self.reactivation_count[channel] >= 1:
776                 continue
777             (center_detour, _) = route_insertion(self.client.position, route, self._channel_center(channel)
)

778             if center_detour > REACTIVATE_DISTANCE_M:
779                 continue
780             evaluation = self._best_candidate(channel, route)
781             if evaluation is None:
782                 continue
783             if evaluation.delta_utility > 0.0 and evaluation.insertion_distance_m <= REACTIVATE_DISTANCE_M:
784                 state.status = q3.LOCALIZING
785                 self.reactivation_count[channel] += 1
786                 self.active_attempts[channel] = min(self.active_attempts[channel],
MAX_ACTIVE_AFTER_DISCOVERY - 1)
787                 self.stagnant_active_steps[channel] = 0
788                 self.deferred_reasons.pop(channel, None)

```

```

789         best[channel] = evaluation
790         print(f'[reactivate] ch={channel:02d} deltaU={evaluation.delta_utility:.5f} detour={
evaluation.insertion_distance_m:.1f}m')
791     localizing = list(self.localizing_channels())
792     for channel in localizing:
793         self.exact_age[channel] += 1
794         if self.active_attempts[channel] >= MAX_ACTIVE_AFTER_DISCOVERY:
795             self._set_deferred(channel, 'active_measurement_budget_exhausted')
796     eligible = [channel for channel in localizing if self.channels[channel].status == q3.LOCALIZING]
797     eligible.sort(key=lambda channel: self._cheap_channel_priority(channel, route), reverse=True)
798     for channel in eligible[:EXACT_CHANNELS_PER_ROUND]:
799         if time.monotonic() >= planning_deadline:
800             print('[plan-budget] stop exact evaluation at soft deadline')
801             break
802         self.exact_age[channel] = 0
803         evaluation = self._best_candidate(channel, route)
804         if evaluation is None:
805             if self._measurement_count(channel) >= MIN_MEASUREMENTS_BEFORE_DEFER:
806                 self._set_deferred(channel, 'no_candidate')
807                 continue
808         hard_detour = self.cleared_count >= 14 and evaluation.insertion_distance_m >
HARD_INSERTION_LIMIT_M
809         if hard_detour:
810             self._set_deferred(channel, f'hard_detour={evaluation.insertion_distance_m:.1f}m')
811             continue
812         enough_trials = self._measurement_count(channel) >= MIN_MEASUREMENTS_BEFORE_DEFER
813         low_information = evaluation.prediction.information_gain < MIN_INFORMATION_GAIN and evaluation.
prediction.spatial_gain < MIN_INFORMATION_GAIN
814         soft_stop = self.cleared_count >= 12 and enough_trials and (evaluation.delta_utility < 0.0) and
(evaluation.insertion_distance_m > SOFT_INSERTION_LIMIT_M) and (low_information or evaluation.
value_per_second < MIN_ACTIVE_VALUE_RATE)
815         if soft_stop:
816             self._set_deferred(channel, f'deltaU={evaluation.delta_utility:.6f}, detour={evaluation.
insertion_distance_m:.1f}m')
817             continue
818         best[channel] = evaluation
819     return best
820
821 def _base_route_actions(self) -> list[P4Action]:
822     route_ids = self._remaining_base_route()
823     unseen = self.searching_channels()
824     actions: list[P4Action] = []
825     previous = self.client.position
826     for base_id in route_ids:
827         point = self.base_points[base_id]
828         expected_discoveries = sum((self.beliefs[ch].present_probability * self.beliefs[ch].
signal_probability(point) for ch in unseen))
829         travel = q3.dist(previous, point) / q3.DOG_SPEED
830         service = 5.0 * len(unseen) + max(0, len(unseen) - 1)
831         score = (DISCOVERY_REWARD * expected_discoveries + BASE_PROGRESS_BONUS) / max(travel + service,
1.0)
832         actions.append(P4Action(kind='BASE', point=point.copy(), base_id=base_id, score=score, channels
=tuple(unseen), note=f'E[new]={expected_discoveries:.3f}'))
833         previous = point
834     return actions
835
836 def _clear_actions(self, route: list[P4Action]) -> list[P4Action]:
837     actions: list[P4Action] = []
838     for channel in self.clearable_channels():
839         state = self.channels[channel]
840         state.recompute_geometry()

```

```

841         if state.mec_center is None or not guarantees_clearance(state, state.mec_center):
842             continue
843         (insertion_distance, _) = route_insertion(self.client.position, route, state.mec_center)
844         cost = insertion_distance / q3.DOG_SPEED + 5.0
845         delta = expected_delta_utility(self.cleared_count, self.client.virtual_time, self.
estimated_total(), 1.0, cost)
846         score = (CLEAR_ACTION_BONUS + max(0.0, delta)) / max(cost, 1.0)
847         actions.append(P4Action(kind='CLEAR', point=state.mec_center.copy(), channel=channel, score=
score, note=f'certified MEC={state.mec_radius:.3f}m, detour={insertion_distance:.1f}m',
insertion_distance_m=insertion_distance, estimated_time_s=cost))
848         return actions
849
850 def _active_actions(self, best: dict[int, CandidateEvaluation]) -> list[P4Action]:
851     actions: list[P4Action] = []
852     for (channel, evaluation) in best.items():
853         if self.channels[channel].status != q3.LOCALIZING:
854             continue
855         actions.append(P4Action(kind='ACTIVE', point=evaluation.point, channel=channel, channels=(
channel,), score=evaluation.value_per_second, note=f'pvis={evaluation.prediction.signal_probability:.3
f}, IG={evaluation.prediction.information_gain:.3f}, deltaU={evaluation.delta_utility:.5f}, detour={
evaluation.insertion_distance_m:.1f}m', insertion_distance_m=evaluation.insertion_distance_m,
estimated_time_s=evaluation.action_time_s))
856         return actions
857
858 def _unseen_channels(self) -> list[int]:
859     return [channel for (channel, state) in self.channels.items() if state.status in (q3.SEARCHING,
SEARCH_DEFERRED)]
860
861 def _coverage_pending_channels(self) -> list[int]:
862     unseen = self._unseen_channels()
863     if not unseen:
864         return []
865     corrected = self.corrected_presence_probabilities()
866     if self.discovered_count() < MIN_REQUIRED_DISCOVERIES:
867         return unseen
868     expected_unseen = sum(corrected.values())
869     pending = [channel for channel in unseen if not self._channel_search_satisfied(channel, corrected[
channel])]
870     if expected_unseen >= EXPECTED_UNSEEN_STOP_THRESHOLD and (not pending):
871         ranked = sorted(unseen, key=lambda ch: corrected[ch], reverse=True)
872         running = 0.0
873         for channel in ranked:
874             if corrected[channel] <= 0.01:
875                 continue
876             pending.append(channel)
877             running += corrected[channel]
878             if running >= min(expected_unseen, 0.6):
879                 break
880     return pending
881
882 def _finish_supplement_search(self, reason: str) -> None:
883     if self.remaining_base_ids:
884         return
885     self.exploration_exhausted = True
886     self._defer_unseen(reason)
887
888 def _supplement_action(self, route: list[P4Action], phase: str, may_exhaust: bool) -> Optional[P4Action
]:
889     if phase not in ('backbone', 'post', 'emergency'):
890         raise ValueError(f'未知补盲阶段 {phase!r}')
891     if phase == 'backbone':

```

```

892         if self.backbone_probe_actions_done >= MAX_BACKBONE_PROBE_ACTIONS:
893             return None
894         insertion_limit = BACKBONE_PROBE_INSERTION_LIMIT_M
895     elif phase == 'post':
896         if self.post_backbone_probe_actions_done >= MAX_POST_BACKBONE_PROBE_ACTIONS:
897             return None
898         insertion_limit = POST_PROBE_INSERTION_LIMIT_M
899     else:
900         if self.emergency_probe_actions_done >= MAX_EMERGENCY_PROBE_ACTIONS:
901             if may_exhaust:
902                 self._finish_supplement_search('emergency_probe_count_limit')
903             return None
904         insertion_limit = EMERGENCY_PROBE_INSERTION_LIMIT_M
905         budget_exhausted = self.supplement_actions_done >= MAX_SUPPLEMENT_ACTIONS or self.
supplement_virtual_spent_s >= SUPPLEMENT_TOTAL_BUDGET_S or self.supplement_detour_spent_m >=
SUPPLEMENT_DETOUR_BUDGET_M
906         if budget_exhausted:
907             if phase != 'backbone' and may_exhaust:
908                 self._finish_supplement_search('supplement_budget_exhausted')
909             return None
910         if self.exploration_exhausted:
911             return None
912         pending = self._coverage_pending_channels()
913         if not pending:
914             if phase != 'backbone':
915                 self._finish_supplement_search('aggressive_search_criteria_reached')
916             return None
917         corrected = self.corrected_presence_probabilities()
918         remaining_time_budget = SUPPLEMENT_TOTAL_BUDGET_S - self.supplement_virtual_spent_s
919         remaining_detour_budget = SUPPLEMENT_DETOUR_BUDGET_M - self.supplement_detour_spent_m
920         if phase == 'backbone':
921             candidate_points = self._backbone_supplement_candidates(route)
922         elif phase == 'post':
923             candidate_points = self._post_supplement_candidates(pending)
924         else:
925             candidate_points = self._emergency_supplement_candidates(pending)
926         best: Optional[P4Action] = None
927         for point in candidate_points:
928             (insertion_distance, _) = route_insertion(self.client.position, route, point)
929             if insertion_distance > min(insertion_limit, remaining_detour_budget):
930                 continue
931             channel_values: list[tuple[float, float, float, int]] = []
932             for channel in pending:
933                 state = self.channels[channel]
934                 if q3.measured_before(state.measured_positions, point):
935                     continue
936                 belief = self.beliefs[channel]
937                 presence = max(corrected.get(channel, belief.present_probability), UNSEEN_PRESENCE_FLOOR)
938                 marginal_coverage = self.discovery_survival_probability[channel] * belief.
directional_signal_probability(point)
939                 if marginal_coverage <= 1e-10:
940                     continue
941                 sectors = self._bearing_sector_count(channel)
942                 fairness = 1.0 + 0.18 * max(0, MIN_DISTINCT_BEARING_SECTORS - sectors)
943                 if presence >= HIGH_RISK_PRESENCE_THRESHOLD:
944                     fairness *= 1.18
945                 expected_discover_and_clear = presence * marginal_coverage * SUPPLEMENT_CLEAR_PROBABILITY
946                 channel_values.append((expected_discover_and_clear * fairness, marginal_coverage, presence,
channel))
947             channel_values.sort(reverse=True)
948             chosen = [channel for (_, marginal, _, channel) in channel_values[:MAX_CHANNELS_PER_SUPPLEMENT]]

```

```

    if marginal > 1e-08]
949         if not chosen:
950             continue
951         expected_by_channel = {channel: expected for (expected, _, _, channel) in channel_values}
952         marginal_by_channel = {channel: marginal for (_, marginal, _, channel) in channel_values}
953         expected = sum((expected_by_channel[ch] for ch in chosen))
954         total_coverage_gain = sum((marginal_by_channel[ch] for ch in chosen))
955         switches = len(chosen) - (1 if self.client.current_channel in chosen else 0)
956         service = 5.0 * len(chosen) + float(switches)
957         projected_time = insertion_distance / q3.DOG_SPEED + service
958         if service > SUPPLEMENT_SENSE_BUDGET_S + 1e-09 or projected_time > remaining_time_budget + 1e
-09:
959             continue
960         phase_bonus = 1.12 if phase == 'emergency' else 1.0
961         score = phase_bonus * DISCOVERY_REWARD * (expected + 0.35 * total_coverage_gain) / max(math.
sqrt(projected_time), 1.0)
962         projected_coverages = [min(1.0, self.discovery_coverage(ch) + marginal_by_channel[ch]) for ch
in chosen]
963         action = P4Action(kind='PROBE', point=np.asarray(point, dtype=float).copy(), channels=tuple(
chosen), score=score, note=f'phase={phase}, marginal_coverage={total_coverage_gain:.3f}, projected_min
={min(projected_coverages):.3f}, expected_unseen={self.expected_unseen_sources():.3f}, detour={
insertion_distance:.1f}m', insertion_distance_m=insertion_distance, estimated_time_s=projected_time,
probe_phase=phase)
964         if best is None or action.score > best.score:
965             best = action
966         if best is None or best.score < MIN_SUPPLEMENT_VALUE_RATE:
967             if phase == 'emergency' and may_exhaust:
968                 self._finish_supplement_search('no_positive_emergency_probe')
969             return None
970         return best
971
972 def _defer_unseen(self, reason: str) -> None:
973     for channel in self._unseen_channels():
974         self.channels[channel].status = SEARCH_DEFERRED
975         self.deferred_reasons[channel] = reason
976
977 def plan(self) -> list[P4Action]:
978     started = time.monotonic()
979     planning_deadline = started + PLAN_TIME_BUDGET_S
980     try:
981         self.mark_search_complete_by_upper_bound()
982         self.fast_finish_mode = self.client.real_time_remaining() <= FAST_FINISH_REMAINING_S and self.
discovered_count() >= MIN_REQUIRED_DISCOVERIES
983         for channel in self.localizing_channels():
984             self.channels[channel].recompute_geometry()
985         base_route = self._base_route_actions()
986         tasks = self._clear_actions(base_route)
987         if not self.fast_finish_mode:
988             best_by_channel = self._defer_or_reactivate(base_route, planning_deadline)
989             tasks.extend(self._active_actions(best_by_channel))
990         preliminary_route = insert_actions_by_minimum_increment(self.client.position, base_route, tasks
)
991
992         if not self.fast_finish_mode:
993             if base_route:
994                 phase = 'backbone'
995             else:
996                 post_exhausted = self.post_backbone_probe_actions_done >=
MAX_POST_BACKBONE_PROBE_ACTIONS
997                 still_risky = self.discovered_count() < MIN_REQUIRED_DISCOVERIES or self.
expected_unseen_sources() >= EMERGENCY_EXPECTED_UNSEEN_THRESHOLD or bool(self.
_coverage_pending_channels())

```

```

997         phase = 'emergency' if post_exhausted and still_risky else 'post'
998         supplement = self._supplement_action(preliminary_route, phase=phase, may_exhaust=phase == '
emergency' and (not preliminary_route))
999         if supplement is None and phase == 'post' and (not preliminary_route) and self.
_coverage_pending_channels():
1000             supplement = self._supplement_action(preliminary_route, phase='emergency', may_exhaust=
True)
1001             if supplement is not None:
1002                 tasks.append(supplement)
1003             route = insert_actions_by_minimum_increment(self.client.position, base_route, tasks)
1004             if self.fast_finish_mode and (not route):
1005                 for channel in list(self.localizing_channels()):
1006                     self._set_deferred(channel, 'fast_finish_no_safe_cached_action')
1007                     self.exploration_exhausted = True
1008                     self._defer_unseen('fast_finish')
1009             return route
1010         finally:
1011             elapsed = time.monotonic() - started
1012             self.plan_durations_s.append(elapsed)
1013             if elapsed > PLAN_TIME_BUDGET_S:
1014                 print(f'[plan-budget] soft limit exceeded: {elapsed:.3f}s > {PLAN_TIME_BUDGET_S:.1f}s')
1015
1016     def _opportunistic_at_point(self, point: np.ndarray) -> None:
1017         choices: list[tuple[float, int]] = []
1018         for channel in self.localizing_channels():
1019             state = self.channels[channel]
1020             if q3.measured_before(state.measured_positions, point):
1021                 continue
1022             prediction = self.beliefs[channel].predict(point)
1023             value = prediction.information_gain + prediction.spatial_gain
1024             if value >= MIN_INFORMATION_GAIN:
1025                 choices.append((value, channel))
1026             choices.sort(reverse=True)
1027             for (_, channel) in choices[:MAX_OPPORTUNISTIC_AT_BASE]:
1028                 response = self.client.measure(point, channel)
1029                 self.handle_measure_response(channel, point, response)
1030
1031     def visit_base(self, base_id: int) -> None:
1032         point = self.base_points[base_id]
1033         search_tasks = self._unseen_channels()
1034         print(f'[base-p4] visit B{base_id}, search={len(search_tasks)}')
1035         for channel in self._measurement_order(search_tasks):
1036             if self.discovered_count() >= 16:
1037                 break
1038             if self.channels[channel].status not in (q3.SEARCHING, SEARCH_DEFERRED):
1039                 continue
1040             self.channels[channel].status = q3.SEARCHING
1041             self.deferred_reasons.pop(channel, None)
1042             response = self.client.measure(point, channel)
1043             self.handle_measure_response(channel, point, response)
1044             self._opportunistic_at_point(point)
1045             self.remaining_base_ids.discard(base_id)
1046             self.visited_base_ids.add(base_id)
1047
1048     def execute_active(self, action: P4Action) -> None:
1049         if action.channel is None:
1050             return
1051         state = self.channels[action.channel]
1052         if state.status != q3.LOCALIZING:
1053             return
1054         if q3.measured_before(state.measured_positions, action.point):

```

```

1055         self._set_deferred(action.channel, 'planner_reselected_measured_point')
1056         return
1057     if self.active_attempts[action.channel] >= MAX_ACTIVE_AFTER_DISCOVERY:
1058         self._set_deferred(action.channel, 'active_measurement_budget_exhausted')
1059         return
1060     before_radius = state.mec_radius
1061     self.active_attempts[action.channel] += 1
1062     response = self.client.measure(action.point, action.channel)
1063     self.handle_measure_response(action.channel, action.point, response)
1064     state = self.channels[action.channel]
1065     if state.status == q3.LOCALIZING:
1066         after_radius = state.mec_radius
1067         if math.isfinite(before_radius) and before_radius > 1e-09 and math.isfinite(after_radius):
1068             reduction = max(0.0, (before_radius - after_radius) / before_radius)
1069         else:
1070             reduction = 1.0
1071         if reduction < MIN_MEC_REDUCTION_RATIO:
1072             self.stagnant_active_steps[action.channel] += 1
1073         else:
1074             self.stagnant_active_steps[action.channel] = 0
1075             self.last_active_mec_radius[action.channel] = after_radius
1076             if self.stagnant_active_steps[action.channel] >= MAX_STAGNANT_ACTIVE_STEPS:
1077                 self._set_deferred(action.channel, f'mec_stagnation={self.stagnant_active_steps[action.
channel]})')
1078             elif self.active_attempts[action.channel] >= MAX_ACTIVE_AFTER_DISCOVERY:
1079                 self._set_deferred(action.channel, 'active_measurement_budget_exhausted')
1080     self._opportunistic_at_point(action.point)
1081
1082 def execute_probe(self, action: P4Action) -> None:
1083     print(f'[probe] phase={action.probe_phase} channels={list(action.channels)} {action.note}')
1084     spent = 0.0
1085     measured_any = False
1086     for channel in self._measurement_order(action.channels):
1087         if self.channels[channel].status not in (q3.SEARCHING, SEARCH_DEFERRED):
1088             continue
1089         switch = 0.0 if self.client.current_channel == channel else 1.0
1090         if spent + switch + 5.0 > SUPPLEMENT_SENSE_BUDGET_S + 1e-09:
1091             break
1092         self.channels[channel].status = q3.SEARCHING
1093         self.deferred_reasons.pop(channel, None)
1094         response = self.client.measure(action.point, channel)
1095         self.handle_measure_response(channel, action.point, response)
1096         spent += switch + 5.0
1097         measured_any = True
1098     if measured_any:
1099         self.supplement_actions_done += 1
1100         if action.probe_phase == 'backbone':
1101             self.backbone_probe_actions_done += 1
1102         elif action.probe_phase == 'post':
1103             self.post_backbone_probe_actions_done += 1
1104         elif action.probe_phase == 'emergency':
1105             self.emergency_probe_actions_done += 1
1106         self.supplement_virtual_spent_s += action.estimated_time_s
1107         self.supplement_detour_spent_m += action.insertion_distance_m
1108     if not self.remaining_base_ids:
1109         if not self._coverage_pending_channels():
1110             self._finish_supplement_search('aggressive_search_criteria_reached')
1111         elif self.supplement_actions_done >= MAX_SUPPLEMENT_ACTIONS or self.
emergency_probe_actions_done >= MAX_EMERGENCY_PROBE_ACTIONS:
1112             self._finish_supplement_search('aggressive_probe_budget_exhausted')
1113

```

```

1114 def execute_clear(self, action: P4Action) -> None:
1115     if action.channel is None:
1116         return
1117     state = self.channels[action.channel]
1118     if state.status != q3.CLEARABLE:
1119         return
1120     point = action.point
1121     if not guarantees_clearance(state, point):
1122         if state.mec_center is None or not guarantees_clearance(state, state.mec_center):
1123             raise RuntimeError(f'频道 {action.channel} 没有经过最远距离验证的清除点')
1124         point = state.mec_center
1125     response = self.client.clear(point, action.channel)
1126     self.handle_clear_response(action.channel, point, response)
1127
1128 def execute_node(self, action: P4Action) -> None:
1129     if action.kind == 'BASE':
1130         if action.base_id is None:
1131             raise ValueError('BASE 缺少 base_id')
1132         self.visit_base(action.base_id)
1133     elif action.kind == 'ACTIVE':
1134         self.execute_active(action)
1135     elif action.kind == 'CLEAR':
1136         self.execute_clear(action)
1137     elif action.kind == 'PROBE':
1138         self.execute_probe(action)
1139     else:
1140         raise ValueError(f'未知动作类型 {action.kind!r}')

```